Program to take input of various datatypes in C

studytonight.com/c/programs/basic/different-data-type-input

Below is a program to explain how to take input from user for different datatypes available in C language. The different datatypes are `int` (integer values), `float` (decimal values) and `char` (character values).

Here is the C language tutorial explaining various datatypes → Datatypes in C

`printf()` is used to display text onto the screen

`&` is used to assign the input value to the variable and store it at that particular location.

`scanf()` is uded to take input from the user using format specifier discussed in upcoming tutorials

`%d` and `%i`, both are used to take numbers as input from the user.

`%f` is the format specifier to take float as input from the user

`%s` is the format specifier to take character as input from the user

```c
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int num1, num2;
    float fraction;
    char character;

    printf("Enter two numbers number\n");

    // Taking integer as input from user
    scanf("%d%i", &num1, &num2);
    printf("\n\nThe two numbers You have entered are %d and %i\n\n", num1, num2);

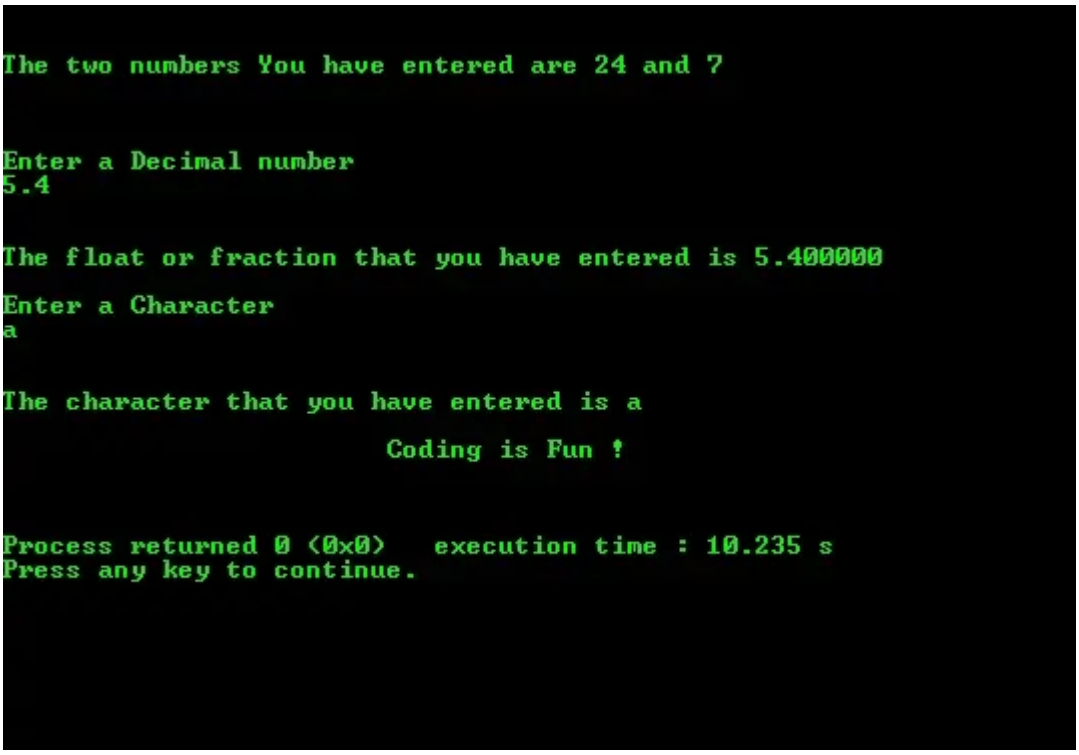
    // Taking float or fraction as input from the user
    printf("\n\nEnter a Decimal number\n");
    scanf("%f", &fraction);
    printf("\n\nThe float or fraction that you have entered is %f", fraction);

    // Taking Character as input from the user
    printf("\n\nEnter a Character\n");
    scanf("%c",&character);
    printf("\n\nThe character that you have entered is %c", character);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");

    return 0;
}
```

Output:


The two numbers You have entered are 24 and 7
Enter a Decimal number
5.4
The float or fraction that you have entered is 5.400000
Enter a Character
a
The character that you have entered is a
Coding is Fun !
Process returned 0 (0x0) execution time : 10.235 s
Press any key to continue.

ASCII value of Character

studytonight.com/c/programs/basic/ascii-value-of-character

Below is a program to find ASCII value of any input character.

`%c` is the format specifier to take character as input

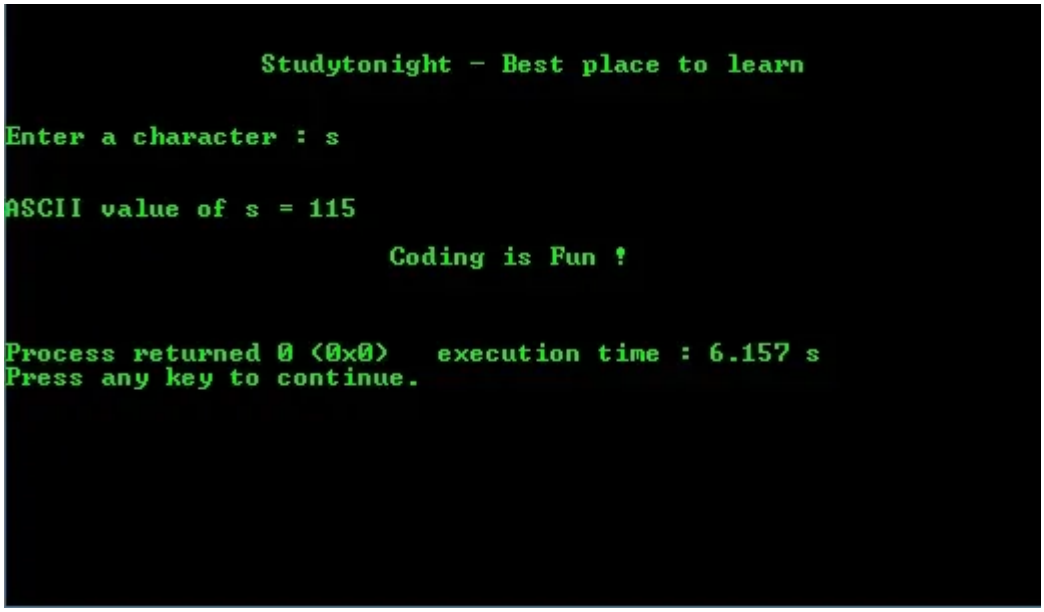
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char c;
    printf("Enter a character : ");
    scanf("%c" , &c);
    printf("\n\nASCII value of %c = %d",c,c);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



How to use gets() function

studytonight.com/c/programs/basic/use-of-gets

Some of the important points about `scanf()` and `gets()` are:

- `scanf()` and `gets()` both are used to take input from the user.
- `scanf()` can only take input until it encounters a space. The words after space are ignored by it.
- `gets()` is used to take a single input at a time but can be used to input a complete sentence with spaces unlike scanf().

Below is a program on use of `gets()`.

`gets()` takes only a single line at a time i.e all the words before hitting \n(enter key).

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char str[50];   // char array of size 50
    printf("Enter your complete name:\n\n\n");

    gets(str);
    printf("\n\nWelcome to Studytonight  %s\n\n\n", str);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Basic ifelse condition program

studytonight.com/c/programs/basic/if-else

Below is a program on `if` - `else` .

Here is the C language tutorial explaining If Else → If Else in C

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int number;
    printf("Please enter a number:\n");
    scanf("%d",&number);
    /*
        For single statements we can skip the curly brackets
    */
    if(number < 100)
        printf("Number is less than 100!\n");
    else if(number == 100)
        printf("Number is 100!\n");
    else
        printf("Number is greater than 100!\n");

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");

    return 0;
}
```

Output:



Programs of Switch Case Statement in C Language

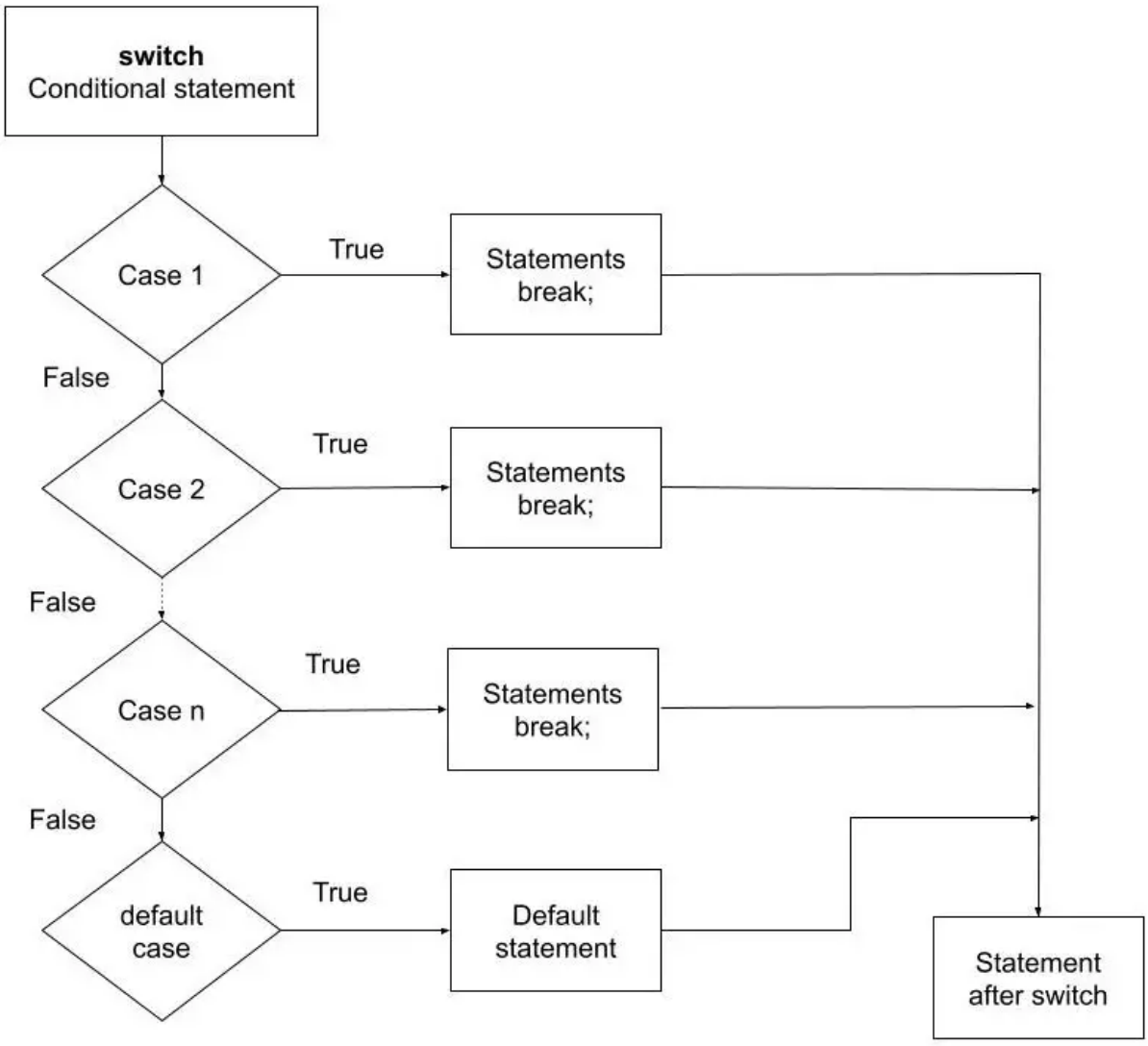
studytonight.com/c/programs/basic/switch-case

We use decision making statements in C to control the order of execution of our program. Switch case in C is one the decision making statements which is mostly used when we have multiple alternatives to choose from.

The syntax of switch statement is:

```
switch(expression)
{
    case value1:
        statement_1;
        break;
    case value2:
        statement_2;
        break;
    //we can have as many cases as we want
    case value_n:
        statement_n;
        break;
    default:
        default statement;    //this is not necessary. It is used only for
convenience
}
```

Following is a simple to help you understand how a switch statement works:



The algorithm of switch statement is as follows:

1. Firstly, the expression inside the **switch(expression)** is evaluated.
2. Then, it is matched with the case value of each statement.
3. If it matches with one of the case values, we execute that particular set of statements with which it has matched. If the `break` statement is used after that case, we break out of switch otherwise keep executing till we reach the end of switch(because there is no break statement) or we reach another break statement.
4. If it does not match, we execute the default condition(if present) and come out of switch.

---

1. Program of switch case with `break` in C Language

Below is a program on switch case with break.

Here is the C language tutorial explaining Switch Case → Switch Case in C

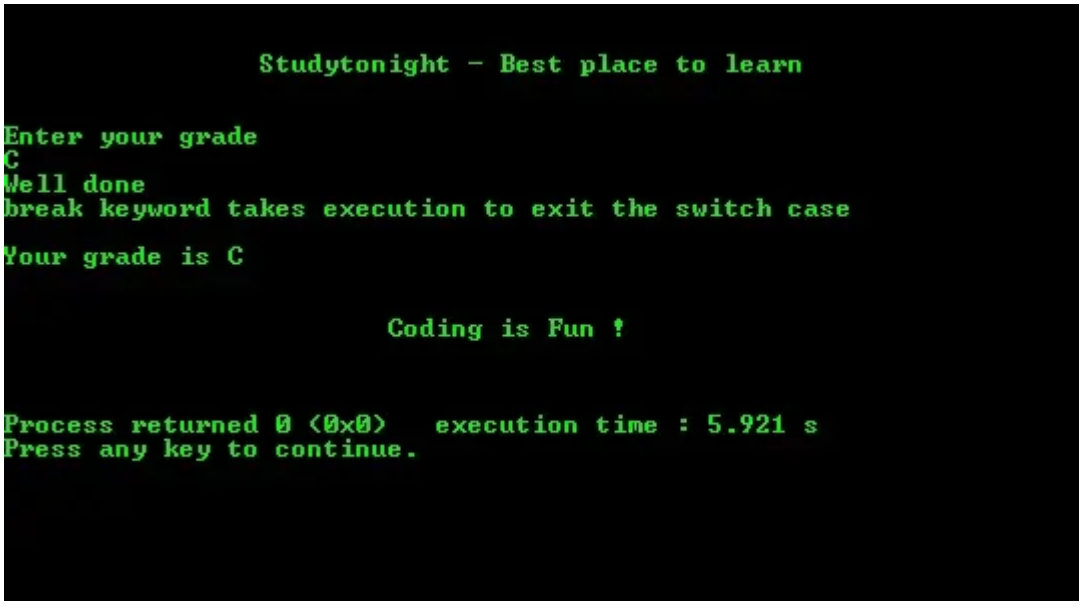
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    // Local Variable Definition
    char grade;
    printf("Enter your grade:\n");
    scanf("%c", &grade);

    switch(grade)
    {
        case 'A':
            printf("Excellent\n");
            break;
        case 'B':
            printf("Keep it up!\n\n");
            break;
        case 'C':
            printf("Well done\nbreak keyword takes execution to exit the switch
case\n\n");
            break;
        case 'D':
            printf("You passed\n");
            break;
        case 'F':
            printf("Better luck next time\n");
            break;
        default:
            printf("Invalid grade\n");
    }
    printf("Your grade is %c\n",grade);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



In the example above,**grade** is called the **check condition** and 'A', 'B', 'C', 'D' and 'F' are the **case values**.

Dry run of the above example:

We input a character in grade variable, here, 'C'. Next, we enter the switch statement. Since grade is a char type, it is valid. Now, we check the value of grade against all case values starting with the first one. 'A' is not equal to 'C' so the statements corresponding to 'A' will not be executed. We check the next one. 'B' is not equal to 'C' so the statements corresponding to 'B' will also not be executed. We check the next one. 'C' is equal to 'C' so we execute the statements corresponding to 'C'. We print "Well done break keyword takes execution to exit the switch case" and then execute the break statement which takes us out of the switch case.

---

2. Program of Switch Case without `break` in C

If there is no `break` statement then the cases after the matched case, other than default will all get executed.

Below is a program on switch case without `break` statement.

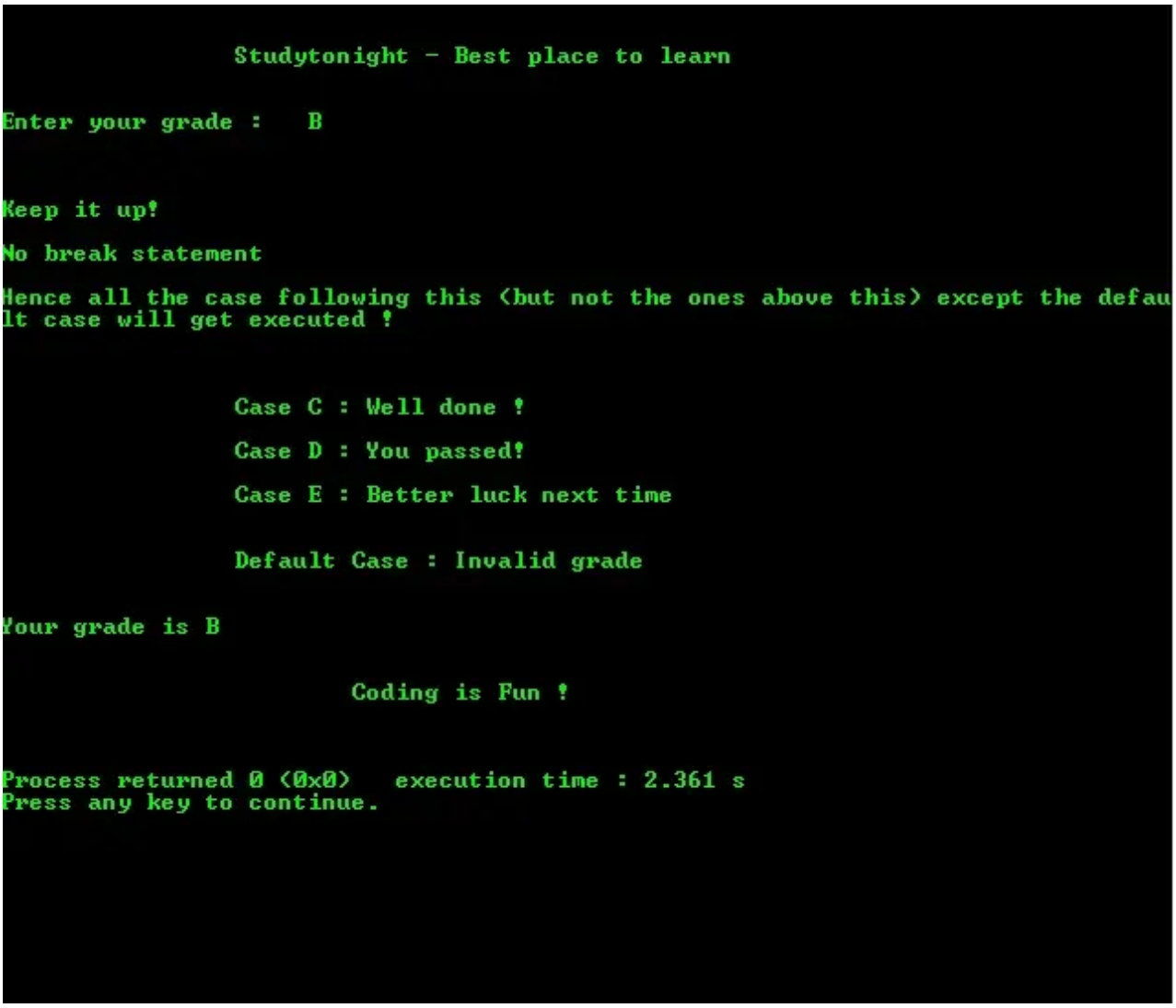
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    /* Local Variable Definition */
    char grade;
    printf("Enter your grade:\n");
    scanf("%c", &grade);

    switch(grade)
    {
        case 'A':
            printf("Excellent\n");
        case 'B':
            printf("\n\n\nKeep it up!\n\nNo break statement\n\nHence all the case
following this(but not the ones above this) except the default case will get
executed !\n\n");
        case 'C':
            printf("\n\n\t\tCase C : Well done !\n\n");
        case 'D':
            printf("\t\tCase D : You passed!\n\n");
        case 'F':
            printf("\t\tCase E : Better luck next time\n\n\n");
        default:
            printf("\t\tDefault Case : Invalid grade\n\n\n");
    }
    printf("Your grade is %c\n",grade);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Some more code examples

Here are some other examples you might come across involving switch case.

1. Valid and invalid expressions

```
int a = 4, b = 6;
float c = 4.5;
char ch1 = 'a', ch2  = 'c';
switch((a * b) % 2)  //valid
switch(c)            //invalid
switch(ch2 + ch1)    //valid
```

2. Switch case with no default statement

```
int x = 4;
switch (x) {
    case 1:
        printf("Choice is 1");
        break;
    case 2:
        printf("Choice is 2");
        break;
    case 3:
        printf("Choice is 3");
        break;
    }
```

The above piece of code will not print anything since `x` does not match with any of the case values.

3. Multiple case together

```
#include <stdio.h>
int main() {

   int x = 1;
   switch(x)
   {
    case 1:
    case 2:
        printf("1 and 2\n");         //it will print till a break or default is
reached
        break;

    case 3:
    case 4:
        printf("3 and 4\n");
        break;

    default:printf("wrong choice!\n");
        break;
   }
   return 0;

}
```

1 and 2

Points to remember (Interview questions for Switch case in C)

- `switch()` can only contain `char` and `int` .
- `break` is used to exit from **switch** statement. It is optional.
- **switch** case can be without **default** case.
- A `char` variable is always initialized within single quotes.

- The expression provided in `switch` should result in a constant value otherwise it will be invalid.
- The case values should be distinct.
- We can also nest `switch` statements.

Program to check if input character is a vowel using Switch Case

studytonight.com/c/programs/basic/check-vowel-using-switch-case

Below is a program to check vowel using switch case.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char ch;
    printf("Input a Character :  ");
    scanf("%c", &ch);

    switch(ch)
    {
        case 'a':
        case 'A':
        case 'e':
        case 'E':
        case 'i':
        case 'I':
        case 'o':
        case 'O':
        case 'u':
        case 'U':
            printf("\n\n%c is a vowel.\n\n", ch);
            break;
        default:
            printf("%c is not a vowel.\n\n", ch);
    }
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Explanation:

- If `break` statement is not used for a case then all the cases following the valid case are executed and evaluated. This way you can make your code easier to understand by writing only `break` statement only once to check multiple conditions in one go.
- `default` is executed only if none of the above cases are true. It is similar to the `else` statement of the if-else code.

Program to reverse the case of input character

studytonight.com/c/programs/basic/reversing-case-of-character

Below is a program to reverse the case of input character.

`getchar()` is similar to `scanf()`.

`islower()` is system defined function under **ctype.h** header file to check if the character is in lowercase or not.

`toupper()` converts the input parameter into equivalent uppercase char.

`putchar()` is similar to `printf()`.

```
#include<stdio.h>
#include<ctype.h> // to use system defined function islower & toupper

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char alphabet;
    printf("Enter an alphabet : ");
    putchar('\n');  // to move to next Line

    alphabet=getchar();

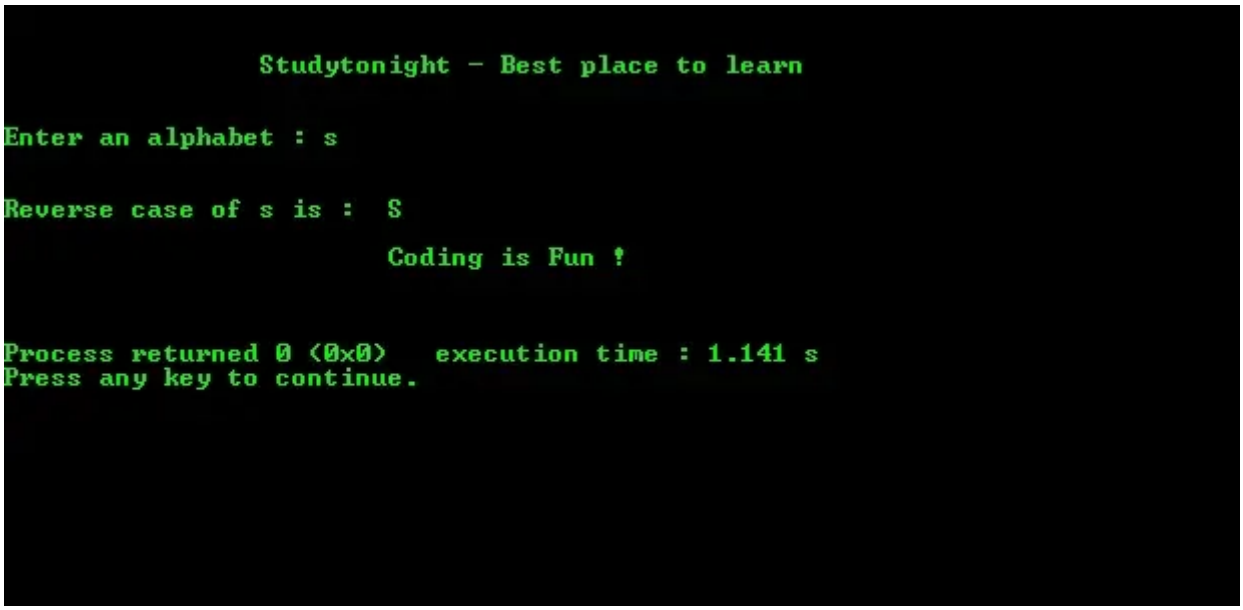
    printf("\n\nReverse case of %c is :  ",alphabet);

    if(islower(alphabet))
        putchar(toupper(alphabet));

    else
        // must be an uppercase character
        printf("%c",tolower(alphabet)) ;

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter an alphabet : s
Reverse case of s is : S
Coding is Fun !
Process returned 0 (0x0) execution time : 1.141 s
Press any key to continue.

Program of Swapping two numbers in C Language

studytonight.com/c/programs/basic/swapping-two-numbers-program

Swapping of two numbers in C Language is the process in which the value of two variables is exchanged using some code. For example,

```
a = 5, b = 4
// After swapping:
a = 4, b = 5
```

We can swap two numbers in various ways as follows:

1. Swapping two variable values using a Temporary Variable
2. Swapping two variable values using Addition and Subtraction
3. Swapping two variable values using Bitwise Operator
4. Swapping two variable values using Multiplication and Division

1. Swapping two numbers in C using a temporary Variable

Let's start with the algorithm steps first,

Algorithm:

1. declare three variables `x`, `y` and `temp`
2. take input in `x` and `y`, say x = 5 and y = 7
3. assign the value of `x` to `temp`, say 5
4. now temp = 5 and x = 5
5. put the value of `y` in `x`, so y = 7 and x = 7
6. then, put the value of `temp` in `y`, so temp = 5 and y = 5

Below is a program to swap two numbers using temporary variable.

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int x = 10, y = 15, temp;
    temp = x;
    x = y;
    y = temp;
    printf("x = %d and y = %d", x, y);
    getch();
}
```

x = 15 and y = 10

2. Swapping two numbers using Addition and Subtraction

Let's start with the algorithm steps first,

Algorithm:

1. Take input of the two numbers, say x = 5 and y = 7
2. Store the sum of both the numbers in the first number(x = 5 + 7 so x = 12) and store the difference of both the numbers in the second number(y = 12 - 7, so y = 5).
3. Then store their difference in the first number(x = 12 - 5 so x = 7) and print.

Below is a program to swap two numbers without using any temporary variable, and use addition and subtraction operator for doing it.

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int x = 10, y = 15;
    x = x + y - (y = x);
    printf("x = %d and y = %d",x,y);
    getch();
}
```

x = 15 and y = 10

3. Swapping two numbers using Bitwise Operator

XOR gives output as 1 when two different bits are XORed and give 0 when two same bits are XORed. The XOR of two numbers x and y returns a number that has all the bits as 1 wherever bits of x and y differ. For example, XOR of 7 (0111) and 5 (0101) is (0010).

Algorithm:

1. Input two numbers, say x = 5 and y = 7
2. XOR the two numbers and store the result in the first number(x = 5 ^ 7 so x = 2)
3. XOR the two numbers again and store the result in the second number (y = 2 ^ 7 so y = 5)
4. XOR the two numbers again and store the result in the first number (x = 2 ^ 5 so x = 7)

Below is the program to swap two numbers using bitwise operator.

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int x = 6, y = 4;
    x = x^y;
    y = x^y;
    x = x^y;
    printf("x = %d and y = %d", x, y);
    getch();
}
```

x = 4 and y = 6

---

4. Swapping two numbers using Multiplication and Division

Let's start with the algorithm steps first,

Algorithm:

1. Take input of the two numbers, say x = 5 and y = 7
2. Store the product of both the numbers in the first number(x = 5 * 7 so x = 35) and store the quotient of both the numbers in the second number(y = 35 / 7, so y = 5).
3. Then store their difference in the first number(x = 35 / 5 so x = 7) and print.

Below is the program to swap two numbers using multiplication and division.

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int x = 6, y = 4;
    x = x*y;
    y = x/y;
    x = x/y;
    printf("x = %d and y = %d", x, y);
    getch();
}
```

x = 4 and y = 6

Points to Remember

1. In the algorithm using addition and division and XOR, if the values are very big, it can result in integer overflow.
2. In the algorithm using division and multiplication, if one of the values is zero, the product will become zero and the algorithm will fail.

Program to print the Largest and Smallest using Global Declaration

studytonight.com/c/programs/basic/largest-and-smallest-using-global-declaration

Some important points about Global variable declaration are:

- It can be done anywhere within the program.
- Unlike local variables that can be used within the scope of a particular function.
- `&` is used to assign the input value to the variable and store it at that particular location.

`%0nd` is used to represent numbers in `n` digit format with leading 0's.

Below is a program to find largest and smallest value using global declaration.

```
#include<stdio.h>

int a,b;
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    printf("\n\nEnter the two values to find the greatest and smallest number:
\n");
    scanf("%d%d", &a, &b);

    if(a == b)
        printf("Both are equal\n");

    else if(a < b)
    {
        printf("\n\nThe largest number is %03d\n", b);
        printf("\nThe smallest number is %03d\n", a);
        printf("\nThe largest number is %03d\n", b);
    }
    else    //Only possibility remaining
    {
        printf("The largest number is %03d\n", a);
        printf("The smallest number is %03d\n", b);
    }
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the two values to find the greatest and smallest number:
10
7
The largest number is 010
The smallest number is 007
Coding is Fun !
Process returned 0 (0x0)   execution time : 4.578 s
Press any key to continue.

Basic for Loop Program

studytonight.com/c/programs/loop/for-loop

Every loop consists of three parts in a sequence

1. **Initialization**: Use to initialize the loop variable.
2. **Condition**: It is checked after each iteration as an entry point to the loop.
3. **Updation**: Incrementing the loop variable to eventually terminate the loop not satisfying the loop condition.

Remember that the loop condition checks the conditional statement before it loops again.

Syntax:

```
for(initialization, condition, incrementation)
{
    code statements;
}
```

Below is a simple program on `for` loop.

Here is the C language tutorial explaining `for` Loop → For Loop in C

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    /*
        Always declare the variables before using them
    */
    int i = 0;  // declaration and initialization at the same time


    for(i = 0; i < 10; i++)
    {
        printf("i = %d\n", i);

        /*
            consequently, when i equals 10, the loop breaks.
            i is updated before the condition is checked-
            hence the value of i after exiting the loop is 10
        */
     }

    printf("\n\The value of i after exiting the loop is %d\n\n", i);

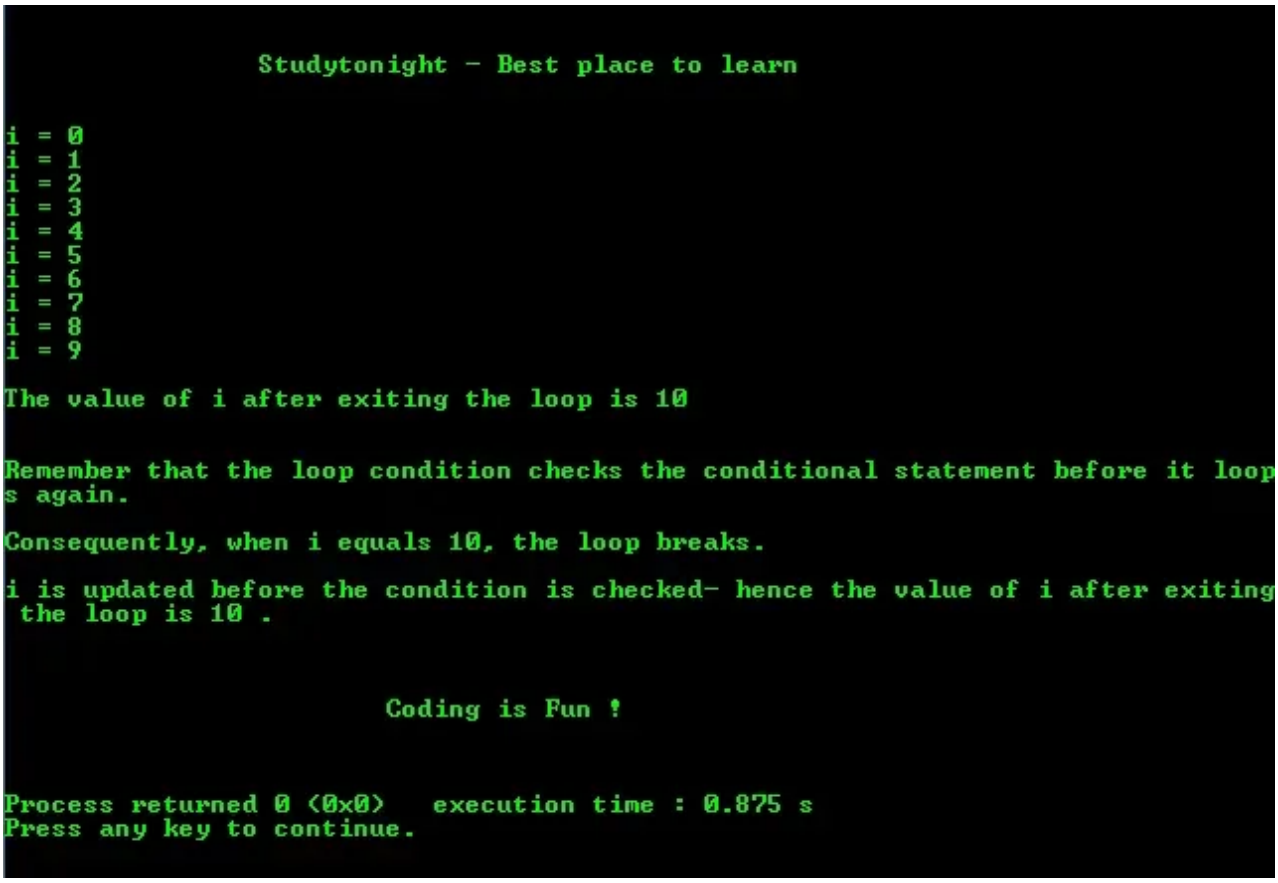
    printf("\nRemember that the loop condition checks the conditional statement
before it loops again.\n\n");

    printf("Consequently, when i equals 10, the loop breaks.\n\n");

    printf("i is updated before the condition is checked- hence the value of i
after exiting the loop is 10 .\n\n");

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
i = 0
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
The value of i after exiting the loop is 10
Remember that the loop condition checks the conditional statement before it loop
s again.
Consequently, when i equals 10, the loop breaks.
i is updated before the condition is checked- hence the value of i after exiting
the loop is 10 .
Coding is Fun !
Process returned 0 (0x0) execution time : 0.875 s
Press any key to continue.

Simple while Loop Program

studytonight.com/c/programs/loop/while-loop

Every loop consists of three parts in sequence

1. **Initialization**: Use to initialize the loop variable.
2. **Condition**: It is checked after each iteration as an entry point to the loop.
3. **Updation**: Incrementing the loop variable to eventually terminate the loop not satisfying the loop condition.

Here is the C language tutorial explaining while Loop → while Loop in C

Below is a simple program on `while` loop.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

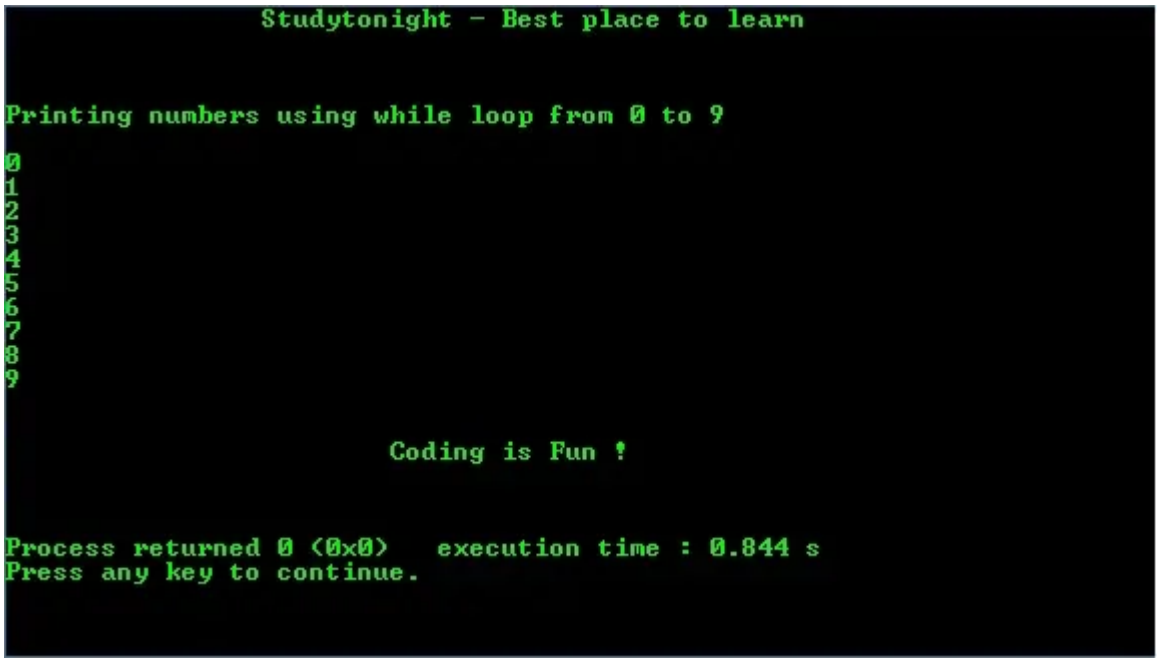
    /*
        always declare the variables before using them
    */
    int i = 0;  // declaration and initialization at the same time

    printf("\nPrinting numbers using while loop from 0 to 9\n\n");

    /*
        while i is less than 10
    */
    while(i<10)
    {
        printf("%d\n",i);

        /*
            Update i so the condition can be met eventually
            to terminate the loop
        */
        i++;    // same as i=i+1;
      }
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Printing numbers using while loop from 0 to 9
0
1
2
3
4
5
6
7
8
9
Coding is Fun !
Process returned 0 (0x0)   execution time : 0.844 s
Press any key to continue.

Basic do while Loop Program

studytonight.com/c/programs/loop/do-while-loop

Every loop consists of three parts in sequence:

1. **Initialization**: Use to initialize the loop variable.
2. **Condition**: It is checked after each iteration as an entry point to the loop.
3. **Updation**: Incrementing the loop variable to eventually terminate the loop not satisfying the loop condition.

Here is the C language tutorial explaining do while Loop → do while Loop in C

Do while loop is used when the actual code must be executed atleast **once**. For example: Incase of menu driven functions.

Below is a simple program on do while loop.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    /*
        always declare the variables before using them
    */
    int i = 10;     // declaration and initialization at the same time

    do // do contains the actual code and the updation
    {
        printf("i = %d\n",i);
        i = i-1;    // updation
    }
    // while loop doesn't contain any code but just the condition
    while(i > 0);

    printf("\n\The value of i after exiting the loop is %d\n\n", i);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
i= 10
i= 9
i= 8
i= 7
i= 6
i= 5
i= 4
i= 3
i= 2
i= 1
The value of i after exiting the loop is 0
Coding is Fun !
Process returned 0 (0x0)   execution time : 0.766 s
Press any key to continue.

Basic program to show use of nested for Loops

studytonight.com/c/programs/loop/nested-loops

Nested loops are usually used to print a pattern in C. They are also used to print out the matrix using a 2 dimensional array and a lot of other patterns like pyramid of numbers etc.

Here is the C language tutorial explaining nested for Loop → nested for Loop in C

Using a loop inside another loop is called **nested loop**.

Below is a simple program on nested loops.

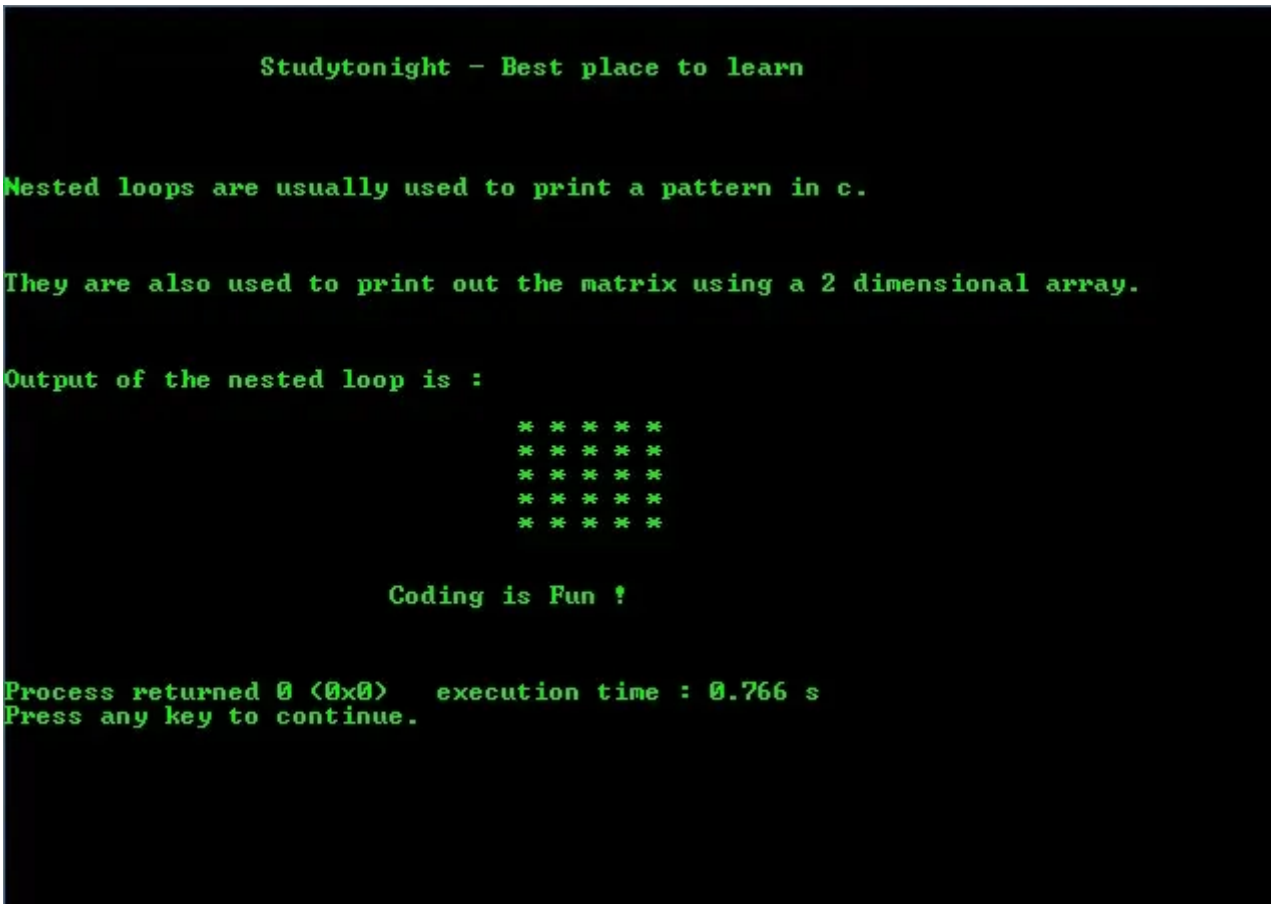
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    printf("\n\nNested loops are usually used to print a pattern in c. \n\n");
    printf("\n\nThey are also used to print out the matrix using a 2 dimensional
array. \n\n");

    int i,j,k;
    printf("\n\nOutput of the nested loop is :\n\n");
    for(i = 0; i < 5; i++)
    {
        printf("\t\t\t\t");
        for(j = 0; j < 5; j++)
        printf("* ");

        printf("\n");
    }
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Nested loops are usually used to print a pattern in c.
They are also used to print out the matrix using a 2 dimensional array.
Output of the nested loop is :
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
Coding is Fun !
Process returned 0 (0x0) execution time : 0.766 s
Press any key to continue.

Program to find Factorial of a Number in C

studytonight.com/c/programs/loop/factorial-program

Before we write the program to find a factorial of a number in C language, let's understand how we can in general find factorial of a number and then we will write a program for it.

What is Factorial of a Number?

The factorial (denoted by `n!`) for a number (say `n`) is the product of all the numbers preceding n with the number itself.

!n = n * (n - 1) * (n - 2) * . . . * 2 * 1

For example,

!6 = 6 * 5 * 4 * 3 * 2 * 1 = 720

!12 = 12 * 11 * 10 * 9 * 8 * 7 * !6 = 479, 001, 600

We can say that,

!n = n * !(n - 1)

Important points-

- Factorial of 0 = 1
- Factorial of negative numbers does not exist.

Algorithm for Factorial Program:

Here are steps to be followed for the factorial program,

```
declare variables n and fact = 1     //n is the number whose factorial is to
be calculated and fact is the variable in which we will store the result
read input from the user in n
initialize loop iterator i = 1 and run the loop till i <= n
do the following in each iteration of the loop
    fact = fact * i
    i++
print fact
```

Now let's see a few implementation of this program.

1. Program to find Factorial of a Number using for loop

In this program, we will use for loop to find factorial of a number.

```
#include<stdio.h>
void main()
{
   int i, n;
   long int fact = 1;
   printf("Enter the number: ");
   scanf("%d" , &n);
   for(i = 1; i <= n; i++)
   {
       fact = fact * i;
   }
   printf("Factorial of %d is %ld", n , fact);
}
```

Enter the number: 5
Factorial of 5 is 120

2. Program to find Factorial of a Number using while loop

In this program, we will use while loop to find factorial of a number.

```
#include<stdio.h>
void main()
{
   int n;
   long int fact = 1;
   printf("Enter the number: ");
   scanf("%d" , &n);
   int i = n;
   while(i) {
       fact = fact * i;
       i--;
   }
   printf("Factorial of %d is %ld", n , fact);
}
```

Enter the number: 0
Factorial of 0 is 1

Things to keep in mind while calculating factorial of a number-

- The variable in which we will store our result should have a large size. `int` can only store factorials upto **12**. We use `long` for higher numbers.
- The variable in which we will store our **result** should always be initialized by 1. This is because if we don't initialize it, it will contain **garbage value** and the numbers will be multiplied with the garbage value. If we initialize it with 0, the product will always be zero.

- We run the loop from 1 to n because we want the product of numbers from 1, 2, 3, ... to n.

Program to print the Fibonacci Series

studytonight.com/c/programs/loop/fibonacci-series-program

Below is a program to print the Fibonacci series using while loop.

```
#include<stdio.h>
#include<conio.h>

void fibonacci(int num);
void main()
{
    int num = 0;
    clrscr();
    printf("Enter number of terms\t");
    scanf("%d", &num);
    fibonacci(num);
    getch();
}

void fibonacci(int num)
{
   int a, b, c, i = 3;
   a = 0;
   b = 1;
   if(num == 1)
   printf("%d",a);

   if(num >= 2)
   printf("%d\t%d",a,b);

   while(i <= num)
   {
      c = a+b;
      printf("\t%d", c);
      a = b;
      b = c;
      i++;
   }
}
```

Enter number of terms 6 0 1 1 2 3 5

Program to check whether a Number is a Palindrome

studytonight.com/c/programs/loop/palindrome-program

Below is a program to check whether a number is a palindrome or not.

A **palindrome** is a number or a string which is similar when read from the front and from the rear. For example: 121 or Oppo etc.

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a, b, c, s = 0;
    clrscr();
    printf("Enter a number:\t");
    scanf("%d", &a);
    c = a;

   // the number is reversed inside the while loop.
   while(a > 0)
   {
      b = a%10;
      s = (s*10)+b;
      a = a/10;
   }

   // here the reversed number is compared with the given number.
   if(s == c)
   {
      printf("The number %d is a palindrome", c);
   }
   else
   {
      printf("The number %d is not a palindrome", c);
   }
  getch();
}
```

Enter the number: 121 The number 121 is a palindrome

Program to find Sum of Digits of a Number

studytonight.com/c/programs/loop/sum-of-digits-program

Below is a program for sum of digits of a number.

```
#include<stdio.h>

int main()
{
        printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

        int n, sum = 0, c, remainder;

        printf("Enter the number you want to add digits of:  ");
        scanf("%d", &n);

        while(n != 0)
        {
                remainder = n%10;
                sum += remainder;
                n = n/10;
        }

        printf("\n\nSum of the digits of the entered number is  =  %d\n\n", sum);
        printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
        return 0;
}
```

Output:



Program to reverse a String

studytonight.com/c/programs/loop/reverse-a-string-program

In this tutorial, we will learn about how to reverse a given String. If the string is `"hello"` then, the output should be `"olleh"`. We can use this concept to check the palindrome. Because the palindrome string will have the same value even after we reverse it.

It is advised to go through these topics if you are not familiar with C programs.

- Overview of C Language
- String and Character Array in C

Mainly there are three ways to reserve a given string:

- By using the new character array.
- By swapping the characters of the string.
- By using standard library functions.

Reverse a String by using the new character array:

Here in this example, firstly we take an input from the user, after taking an input we have to calculate the length of the string. To calculate the length we run a loop from the starting of the character array till a null character found (`'\0'`) and in each iteration, we increase the count variable. We assign one less than it to the j because the array starts from zero. After this, we simply copy the characters from the ending one by one from the original character array to a new character array.

```
#include <stdio.h>
int main()
{
  char str[1000], rev[1000];
  int i, j, count = 0;
  scanf("%s", str);
  printf("\nString Before Reverse: %s", str);
  //finding the length of the string
  while (str[count] != '\0')
  {
    count++;
  }
  j = count - 1;

  //reversing the string by swapping
  for (i = 0; i < count; i++)
  {
    rev[i] = str[j];
    j--;
  }

  printf("\nString After Reverse: %s", rev);
}
```

Hello

String Before Reverse: Hello
String After Reverse: olleH

Reverse a String by swapping the characters of the string

This example will sort strings by swapping the characters.

```
#include <stdio.h>
#include <string.h>
void reverseStr(char str[])
{
  int n = strlen(str);

  for (int i = 0; i < n / 2; i++)
  {
    char ch = str[i];
    str[i] = str[n - i - 1];
    str[n - i - 1] = ch;
  }
}

int main()
{
  char str[1000];
  scanf("%s", str);
  printf("\nString Before Reverse: %s", str);
  reverseStr(str);
  printf("\nString After Reverse: %s", str);
  return 0;
}
```

Hello

String Before Reverse: Hello
String After Reverse: olleH

Program to find average of N Numbers

studytonight.com/c/programs/numbers/average-of-n-numbers

Below is a program to calculate average of **n** numbers.

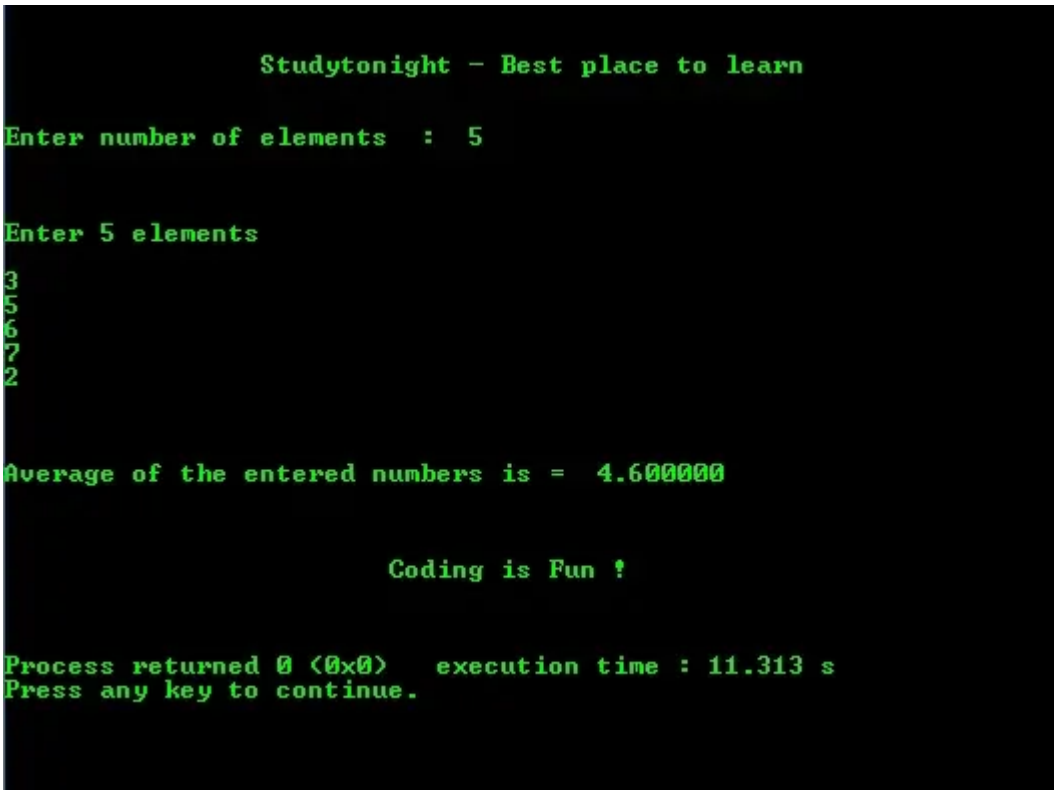
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int n, i;
    float sum = 0, x;

    printf("Enter number of elements:  ");
    scanf("%d", &n);
    printf("\n\n\nEnter %d elements\n\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%f", &x);
        sum += x;
    }
    printf("\n\n\nAverage of the entered numbers is =  %f", (sum/n));
    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter number of elements : 5
Enter 5 elements
3
5
6
7
2
Average of the entered numbers is = 4.600000
Coding is Fun !
Process returned 0 (0x0) execution time : 11.313 s
Press any key to continue.

Program to find Armstrong Number between 1 to 500

studytonight.com/c/programs/numbers/armstrong-number-program

An **Armstrong number** or **Narcissistic number** is a **n** digit number such that the sum of its digits raised to the nth power is equal to the number itself.

For example, Let's take an armstrong number: `153`, which is 3 digit number, here $1^3 + 5^3 + 3^3$ is `1 + 125 + 27` which is equal to **153**.

Below is a program to find armstrong numbers between 1 to 500.

```
#include<stdio.h>
#include<math.h>

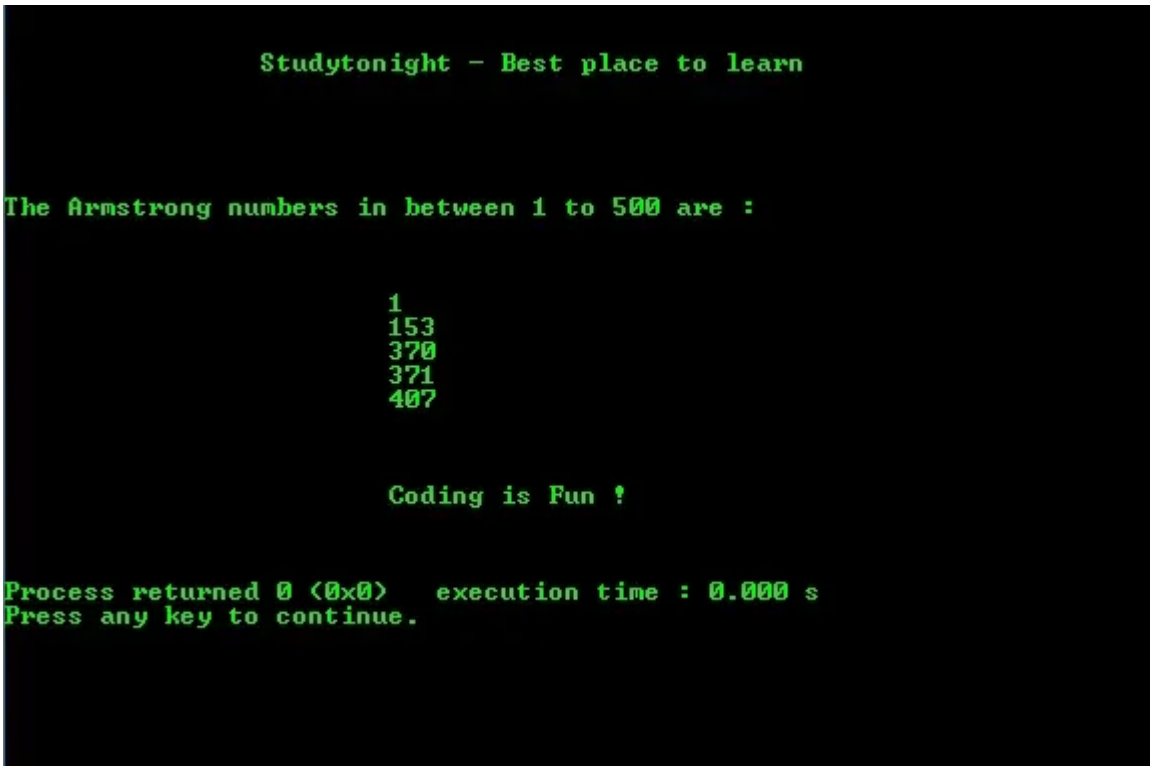
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n,sum,i,t,a;
    printf("\n\n\nThe Armstrong numbers in between 1 to 500 are : \n\n\n");

    for(i = 1; i <= 500; i++)
    {
        t = i;  // as we need to retain the original number
        sum = 0;
        while(t != 0)
        {
            a = t%10;
            sum += a*a*a;
            t = t/10;
        }

        if(sum == i)
        printf("\n\t\t\t%d", i);
    }

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to check whether a number is Armstrong Number

Below is a program to check whether a number is armstrong or not.

```
#include<stdio.h>
#include<math.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int n, sum = 0, c, t, a;

    printf("Enter a number:  ");
    scanf("%d", &n);

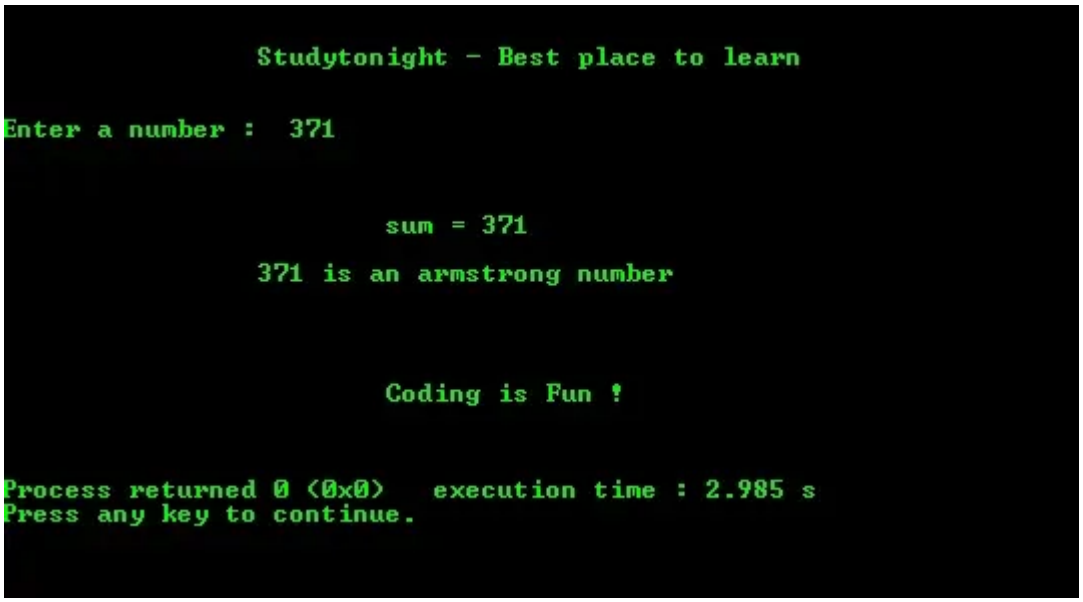
    t = n;  // as need to retain the original number
    while(n != 0)
    {
        a = n%10;
        sum += a*a*a;
        n = n/10;
    }

    printf("\n\n\n\t\t\tsum = %d", sum);

    if(sum == t)
        printf("\n\n\t\t%d is an armstrong number\n", t);
    else
        printf("\n\n\t\t%d is not an armstrong number\n", t);

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Checking for Odd and Even Numbers using Bitwise Operator

studytonight.com/c/programs/numbers/checking-for-odd-or-even

Below is a program to find whether a number is even or odd using bitwise operator.

`x&1` returns **true** if the LSB(Least significant Bit) of binary representation of an integer x is 1. It returns **false** if the LSB or the Right most bit in a binary sequence is 0.

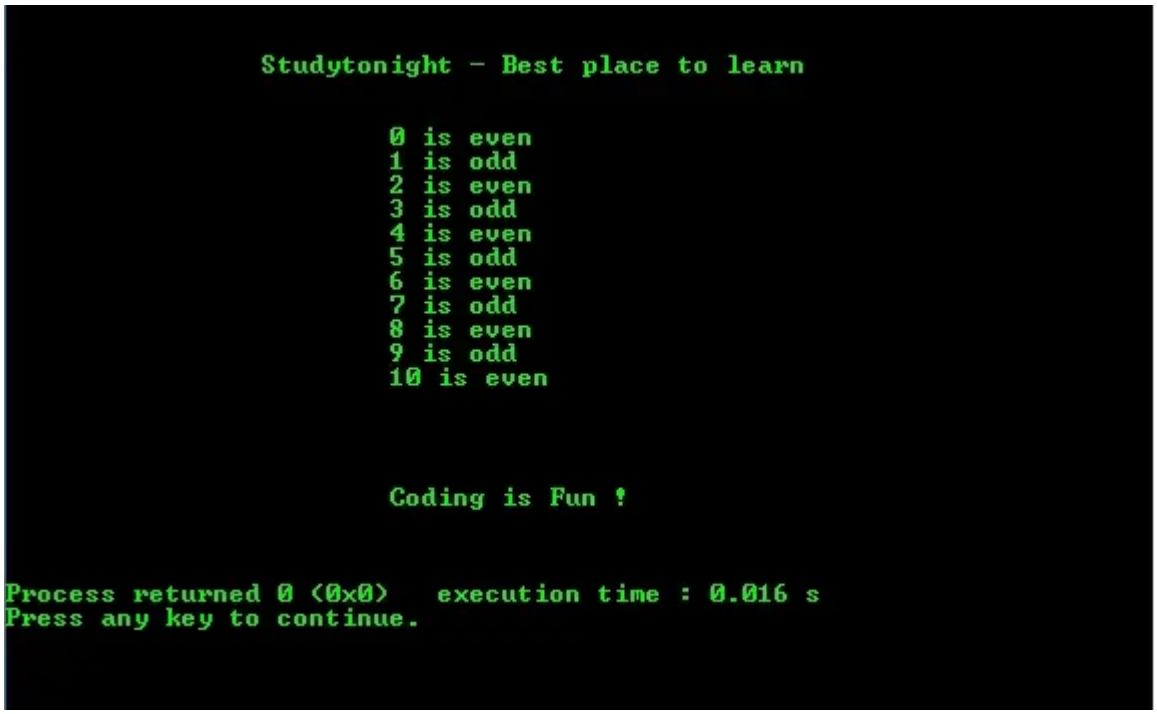
In binary representation of an integer, if LSB is 1 then it is **odd** and if LSB is 0 then it is **even**.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int x;
    for(x = 0; x <= 10; x++)
    {
        if(x&1) // if number is odd
            printf("\t\t\t%d is odd\n",x);
        else if(!(x&1)) // ! is used inside if to reverse the boolean value
            printf("\t\t\t%d is even\n",x);
    }

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



We have used a simple `for` loop to input numbers and show how to use the Bitwise operator. You can take input from user using `scanf()` and use th same logic to find if the input number is odd or even.

Checking if inout number is Odd or Even without using `%` (Mod) Operator

Below is a program to find whether a number is even or odd without using `%` (Mod) operator.

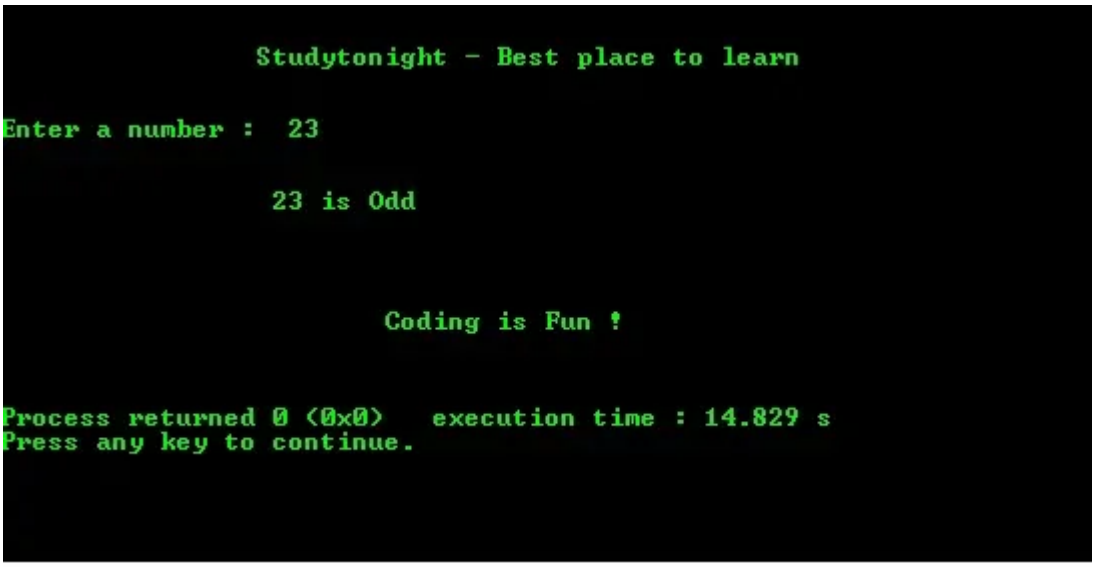
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n;
    printf("Enter a number:  ");
    scanf("%d",&n);

    if((n/2)*2 == n)
        printf("\n\n\t\t %d is Even\n", n);
    else
        printf("\n\n\t\t %d is Odd\n", n);

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter a number : 23
23 is Odd
Coding is Fun !
Process returned 0 (0x0) execution time : 14.829 s
Press any key to continue.

Program to find Factors of a Number

studytonight.com/c/programs/numbers/factors-of-a-number

Below is a program to find factors of a number.

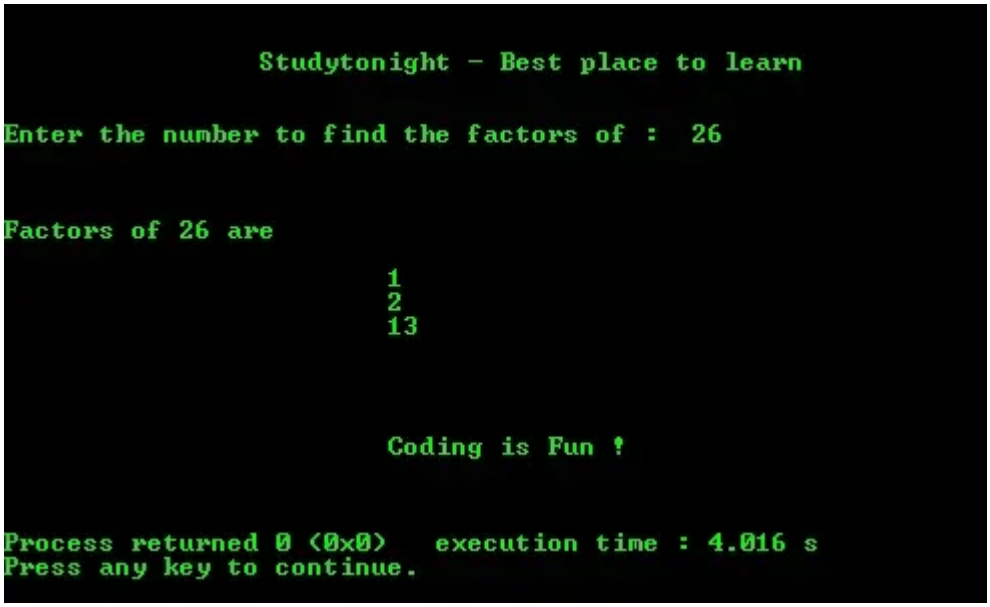
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int  num, i;
    printf("Enter the number to find the factors of :  ");
    scanf("%d",&num);
    printf("\n\n\nFactors of %d are \n\n", num);

    for(i = 1; i <= num/2; i++)
    {
        if(num%i == 0)
            printf("\t\t\t%d\n", i);
    }

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to find Sum of N input Numbers in C

studytonight.com/c/programs/numbers/sum-of-n-numbers

Below is a program on sum of n numbers.

```
#include<stdio.h>

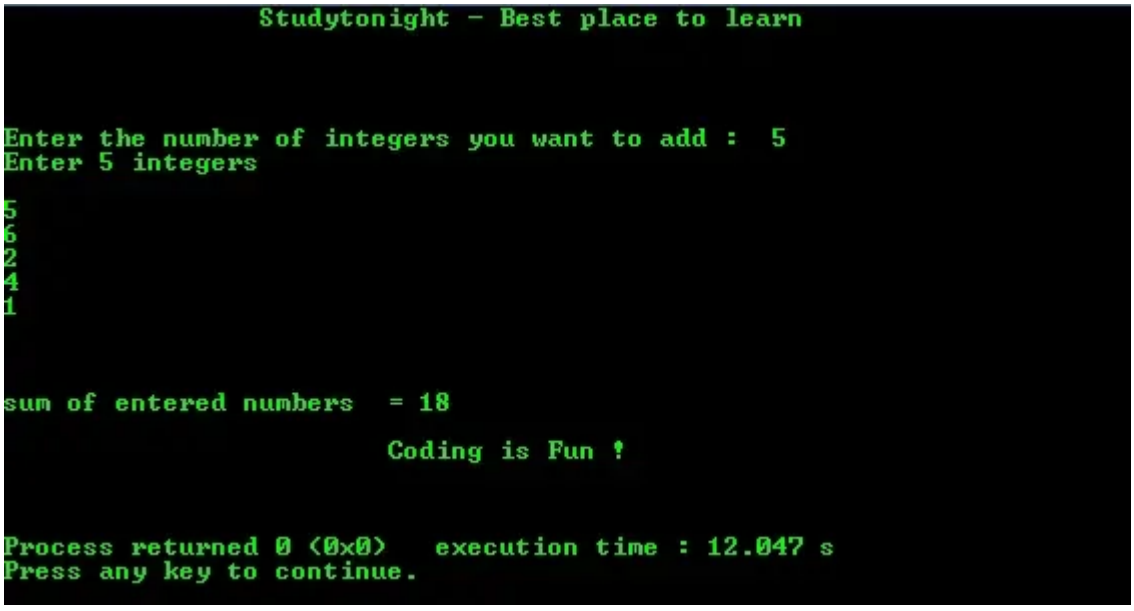
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n,sum=0,c,value;

    printf("\n\nEnter the number of integers you want to add:  ");
    scanf("%d", &n);

    printf("Enter %d integers\n\n",n);
    for(c = 1; c <= n; c++)
    {
        scanf("%d", &value);
        /*
            need to initialise sum before using otherwise
            garbage value will get printed
        */
        sum += value;
    }

    printf("\n\n\nsum of entered numbers  = %d", sum);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to find first N Prime Numbers

studytonight.com/c/programs/numbers/first-n-prime-numbers

Below is a program to find first `n` prime numbers using nested `for` loops, where the value of `n` is input by the user.

Before you continue with program, check this topics to understand the program :

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n,i = 3, count, c;

    printf("\nEnter the number of prime numbers required :  ");
    scanf("%d", &n);

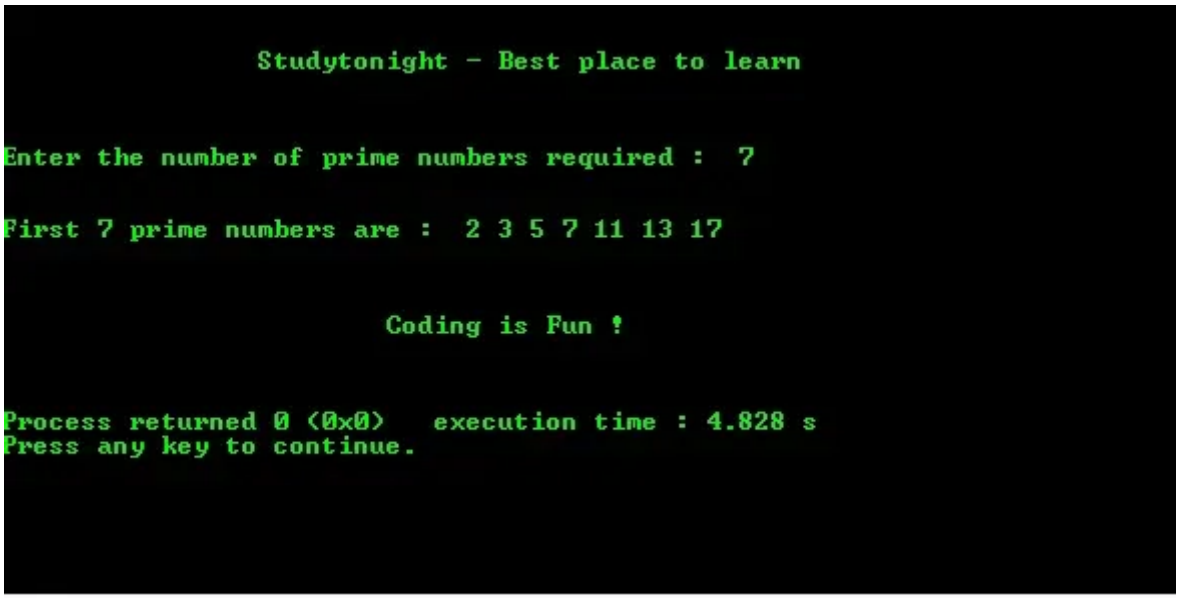
    if(n >= 1)
    {
        printf("\n\nFirst %d prime numbers are :  ", n);
        printf("2 ");
    }

    // iteration for n prime numbers
    // i is the number to be checked in each iteration starting from 3
    for(count = 2; count <= n; i++)
    {
        // iteration to check c is prime or not
        for(c = 2; c < i; c++)
        {
            if(i%c == 0)
                break;
        }

        if(c == i)  // c is prime
        {
            printf("%d ", i);
            count++;    // increment the count of prime numbers
        }

    }
    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the number of prime numbers required : 7
First 7 prime numbers are : 2 3 5 7 11 13 17
Coding is Fun !
Process returned 0 (0x0) execution time : 4.828 s
Press any key to continue.

Program to find the Largest number among n input Numbers

studytonight.com/c/programs/numbers/finding-largest-among-n-numbers

Below is a program to find largest number among `n` user input numbers.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n,i;
    float c,big;

    printf("\n\nEnter the number of elements you wish to find the greatest element
of: ");
    scanf("%d", &n);
    printf("\n\nEnter %d numbers :\n", n);

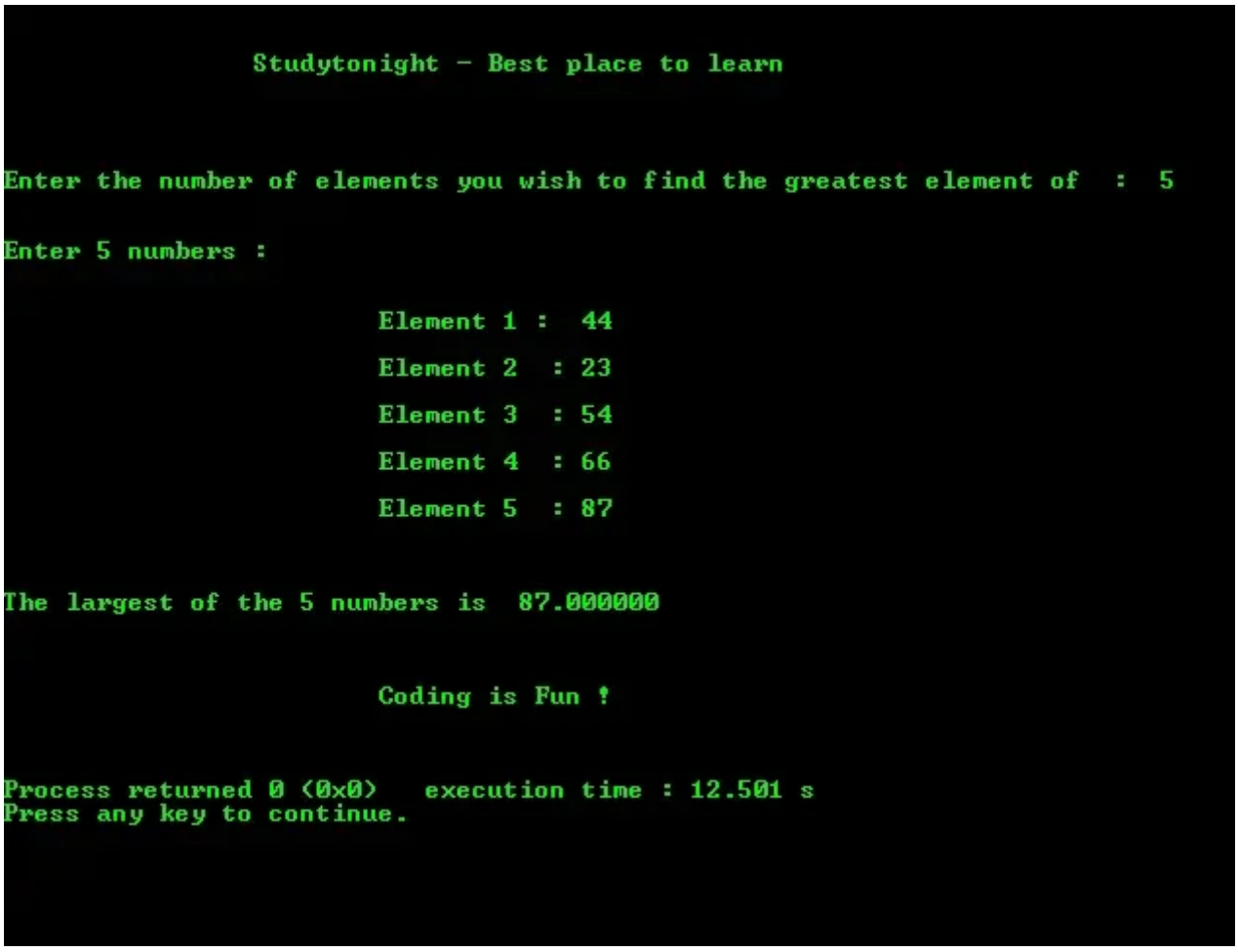
    printf("\n\n\t\t\tElement 1: ");

    //Important step- always initialize big to the first element
    scanf("%f", &big);

    for(i = 2; i <= n; i++)
    {
        printf("\n\t\t\tElement %d  : ", i);
        scanf("%f", &c);
        /*
            if input number is larger than the
            current largest number
        */
        if(big < c)
            big = c;    // update big to the larger value
    }

    printf("\n\n\nThe largest of the %d numbers is  %f ", n, big);
    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the number of elements you wish to find the greatest element of : 5
Enter 5 numbers :
Element 1 : 44
Element 2 : 23
Element 3 : 54
Element 4 : 66
Element 5 : 87
The largest of the 5 numbers is 87.000000
Coding is Fun !
Process returned 0 (0x0) execution time : 12.501 s
Press any key to continue.

Program to find exponential without using pow() method

studytonight.com/c/programs/numbers/finding-exponential-without-pow()-method

Below is a program to find exponential without using `pow()` method.

`long long int` is of double the size of `long int`.

`%lld` is the format specifier for `long long int`.

```
#include<stdio.h>
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int n, exp, exp1;
    long long int value = 1;

    printf("Enter the number and its exponential:\n\n");
    scanf("%d%d",&n, &exp);

    exp1 = exp;   // storing original value for future use

    // same as while((--exp)!=-1)
    while(exp-- > 0)
    {
        value *= n; // multiply n to itself exp times
    }

    printf("\n\n %d^%d = %lld\n\n", n, exp1, value);
    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the number and its exponential:
4
3
4^3 = 64
Coding is Fun !
Process returned 0 (0x0) execution time : 3.688 s
Press any key to continue.

Program to check if input Number is int or float

studytonight.com/c/programs/numbers/input-number-is-int-or-float

Below is a program to check whether the user input number is of integer or float datatype.

`strlen()` does not count the null character '\0'.

```
#include<stdio.h>

#include<conio.h>
#include<string.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char number[10];
    int flag = 0;
    int length, i = 0;

    printf("\n\nEnter a number: ");
    scanf("%s", number);

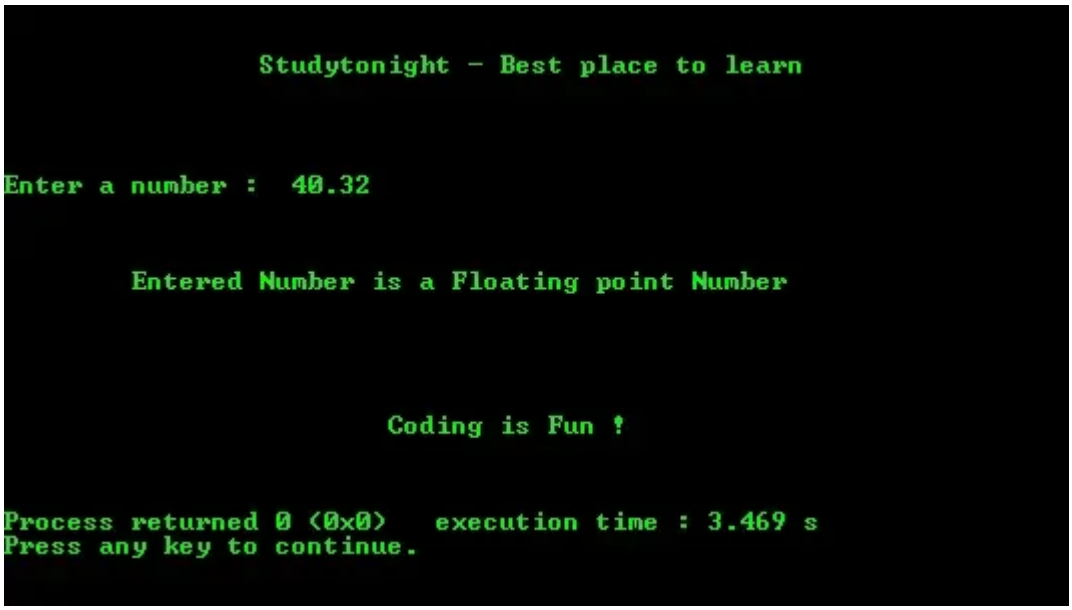
    length = strlen(number);

    // till string does not end
    while(number[i++] != '\0')    // same as while(length-->0)
    {
        if(number[i] == '.')    // decimal point is present
        {
            flag = 1;
            break;
        }
    }

    // if(0) is same as if(false)
    if(flag)
        printf("\n\n\n\tEntered Number is a Floating point Number\n\n");
    else
        printf("\n\n\n\tEntered Number is a integer Number\n\n");

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter a number : 40.32
Entered Number is a Floating point Number
Coding is Fun !
Process returned 0 (0x0) execution time : 3.469 s
Press any key to continue.

Program to print the Multiplication Table of any Number

studytonight.com/c/programs/numbers/multiplication-table-of-a-number

Below is a program to print the multiplication table of any user input number.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n,i;

    printf("Enter an integer you need to print the table of: ");
    scanf("%d", &n);
    printf("\n\n\n");

    for(i = 1; i <= 10; i++)
    {
        printf("\n\t\t\t%d * %d   =   %d \n", n, i, n*i);
    }

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter an integer you need to print the table of : 8
8 * 1 = 8
8 * 2 = 16
8 * 3 = 24
8 * 4 = 32
8 * 5 = 40
8 * 6 = 48
8 * 7 = 56
8 * 8 = 64
8 * 9 = 72
8 * 10 = 80
Coding is Fun !
Process returned 0 (0x0) execution time : 2.109 s
Press any key to continue.

Program to print the reverse of an Array

studytonight.com/c/programs/array/reverse-an-array

Below is a simple program to reverse an array.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int c, d, n, a[100], b[100];
    printf("\n\nEnter number of elements in array :");
    scanf("%d", &n);
    printf("\n\nEnter %d elements\n", n);

    for(c = 0; c < n; c++)
        scanf("%d", &a[c]);

    /*
        temporarily storing elements into array b
        starting from end of array a
    */
    for(c = n-1, d = 0; c >= 0; c--, d++)
        b[d] = a[c];

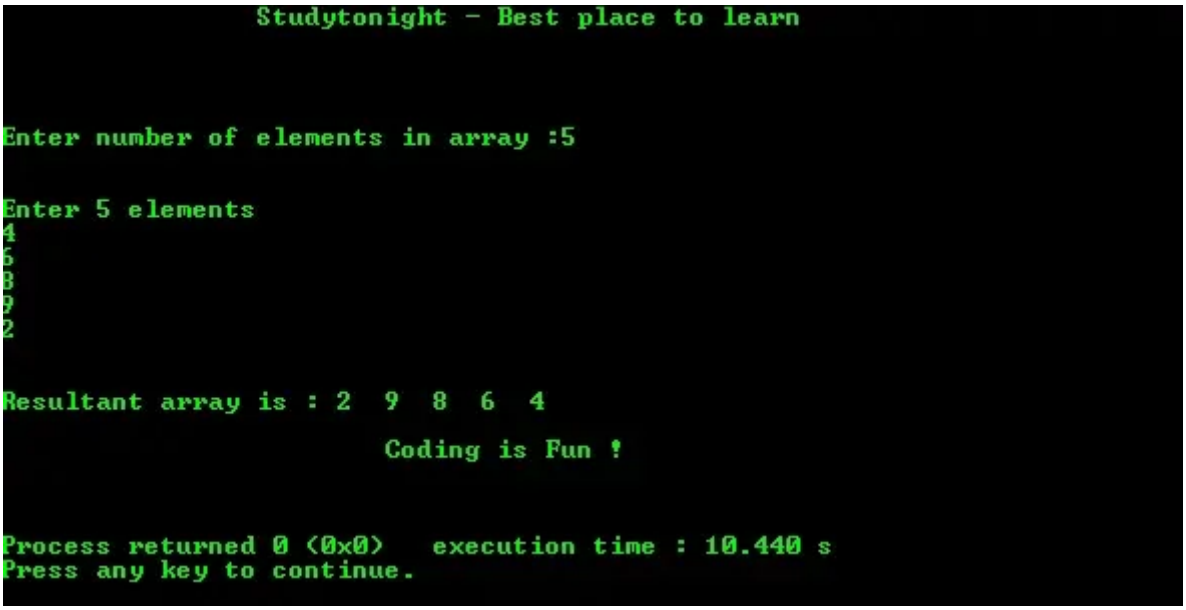
    /*
        copying reversed array into original.
        Here we are modifying original array to reverse it.
    */

    for(c = 0; c < n; c++)
        a[c] = b[c];

    printf("\n\n Resultant array is: ");
    for(c = 0; c < n; c++)
        printf("%d", a[c]);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter number of elements in array :5
Enter 5 elements
4
6
8
9
2
Resultant array is : 2 9 8 6 4
Coding is Fun !
Process returned 0 (0x0) execution time : 10.440 s
Press any key to continue.

Program to insert an element in an Array

studytonight.com/c/programs/array/inserting-an-element-in-array

Below is a simple program to insert an element in an array.

Here is the C language tutorial explaining Arrays → Arrays in C

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int array[100], position, c, n, value;

    printf("\n\nEnter number of elements in array:");
    scanf("%d", &n);

    printf("\n\nEnter %d elements\n", n);
    for(c = 0; c < n; c++)
        scanf("%d", &array[c]);

    printf("\n\nEnter the location where you want to insert new element:  ");
    scanf("%d", &position);

    printf("\n\nEnter the value to insert: ");
    scanf("%d", &value);

    // shifting the elements from (position to n) to right
    for(c = n-1; c >= position-1; c--)
        array[c+1] = array[c];

    array[position - 1] = value;    // inserting the given value

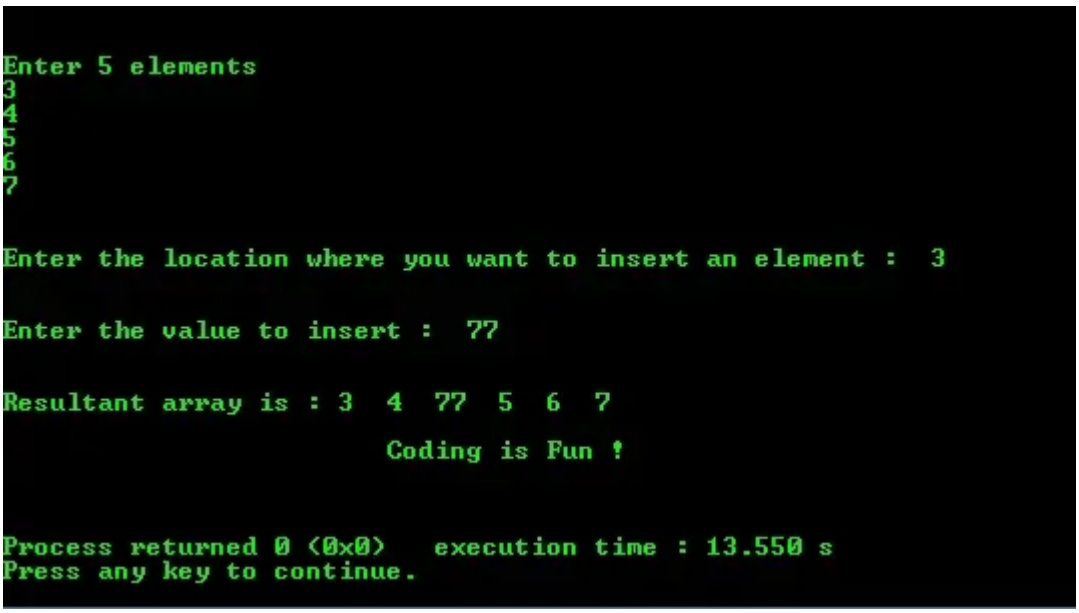
    printf("\n\nResultant array is: ");
    /*
        the array size gets increased by 1
        after insertion of the element
    */
    for(c = 0; c <= n; c++)
        printf("%d  ", array[c]);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

In the above program we take an array as user input and then ask the user for a new number that they wish to add to the original array, and the position where they want to add the new number.

The we shift the existing numbers from the index **position** to the end of the array one position to the right, therby vacating a space for the new element. And then we add the new number at the user specified **position** index.

Output:



Program to Delete an Element from Array in C

studytonight.com/c/programs/array/deleting-an-element-from-array

Below is a simple program to delete an element from array, where the position of element to be deleted is given by user.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int array[100], position, c, n;
    printf("\n\nEnter number of elements in array:");
    scanf("%d", &n);

    printf("\n\nEnter %d elements\n", n);
    for(c = 0; c < n; c++)
        scanf("%d", &array[c]);

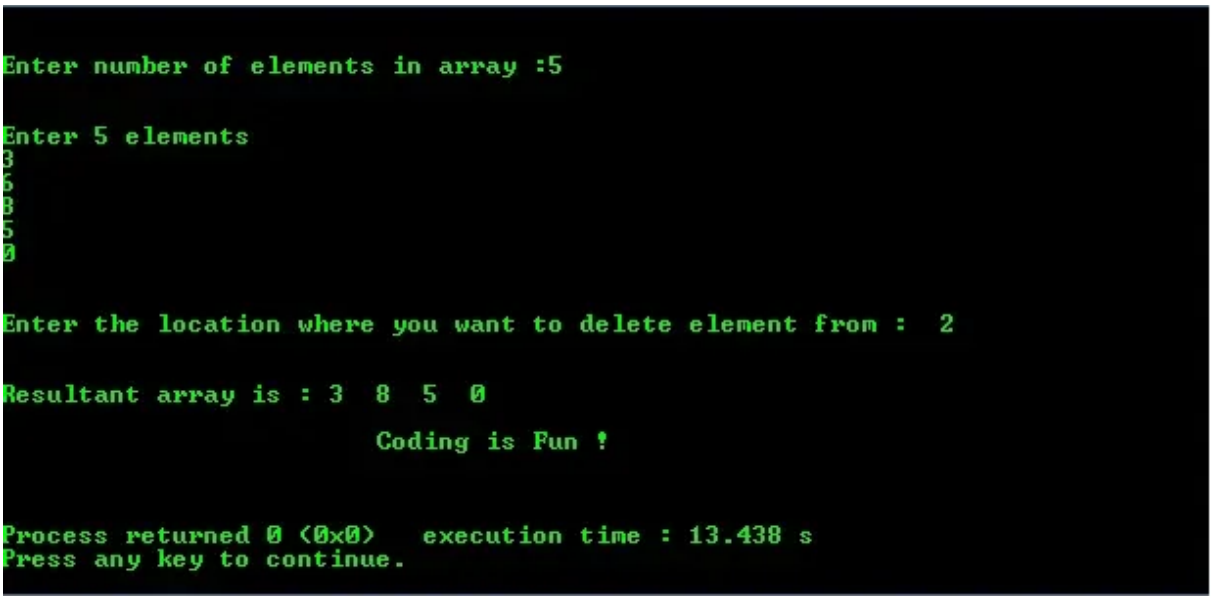
    printf("\n\nEnter the location where you want to delete element from:  ");
    scanf("%d", &position);

    if(position >= n+1)
        printf("\n\nDeletion not possible\n\n");
    else
        // updating the locations with next elements
        for(c = position-1; c < n-1; c++)
        array[c] = array[c+1];

    printf("\n\nResultant array is: ");
    /*
        the array size gets reduced by 1
        after deletion of the element
    */
    for(c = 0; c < n-1; c++)
        printf("%d  ", array[c]);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to Delete an element from array based on value

Below is a simple program to delete an element from array, where the element to be deleted is given by user:

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int array[10], element, c, n, pos;
    /*
        initialization as garbage value is
        stored by default in c variables
    */
    int found = 0;
    printf("\n\nEnter number of elements in array:");
    scanf("%d", &n);

    printf("\n\nEnter %d elements\n", n);

    for(c = 0; c < n; c++)
        scanf("%d", &array[c]);

    printf("\n\nThe input array is: ");
    for(c = 0; c < n; c++)
    printf("%d", array[c]);

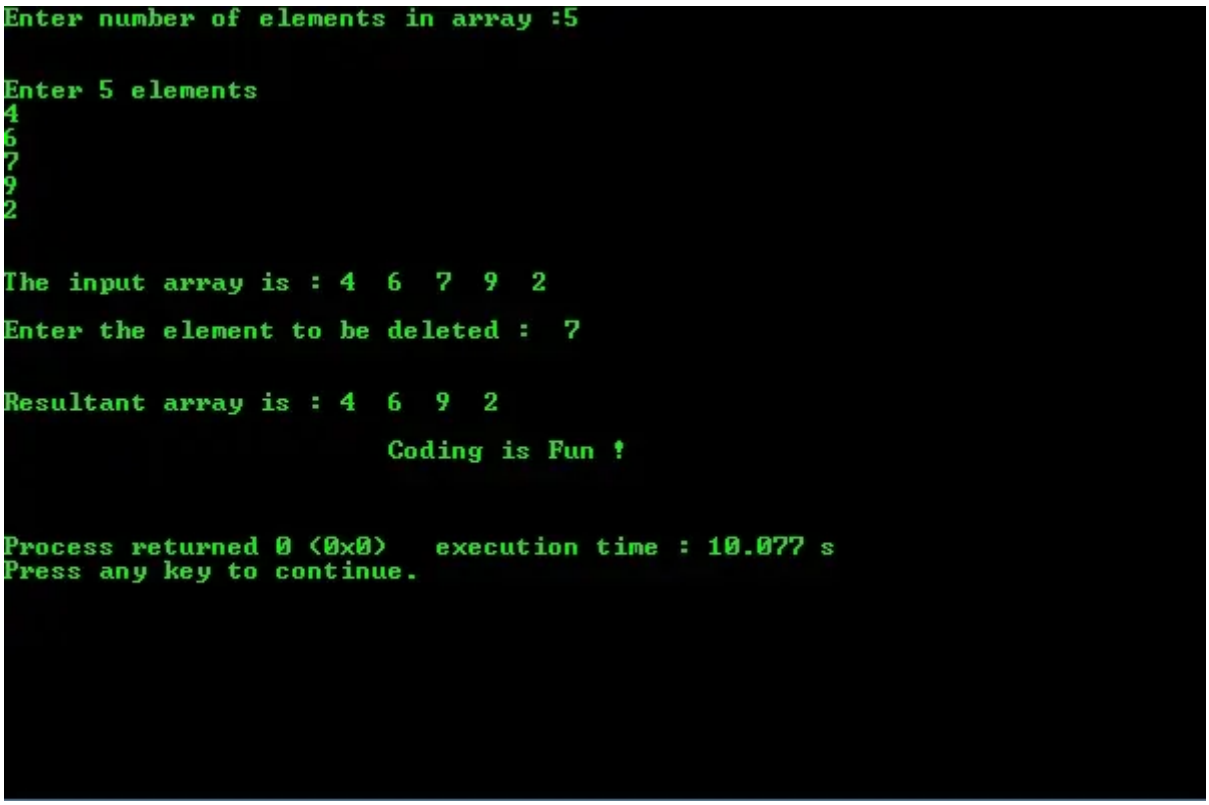
    printf("\n\nEnter the element to be deleted: ");
    scanf("%d", &element);

    // check the element to be deleted is in array or not
    for(c = 0; c < n; c++)
    {
        if(array[c] == element)
        {
            found = 1;
            pos = c;
            break;  // terminate the loop
        }
    }
    if(found == 1) // the element to be deleted exists in the array
    {
        for(c = pos; c < n-1; c++)
            array[c] = array[c+1];
    }
    else
        printf("\n\nElement %d is not found in the array\n\n", element);

    printf("\n\nResultant array is: ");
    /*
        the array size gets reduced by 1
        after deletion of the element
    */
    for(c = 0; c < n-1; c++)
        printf("%d  ",array[c]);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Enter number of elements in array :5
Enter 5 elements
4
6
7
9
2
The input array is : 4 6 7 9 2
Enter the element to be deleted : 7
Resultant array is : 4 6 9 2
Coding is Fun !
Process returned 0 (0x0) execution time : 10.077 s
Press any key to continue.

Program to find Largest and Smallest Element in an Array

studytonight.com/c/programs/array/largest-and-smallest-element-in-array

Below is a program to find the largest and smallest elements in array.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int a[50], size, i, big, small;

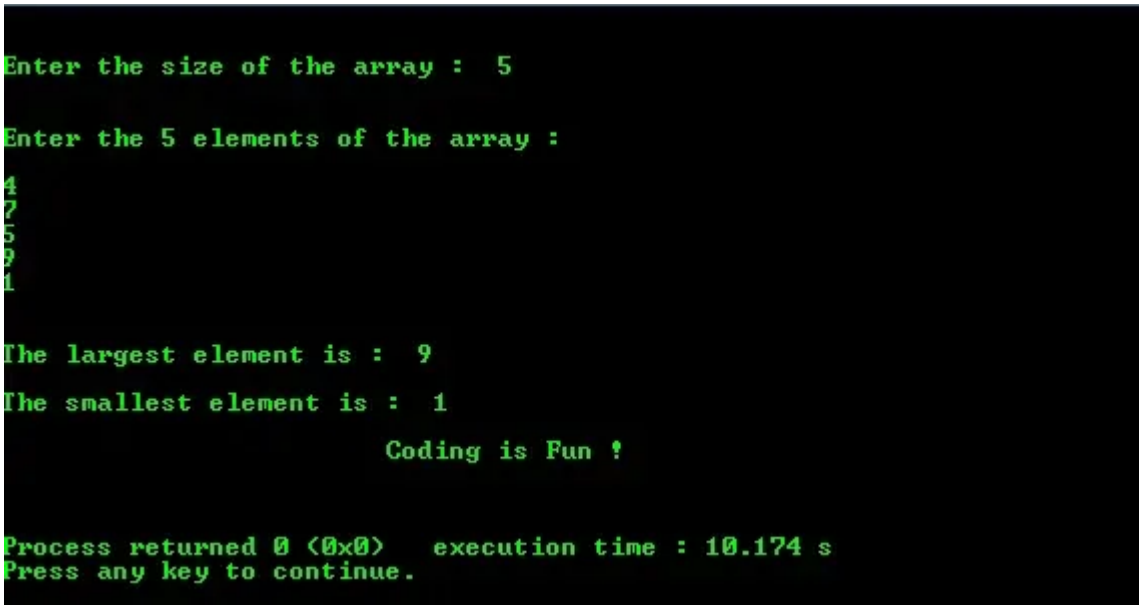
    printf("\nEnter the size of the array: ");
    scanf("%d", &size);

    printf("\n\nEnter the %d elements of the array: \n\n", size);
    for(i = 0; i < size; i++)
    scanf("%d", &a[i]);

    big = a[0]; // initializing
    /*
        from 2nd element to the last element
        find the bigger element than big and
        update the value of big
    */
    for(i = 1; i < size; i++)
    {
        if(big < a[i])   // if larger value is encountered
        {
            big = a[i]; // update the value of big
        }
    }
    printf("\n\nThe largest element is: %d", big);

    small = a[0];   // initializing
    /*
        from 2nd element to the last element
        find the smaller element than small and
        update the value of small
    */
    for(i = 1; i < size; i++)
    {
        if(small>a[i])   // if smaller value is encountered
        {
            small = a[i];   // update the value of small
        }
    }
    printf("\n\nThe smallest element is: %d", small);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Enter the size of the array : 5
Enter the 5 elements of the array :
4
7
5
9
1
The largest element is : 9
The smallest element is : 1
Coding is Fun !
Process returned 0 (0x0) execution time : 10.174 s
Press any key to continue.

Program to find Sum of N input Numbers using Array

studytonight.com/c/programs/array/sum-of-n-numbers-using-arrays

Below is a program to find and print the sum of n numbers using arrays.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n, sum = 0, c, array[100];

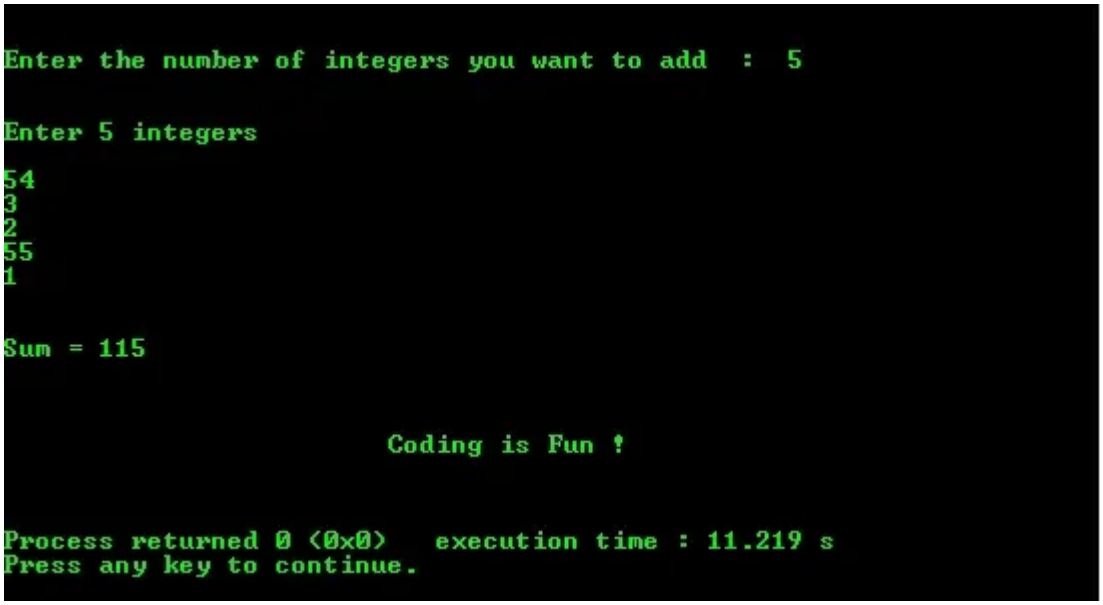
    printf("Enter the number of integers you want to add: ");
    scanf("%d", &n);

    printf("\n\nEnter %d integers \n\n", n);

    for(c = 0; c < n; c++)
    {
        scanf("%d", &array[c]);
        sum += array[c];    // same as sum = sum + array[c]
    }

    printf("\n\nSum = %d\n\n", sum);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Simple Program to Sort Array elements

studytonight.com/c/programs/array/sort-array-element-program

Below is a program to sort array elements in an array.

```
#include<stdio.h>
#include<conio.h>

void sorting(int *x, int y);

void main()
{
    int a[20], i, c, n;
    clrscr();
    printf("Enter number of elements you want to sort: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    scanf("%d", &a[i]);

    sorting(a, n);

    for(i = 0; i <n; i++)
    printf("%d\t", a[i]);

    getch();
}

void sorting(int *x, int y)
{
    int i, j, temp;
    for(i = 1; i <= y-1; i++)
    {
        for(j = 0; j < y-i; j++)
        {
            if(*(x+j) > *(x+j+1))
            {
                temp = *(x+j);
                *(x+j) = *(x+j+1);
                *(x+j+1) = temp;
            }
        }
    }
}
```

Enter number of elements you want to sort: 6 5 3 4 2 1 6 1 2 3 5 6

Simple Program to remove Duplicate Element in an Array

studytonight.com/c/programs/array/remove-duplicate-element-program

In this tutorial, we will learn how to remove a duplicate element from an array. Before moving forward with the program, if you are not familiar with what is an Array please read this article: Array in C language.

Remove duplicates from the sorted array:

Here we are implementing the program to remove a duplicate element from a sorted array. We will create a temporary array and copy elements from one array to another only in case when there is no next matching element.

Note: This program will work only for sorted array so while providing an input make sure that the given array is in a sorted array, otherwise it will give unexpected outputs.

```
#include <stdio.h>
int remove_duplicate(int arr[], int n)
{

  if (n == 0 || n == 1)
    return n;

  int temp[n];

  int j = 0;
  int i;
  for (i = 0; i < n - 1; i++)
    if (arr[i] != arr[i + 1])
      temp[j++] = arr[i];
  temp[j++] = arr[n - 1];

  for (i = 0; i < j; i++)
    arr[i] = temp[i];

  return j;
}

int main()
{
  int n;
  scanf("%d", &n);
  int arr[n];
  int i;
  for (i = 0; i < n; i++)
  {
    scanf("%d", &arr[i]);
  }
  printf("\nArray Before Removing Duplicates: ");
  for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

  n = remove_duplicate(arr, n);

  printf("\nArray After Removing Duplicates: ");
  for (i = 0; i < n; i++)
    printf("%d ", arr[i]);

  return 0;
}
```

10
1 2 2 3 4 5 6 7 7 8

Array Before Removing Duplicates: 1 2 2 3 4 5 6 7 7 8
Array After Removing Duplicates: 1 2 3 4 5 6 7 8

Remove duplicates from the unsorted array:

In this program, we will take extra space to store a new array without the reputation of duplicate elements. We will run a nested loop first one is to copy the element from the array to temp array if that element is existing already in temp variable then we will break the inner loop and continue executing for remaining elements the same process.

```
#include <stdio.h>
int main()
{
  int n, count = 0;
  scanf("%d", &n);
  int arr[n], temp[n];
  for (int i = 0; i < n; i++)
  {
    scanf("%d", &arr[i]);
  }

  printf("\nArray Before Removing Duplicates: ");
  for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);

  for (int i = 0; i < n; i++)
  {
    int j;
    for (j = 0; j < count; j++)
    {
      if (arr[i] == temp[j])
        break;
    }
    if (j == count)
    {
      temp[count] = arr[i];
      count++;
    }
  }

  printf("\nArray After  Removing Duplicates: ");
  for (int i = 0; i < count; i++)
    printf("%d ", temp[i]);

  return 0;
}
```

10
1 2 2 3 4 5 6 7 7 8

Array Before Removing Duplicates: 1 2 2 3 4 5 6 7 7 8
Array After Removing Duplicates: 1 2 3 4 5 6 7 8

Program to check whether a two dimensional array is a Sparse Matrix

studytonight.com/c/programs/array/checking-for-sparse-matrix

A Sparse Matrix is a matrix(two-dimensional array) in which number of 0's is greater than the number of non-zero elements.

Below is a program to check whether a matrix is sparse or not.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n, m, c, d, matrix[10][10];
    int counter = 0;
    printf("\nEnter the number of rows and columns of the matrix \n\n");
    scanf("%d%d",&m,&n);

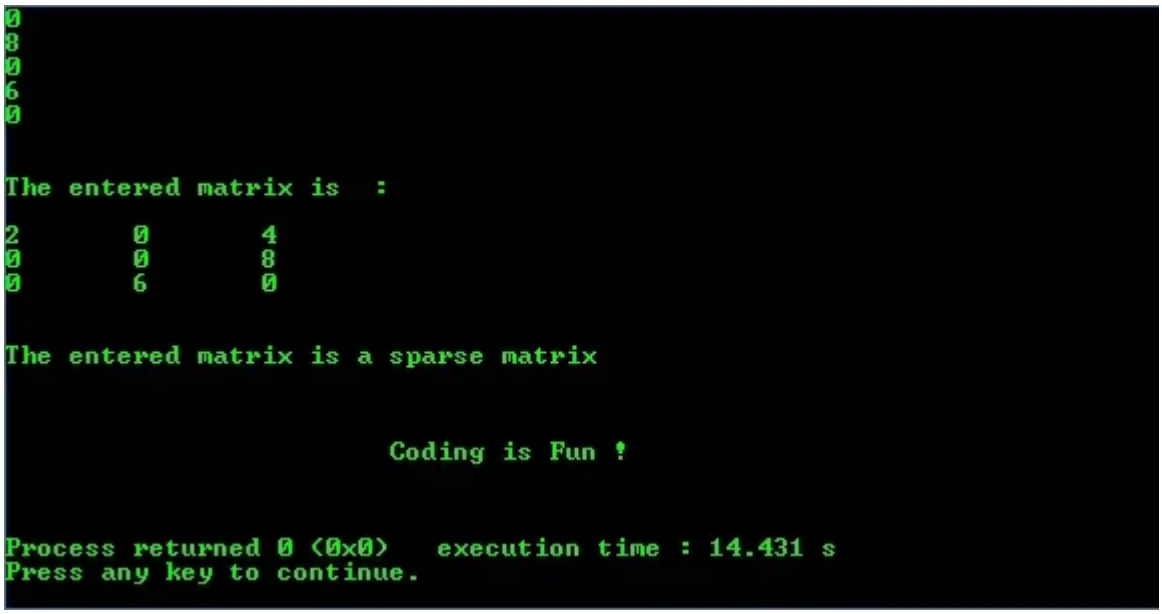
    printf("\nEnter the %d elements of the matrix \n\n", m*n);
    for(c = 0; c < m; c++)   // to iterate the rows
    {
        for(d = 0; d < n; d++)   // to iterate the columns
        {
            scanf("%d", &matrix[c][d]);
            if(matrix[c][d] == 0)
            counter++;  // same as counter=counter +1
        }
    }

    // printing the matrix
    printf("\n\nThe entered matrix is: \n\n");
    for(c = 0; c < m; c++)   // to iterate the rows
    {
        for(d = 0; d < n; d++)   // to iterate the columns
        {
            printf("%d\t", matrix[c][d]);
        }
    printf("\n"); // to take the control to the next row
    }

    // checking if the matrix is sparse or not
    if(counter > (m*n)/2)
        printf("\n\nThe entered matrix is a sparse matrix\n\n");
    else
        printf("\n\nThe entered matrix is not a sparse matrix\n\n");

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


0
8
0
6
0
The entered matrix is :
2 0 4
0 0 8
0 6 0
The entered matrix is a sparse matrix
Coding is Fun !
Process returned 0 (0x0) execution time : 14.431 s
Press any key to continue.

Program to check whether given Square Matrix is symmetric or not

studytonight.com/c/programs/array/check-square-matrix-is-symmetric-or-not

Few important points to remember:

- A Square Matrix is said to be symmetric if it is equal to it's **transpose**.
- Transpose of a matrix is achieved by exchanging **indices** of rows and columns.
- Transpose is only defined for a square matrix.

Below is a program to check whether sqaure matrix is symmetric or not.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int c, d, a[10][10], b[10][10], n, temp;
    printf("\nEnter the dimension of the matrix: \n\n");
    scanf("%d", &n);

    printf("\nEnter the %d elements of the matrix: \n\n",n*n);
    for(c = 0; c < n; c++) // to iterate the rows
        for(d = 0; d < n; d++) // to iterate the columns
            scanf("%d", &a[c][d]);

    // finding transpose of a matrix and storing it in b[][]
    for(c = 0; c < n; c++) // to iterate the rows
        for(d = 0; d < n; d++) //to iterate the columns
            b[d][c] = a[c][d];

    // printing the original matrix
    printf("\n\nThe original matrix is: \n\n");
    for(c = 0; c < n; c++)   // to iterate the rows
    {
        for(d = 0; d < n; d++)   // to iterate the columns
        {
            printf("%d\t", a[c][d]);
        }
    printf("\n");
    }

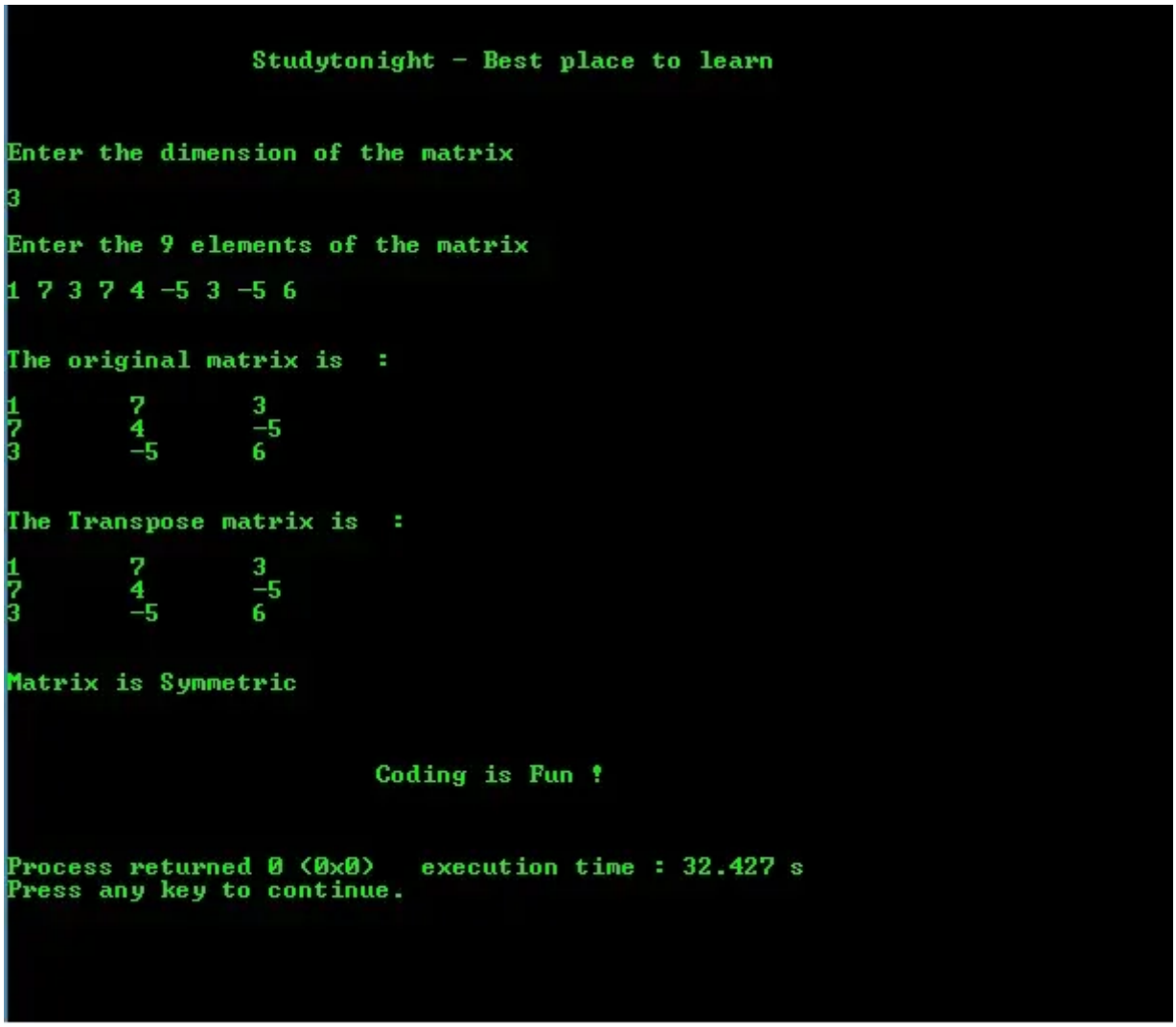
    // printing the transpose of the entered matrix
    printf("\n\nThe Transpose matrix is: \n\n");
    for(c = 0; c < n; c++) // to iterate the rows
    {
        for(d = 0; d < n; d++)   // to iterate the columns
        {
            printf("%d\t", b[c][d]);
        }
        printf("\n");
    }

    // checking if the original matrix is same as its transpose
    for(c = 0; c < n; c++)   // to iterate the rows
    {
        for(d = 0; d < n; d++)   // to iterate the columns
        {
            /*
                even if they differ by a single element,
                the matrix is not symmetric
            */
            if(a[c][d] != b[c][d])
            {
                printf("\n\nMatrix is not Symmetric\n\n");
                exit(0);    // a system defined method to terminate the program
            }
```

```
        }
    }

    /*
        if the program is not terminated yet,
        it means the matrix is symmetric
    */
    printf("\n\nMatrix is Symmetric\n\n");
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to find Deteminant of 2x2 Matrix

studytonight.com/c/programs/array/determinant-of-2x2-matrix

Below is a program to find the **determinant** of a 2x2 matrix.

Please note that, when we say a 2x2 matrix, we mean an array of 2x2.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int a[2][2], i, j;
    long determinant;

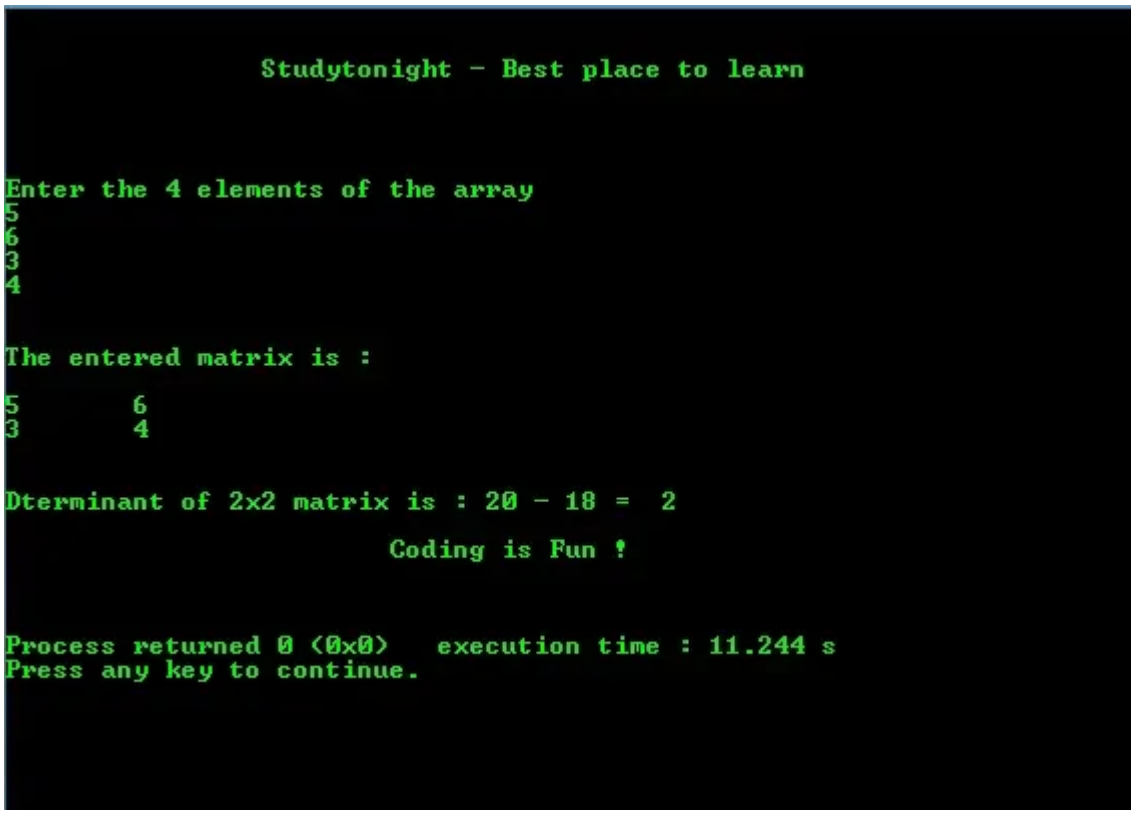
    printf("\n\nEnter the 4 elements of the array\n");
    for(i = 0; i < 2; i++)
    for(j = 0; j < 2; j++)
    scanf("%d", &a[i][j]);

    printf("\n\nThe entered matrix is: \n\n");
    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 2; j++)
        {
            printf("%d\t", a[i][j]);   // to print the complete row
        }
        printf("\n"); // to move to the next row
    }

    // finding the determinant of a 2x2 matrix
    determinant = a[0][0]*a[1][1] - a[1][0]*a[0][1];
    printf("\n\nDterminant of 2x2 matrix is : %d - %d =  %d", a[0][0]*a[1][1],
a[1][0]*a[0][1], determinant);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the 4 elements of the array
5
6
3
4
The entered matrix is :
5 6
3 4
Dterminant of 2x2 matrix is : 20 - 18 = 2
Coding is Fun !
Process returned 0 (0x0) execution time : 11.244 s
Press any key to continue.

Find Normal and Trace of a Square Matrix in C Language

studytonight.com/c/programs/array/normal-and-trace-of-square-matrix

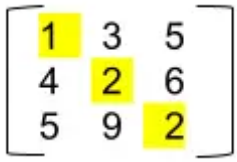
Do you know waht Normal and Trace of a square matrix are? Well before we jump to the program, we should first try to understand how we calculate the Normal and Trace for a Matric.

What are Normal and Trace of a Matrix?

The square root of the sum of the squares of each element of the matrix is said to be **Normal** of a matrix.

The sum of the diagonal elements of a matrix is called **Trace**. Let's consider the following matrix A.

Here, the elements which are highlighted are diagonal elements of the matrix. If we sum them, we will find trace of the matrix as follows :-



Trace of A = 1 + 2 + 2 = 5

To find the normal, we will square each element and find sum of those squared elements. After that, we will calculate square root of the sum calculated. The resulting number will be Normal of our matrix. Let's calculate Normal for A.

Normal = $(\sqrt{(1^2 + 3^2 + 5^2 + 4^2 + 2^2 + 6^2 + 5^2 + 9^2 + 2^2)})$

Normal = $(\sqrt{(195)})$

Normal = 13.96

Few important points to remember:

- **Normal** and **Trace** are only defined for a square matrix.
- **Square Matrix:** Matrix in which, the **number of rows = number of columns**.
- **Diagonal Element:** An element having same indices for row and column.

Algorithm for Normal and Trace of a Square Matrix Program in C

To find Trace of a matrix, the algorithm is as below:

1. Let the matrix be **A[m][n]**
2. trace = 0
3. for i = 0 to m
4. for j = 0 to n
5. if i == j
6. trace += A[i][j]

The algorithm for finding Normal of a matrix is as follows:

1. Let the matrix be **A[m][n]**
2. sum = 0, normal = 0
3. for i = 0 to m
4. for j = 0 to n
5. A[i][j] *= A[i][j], sum += A[i][j]
6. normal = sqrt(sum)

C Program to find Normal and Trace of Square Matrix

So let's see the complete program now. We are using the **math.h** header library in this program.

In some latest versions of C compilers (gcc), you have to **explicitly specify** that you want to use **math.h** header in the program, when you compile it.

You can do so by adding the `-lm` flag at the end of the gcc command. For example, `gcc program.c -o program -lm`

So when you run this code in your local setup, remember this.

```
#include<stdio.h>
/*
    to use the sqrt method to find
    the square root of a number we include
    math.h header file
*/
#include<math.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int i, j, n, aj[10][10], sum = 0, sum1 = 0, a = 0, normal;

    printf("\nEnter the number of rows (columns) of the matrix: \n\n");
    scanf("%d", &n);

    printf("\nEnter the %d elements of the first matrix: \n\n", n*n);

    for(i = 0; i < n; i++)   // to iterate the rows
    {
        for(j = 0; j < n; j++)   // to iterate the columns
        {
            scanf("%d", &aj[i][j]);
            a = aj[i][j]*aj[i][j];  // finding square of each element
            sum1 += a;  // same as sum1 = sum1 + a
        }
    }
    normal = sqrt((double)sum1);    // typecasting to double value

    printf("\n\nThe normal of the given matrix is: %d", normal);
    for(i = 0; i < n; i++)
    {
        sum = sum + aj[i][i];   // sum of the diagonal elements
    }
    printf("\n\nThe Trace of the given matrix is: %d", sum);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Program Output:

The output of the program will be,

```
                Studytonight - Best place to learn

Normal and Trace are only defined for a square matrix.

Square matrix - a matrix in which the no. of rows = no. of columns.

Normal : Square root of the sum of the squares of each element of the matrix.

Trace : Sum of the diagonal elements of a matrix.

Diagonal element : An element having same indices for row and column

Enter the number of rows (columns) of the matrix

3

Enter the 9 elements of the first matrix

1 2 3 5 6 0 4 7 8

The normal of the given matrix is  :   14

The Trace of the given matrix is  :   15

                        Coding is Fun !


Process returned 0 (0x0)   execution time : 25.592 s
Press any key to continue.
```

Program to perform addition and subtraction of Matrices

studytonight.com/c/programs/array/addition-and-subtraction-of-matrices

Below is a program to perform Addition and Subtraction on two matrices.

`\n` is used to take the control to the next row.

`\t` is used to take control 5 spaces(tab) ahead.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int n, m, c, d, first[10][10], second[10][10], sum[10][10], diff[10][10];
    printf("\nEnter the number of rows and columns of the first matrix \n\n");
    scanf("%d%d", &m, &n);

    printf("\nEnter the %d elements of the first matrix \n\n", m*n);
    for(c = 0; c < m; c++)   // to iterate the rows
        for(d = 0; d < n; d++)   // to iterate the columns
            scanf("%d", &first[c][d]);

    printf("\nEnter the %d elements of the second matrix \n\n", m*n);
    for(c = 0; c < m; c++)   // to iterate the rows
        for(d = 0; d < n; d++)   // to iterate the columns
            scanf("%d", &second[c][d]);

    /*
        printing the first matrix
    */
    printf("\n\nThe first matrix is: \n\n");
    for(c = 0; c < m; c++)   // to iterate the rows
    {
        for(d = 0; d < n; d++)   // to iterate the columns
        {
            printf("%d\t", first[c][d]);
        }
    printf("\n");
    }

    /*
        printing the second matrix
    */
    printf("\n\nThe second matrix is: \n\n");
    for(c = 0; c < m; c++)   // to iterate the rows
    {
        for(d = 0; d < n; d++)   // to iterate the columns
        {
            printf("%d\t", second[c][d]);
        }
    printf("\n");
    }

    /*
        finding the SUM of the two matrices
        and storing in another matrix sum of the same size
    */
    for(c = 0; c < m; c++)
        for(d = 0; d < n; d++)
            sum[c][d] = first[c][d] + second[c][d];

    // printing the elements of the sum matrix
    printf("\n\nThe sum of the two entered matrices is: \n\n");
```

```
    for(c = 0; c < m; c++)
    {
        for(d = 0; d < n; d++)
        {
            printf("%d\t", sum[c][d]);
        }
        printf("\n");
    }

    /*
        finding the DIFFERENCE of the two matrices
        and storing in another matrix difference of the same size
    */
    for(c = 0; c < m; c++)
        for(d = 0; d < n; d++)
            diff[c][d] = first[c][d] - second[c][d];

    // printing the elements of the diff matrix
    printf("\n\nThe difference(subtraction) of the two entered matrices is:
\n\n");
    for(c = 0; c < m; c++)
    {
        for(d = 0; d < n; d++)
        {
            printf("%d\t", diff[c][d]);
        }
        printf("\n");
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:

```
                Studytonight - Best place to learn


Enter the number of rows and columns of the first matrix

3
3

Enter the 9 elements of the first matrix

1 4 3 6 7 4 0 8 4

Enter the 9 elements of the second matrix

2 4 5 9 6 7 0 3 4


The first matrix is  :

1       4       3
6       7       4
0       8       4


The second matrix is  :

2       4       5
9       6       7
0       3       4


The sum of the two entered matrices is :

3       8       8
15      13      11
0       11      8


The difference(subtraction) of the two entered matrices is :

-1      0       -2
-3      1       -3
0       5       0


                        Coding is Fun !


Process returned 0 (0x0)   execution time : 54.887 s
Press any key to continue.
```

Program for Matrix Multiplication in C

studytonight.com/c/programs/array/matrix-multiplication-program

Before we see the program for matrix multiplication in C, let's first know what are matrix in C and what are the different operations that we can perform on matrix in C.

What is a Matrix?

A two dimensional array is called a **matrix**. For example,

```
int arr[4][5];
float marks[5][2];
char ch[4][6];

// define just the number of columns and not rows
int arrrr[][3];
```

While declaring a matrix, we always define the **number of columns** even if we leave the number of rows empty. This is the default syntax for declaring a matrix in C. To learn more about this, check out our tutorial on Arrays in C.

Types of Matrix Operations

We can perform **addition**, **subtraction**, **multiplication** and **division** operations on matrix. Out of all those, multiplication is the most complex. Before we dive into that, let's see how we can **calculate sum of individual rows** of a matrix.

```
#include <stdio.h>
int main()
{
     int arr[2][3], total[2];
     for(int i = 0; i < 2; i++) {
         for(int j = 0; j < 3; j++){
              scanf("%d", &arr[i][j]);
         }
     }
     for(int i = 0; i < 2; i++) {
         int sum = 0;
         for(int j = 0; j < 3; j++){
              sum += arr[i][j];
         }
         total[i] = sum;
     }
     for(int i = 0; i < 2; i++)
         printf("%d : %d\n", i + 1, total[i]);

     return 0;
}
```

1 2 3
1 2 4
1 : 6
2 : 7

Matrix Multiplication in C Program

We may have to multiply a matrix with a number (scalar value) or we may have to multiply two matrices. So let's explore both the scenarios.

1. Multiplying Matrix with a scalar value

Matrix multiplication with a single number is easy and can be done as follows:

```
#include <stdio.h>
int main()
{
    int arr[2][2], n;
    // enter the scalar value
    scanf("%d", &n);

    // input the matrix values
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++){
            scanf("%d", &arr[i][j]);
        }
    }
    // multiply every value of matrix with scalar value
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++){
            arr[i][j] = arr[i][j] * n;
        }
    }

    // print the updated matrix values
    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 2; j++){
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

5
1 2 3 4
5 10
15 20

2. Multiplication of Two Matrices

There are two things we have to keep in mind while performing matrix multiplication on two matrices:

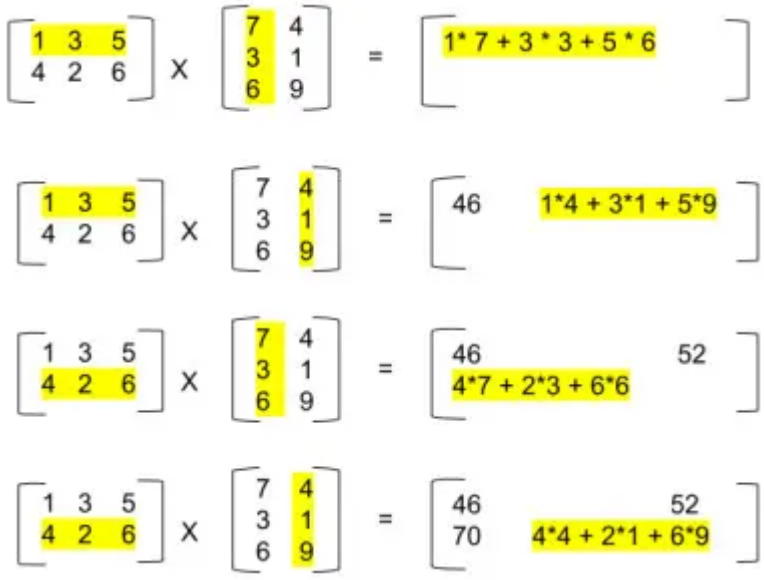
1. The number of columns of the first matrix should be equal to the number of rows of the second matrix.

2. The resultant matrix will have the same number of rows as the first matrix and the same number of columns as the second matrix.

Suppose we have two matrices:

```
A = {{1, 3, 5},
     {4, 2, 6}};
B = {{7, 4},
     {3, 1},
     {6, 9}};
```

To multiply these two matrices, we will perform **dot product** on their rows and columns, means the first element of the firast row of A will be multiplied to the first element of first column of B, the second element of first row of A will be multiplied to the second element of first column of B, and so on.



Algorithm for Matrix Multiplication in C

The standard algorithm for matrix multiplication is as follows:

Let **A** be a **m * k matrix**, **B** be a **k * n matrix**. We will store our result in **C[m][n]**.

```
for i = 1 to m
     for j = i to n
          c[i][j] = 0
          for p = 1 to k
               C[i][j] = C[i][j] + A[i][p] * B[p][j]
```

Below is a program on Matrix Multiplication:

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int n, m, c, d, p, q, k, first[10][10], second[10][10], pro[10][10],sum =
0;

    printf("\nEnter the number of rows and columns of the first matrix:
\n\n");
    scanf("%d%d", &m, &n);

    printf("\nEnter the %d elements of the first matrix: \n\n", m*n);
    for(c = 0; c < m; c++)   // to iterate the rows
        for(d = 0; d < n; d++)   // to iterate the columns
            scanf("%d", &first[c][d]);

    printf("\nEnter the number of rows and columns of the first matrix:
\n\n");
    scanf("%d%d", &p, &q);

    if(n != p)
        printf("Matrices with the given order cannot be multiplied with each
other.\n\n");

    else    // matrices can be multiplied
    {
        printf("\nEnter the %d elements of the second matrix: \n\n",m*n);

        for(c = 0; c < p; c++)   // to iterate the rows
            for(d = 0; d < q; d++)   // to iterate the columns
                scanf("%d", &second[c][d]);

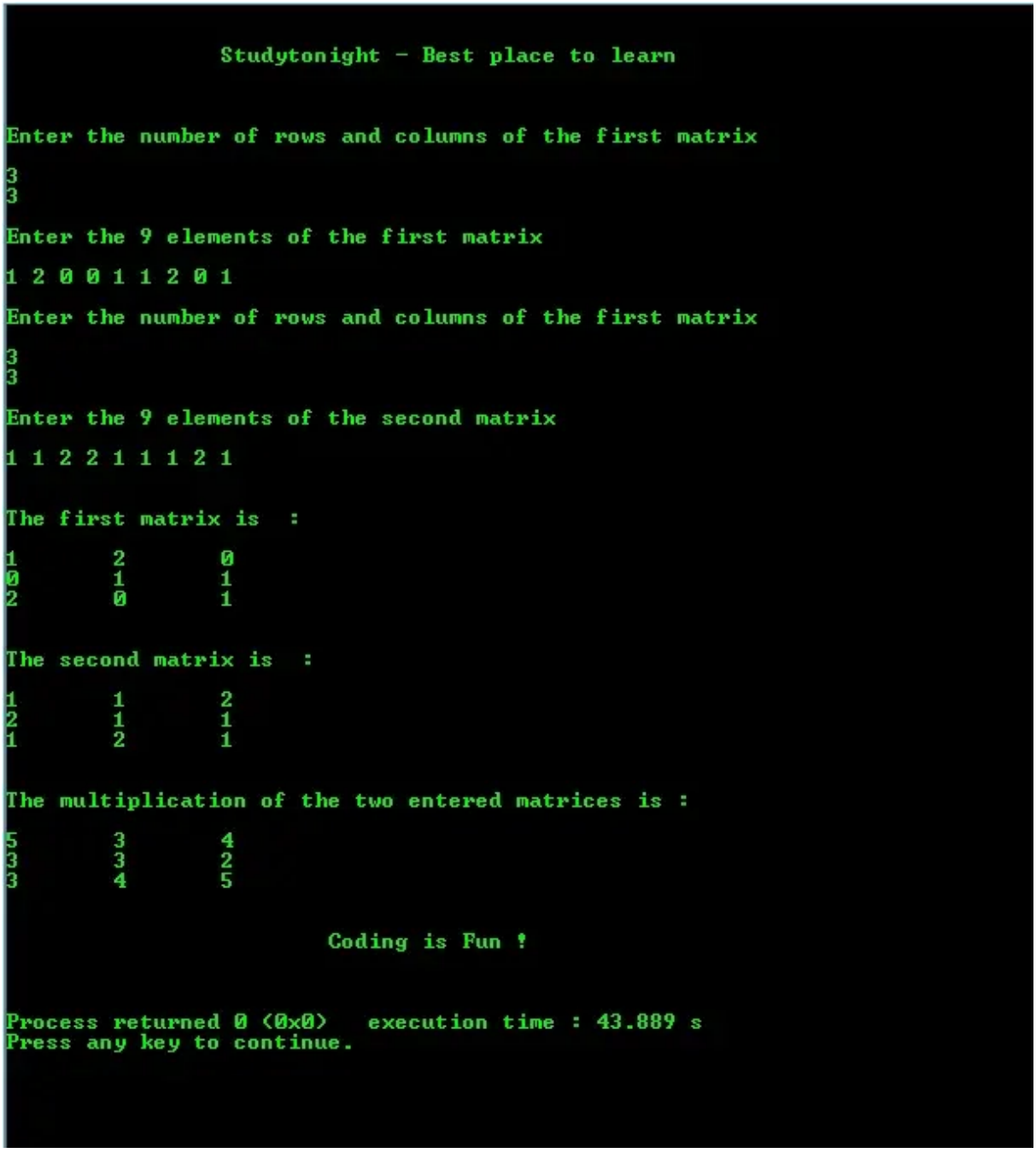
        // printing the first matrix
        printf("\n\nThe first matrix is: \n\n");
        for(c = 0; c < m; c++)   // to iterate the rows
        {
            for(d = 0; d < n; d++)   // to iterate the columns
            {
                printf("%d\t", first[c][d]);
            }
            printf("\n");
        }

        // printing the second matrix
        printf("\n\nThe second matrix is: \n\n");
        for(c = 0; c < p; c++)   // to iterate the rows
        {
            for(d = 0; d < q; d++)   // to iterate the columns
            {
                printf("%d\t", second[c][d]);
            }
            printf("\n");
        }
```

```
        for(c = 0; c < m; c++)   // to iterate the rows
        {
            for(d = 0; d < q; d++)   // to iterate the columns
            {
                for(k = 0; k < p; k++)
                {
                    sum = sum + first[c][k]*second[k][d];
                }
            pro[c][d] = sum;    // resultant element of pro after
multiplication
            sum = 0;    // to find the next element from scratch
            }
        }

        // printing the elements of the product matrix
        printf("\n\nThe multiplication of the two entered matrices is: \n\n");
        for(c = 0; c < m; c++)   // to iterate the rows
        {
            for(d = 0; d < q; d++)   // to iterate the columns
            {
                printf("%d\t", pro[c][d]);
            }
            printf("\n"); // to take the control to the next row
        }
    }
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Program Output:


Studytonight - Best place to learn
Enter the number of rows and columns of the first matrix
3
3
Enter the 9 elements of the first matrix
1 2 0 0 1 1 2 0 1
Enter the number of rows and columns of the first matrix
3
3
Enter the 9 elements of the second matrix
1 1 2 2 1 1 1 2 1
The first matrix is :
1 2 0
0 1 1
2 0 1
The second matrix is :
1 1 2
2 1 1
1 2 1
The multiplication of the two entered matrices is :
5 3 4
3 3 2
3 4 5
Coding is Fun !
Process returned 0 (0x0) execution time : 43.889 s
Press any key to continue.

Basic Program for Pointers

studytonight.com/c/programs/pointer/simple-pointer-program

Few important points to remember:

- `*` is used to access the value stored in the pointer variable.
- `&` is used to store the address of a given variable.

Here is the C language tutorial on Pointers in C → Pointers in C

Below is a simple program on pointer.

`int *p;` is a pointer variable declaration where `p` is a pointer to an `int` variable i.e. it stores the location of an integer.

`%x` is a format specifier to print hexadecimal value. It is usually used to print the location.

```
#include <stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int var = 24;   // actual variable declaration
    int *p;

    p = &var;   // storing address of int variable var in pointer p

    printf("\n\nAddress of var variable is: %x \n\n", &var);

    // address stored in pointer variable
    printf("\n\nAddress stored in pointer variable p is: %x", p);

    // access the value using the pointer variable
    printf("\n\nValue of var variable or the value stored at address p is   %d ",
*p);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Address of var variable is : 28ff08
Address stored in pointer variable p is : 28ff08
Value of var variable or the value stored at address p is 24
Coding is Fun !
Process returned 0 (0x0) execution time : -0.000 s
Press any key to continue.

Program for Dynamic Memory Allocation using malloc()

studytonight.com/c/programs/pointer/memory-management-program

Below is a program on dynamic memory allocation using `malloc()` and clearing out memory space using `free()`.

`sizeof()` returns the number of bytes occupied by any datatype, in this case by an integer.

```
#include <stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n, i, *ptr, sum = 0;

    printf("\n\nEnter number of elements: ");
    scanf("%d", &n);

    // dynamic memory allocation using malloc()
    ptr = (int *) malloc(n*sizeof(int));

    if(ptr == NULL) // if empty array
    {
        printf("\n\nError! Memory not allocated\n");
        return 0;   // end of program
    }

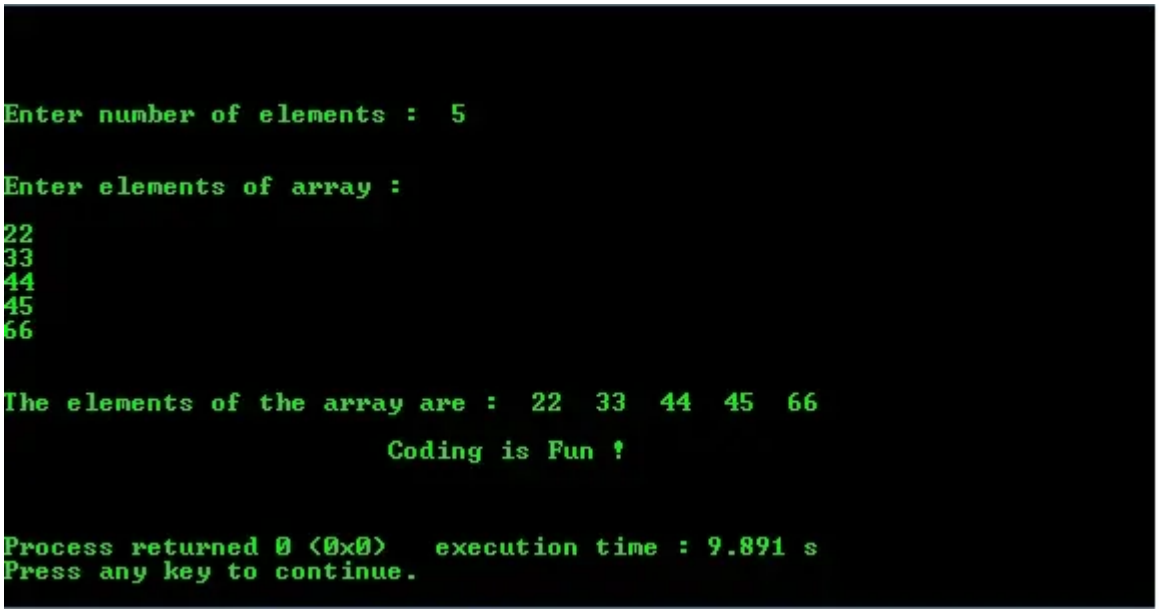
    printf("\n\nEnter elements of array: \n\n");
    for(i = 0; i < n; i++)
    {
        // storing elements at contiguous memory locations
        scanf("%d", ptr+i);
        sum = sum + *(ptr + i);
    }

    // printing the array elements using pointer to the location
    printf("\n\nThe elements of the array are: ");
    for(i = 0; i < n; i++)
    {
        printf("%d  ",ptr[i]);    // ptr[i] is same as *(ptr + i)
    }

    /*
        freeing memory of ptr allocated by malloc
        using the free() method
    */
    free(ptr);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Enter number of elements : 5
Enter elements of array :
22
33
44
45
66
The elements of the array are : 22 33 44 45 66
Coding is Fun !
Process returned 0 (0x0) execution time : 9.891 s
Press any key to continue.

Program to access Array of int Pointers

studytonight.com/c/programs/pointer/array-of-pointers

Below is a program to access an array of `int` pointers:

```
#include <stdio.h>
/*
    Global declaration.
    Value of a const variable cannot be changed
    throughout the execution of program
*/
const int MAX = 5;

int main()
{

    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int var[]={10, 20, 30, 40, 50}; // initializing an array(here var) of int
pointers
    int i = 0;

    /*
        ptr is an array of int pointers i.e.
        it stores the address of each array element
    */
    int *ptr[MAX];

    for(i = 0; i < MAX; i++)
    {
        /*
            Assign the address of each of the array
            element to the ptr array
        */
        ptr[i] = &var[i];
    }

    for(i = 0; i < MAX; i++)
    {
        /*
            ptr[i] stores the address of the element var[i].
            Hence, *ptr[i] returns the value of the element
            stored at location ptr[i]
        */
        printf("Value of var[%d] = %i\n\n", i, *ptr[i]);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Explanation:

```
printf("Value of var[%d] = %i\n\n", i, *ptr[i]);
```

> Here `ptr[i]` stores the address of the element `var[i]`. Hence, `*ptr[i]` returns the value of the element stored at location `var[i]`.

Program to access Array of `char` pointers

Array of `char` pointers is used to access the complete string just using the address of the first char(base address) of each string.

Below is a program to access an array of `char` pointers.

```
#include <stdio.h>
const int MAX = 4;  // Global declaration.

int main()
{

        printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char *names[] = {"Google", "Amazon", "Facebook", "Apple"}; // initializing
an array(here names) of char pointers
    int i = 0;

    for(i = 0; i < MAX; i++)
    {
        printf("Value of names[%d] = %s\n\n", i, names[i]);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Explanation:

```
printf("Value of names[%d] = %s\n\n",i,names[i]);
```

This statement is used for printing the complete name just using the pointer to the first character of each element of the `names` array.

Accessing array elements(Traversing array) by incrementing a Pointer

studytonight.com/c/programs/pointer/pointer-increment-and-decrement

Name of the array refers to the **base address** of the array.

Here we have a tutorial to understand How Pointer arithmetic works?

Below is a program to access elements of an array using pointer increment.

```
#include <stdio.h>

const int MAX = 3;  // Global declaration
int main()
{
        printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
        int var[] = {100, 200, 300};
    int i, *ptr;

    /*
        storing address of the first element
        of the array in pointer variable
    */
    ptr = var;

    for(i = 0; i < MAX; i++)
    {
        printf("\n\n\nAddress of var[%d] = %x ", i, ptr);
        printf("\nValue of var[%d] = %d ", i, *ptr);

       // move to the next location
        ptr++;
    }
        printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Traversing array elements by decrementing a Pointer

Below is a program to access elements of an array using pointer decrement.

```
#include <stdio.h>

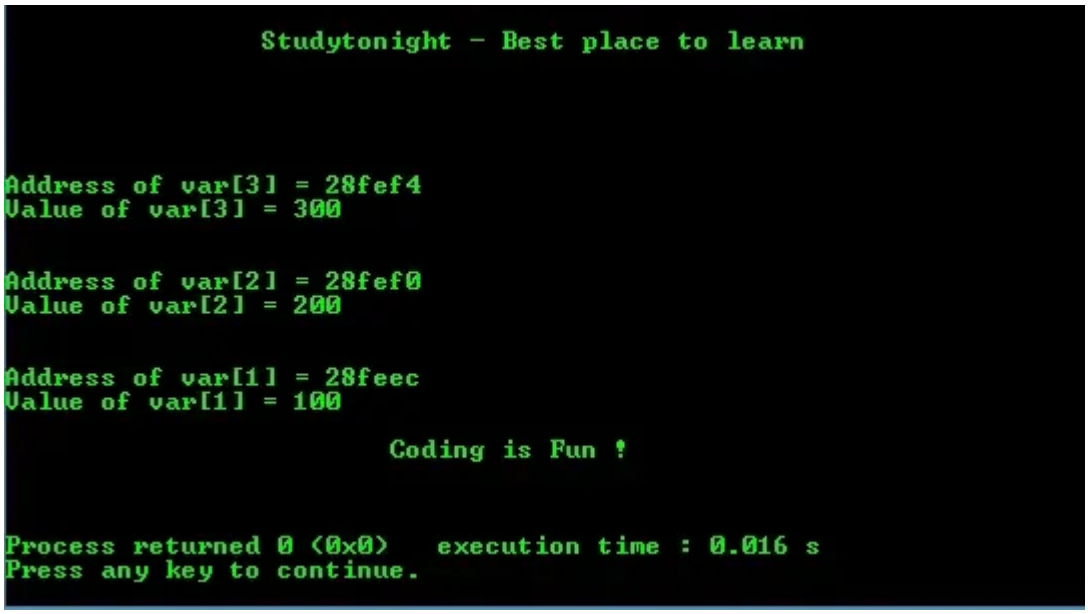
const int MAX = 3;  // Global declaration
int main()
{
        printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
        int var[] = {100, 200, 300};
    int i, *ptr;

    /*
        storing address of the last element
        of the array in pointer variable
    */
    ptr = &var[MAX-1];

        for(i = MAX; i > 0; i--)
    {
        printf("\n\n\nAddress of var[%d] = %x ", i, ptr);
        printf("\nValue of var[%d] = %d ", i, *ptr);

                // move to the previous location
        ptr--;
        }
        printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Address of var[3] = 28fef4
Value of var[3] = 300
Address of var[2] = 28fef0
Value of var[2] = 200
Address of var[1] = 28feec
Value of var[1] = 100
Coding is Fun !
Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

Pointer Comparison in C

studytonight.com/c/programs/pointer/pointer-comparison-program

In C language pointers can be compared if the two pointers are pointing to the same array.

All relational operators can be used for pointer comparison, but a pointer cannot Multiplied or Divided.

Below is a program on pointer comparison for same type of pointer:

```
#include <stdio.h>

int main()
{
    int *ptrA,*ptrB;

    ptrA = (int *)1;
    ptrB = (int *)2;

    if(ptr2 > ptr1)
        printf("PtrB is greater than ptrA");

return(0);
}
```

Below is a program on pointer comparison for different type of pointer:

```
#include <stdio.h>

int main()
{
    int *ptrA;
    float *ptrB;

    ptrA = (int *)1000;
    ptrB = (float *)2000;

    if(ptrB > ptrA)
       printf("PtrB is greater than ptrA");

    return(0);
}
```

Basic C Program for Pointer to a Pointer

studytonight.com/c/programs/pointer/pointer-to-a-pointer

A pointer variable stores the address of a value. Similarly, a pointer to a pointer stores the address of the pointer variable. Pointer to a pointer is executed making use of `**`.

Below is a program on pointer to a pointer.

`int var;` is a integer variable which stores value.

`int *ptr;` is a pointer variable which stores the address of an integer variable.

`int **pptr;` is a pointer to a pointer variable which stores the address of a pointer variable.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int var;
    int *ptr;
    int **pptr;

    var = 50;

    // take the address of the variable var
    ptr = &var;

    // taking the address of ptr using address of operator-&
    pptr = &ptr;

    // take the value using the pptr
    printf("\n\nValue of var = %d\n\n", var);

    printf("\n\nValue available at *ptr = %d\n\n", *ptr);

    printf("\n\nValue available at **pptr = %d\n\n", **pptr);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Value of var = 50
Value available at *ptr = 50
Value available at **pptr = 50
Coding is Fun !
Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

Basic C Program to Concatenate Strings using Pointer

studytonight.com/c/programs/pointer/concatenate-strings-using-pointer

Below is a program to concatenate strings using pointer:

```
#include <stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aa[100], bb[100];

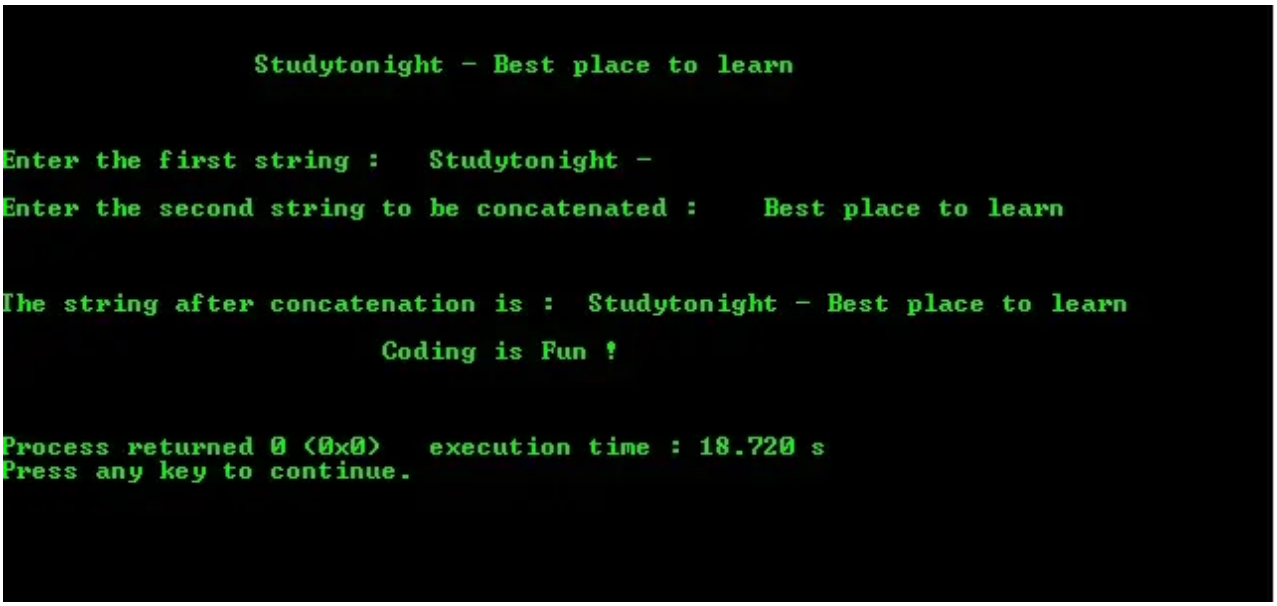
    printf("\nEnter the first string: ");
    gets(aa);   // inputting first string

    printf("\nEnter the second string to be concatenated: ");
    gets(bb);   // inputting second string

    char *a = aa;
    char *b = bb;

    // pointing to the end of the 1st string
    while(*a)   // till it doesn't point to NULL-till string is not empty
    {
        a++;    // point to the next letter of the string
    }
    while(*b)   // till second string is not empty
    {
        *a = *b;
        b++;
        a++;
    }
    *a = '\0';  // string must end with '\0'
    printf("\n\n\nThe string after concatenation is: %s ", aa);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Explanation:

- In `char *a = aa;` , `a` is a pointer to a **char** array and stores the base address of the `aa` .
- In `char *b = bb;` , `b` is a pointer to a **char** array and stores the base address of the `bb` .
- And `*a = *b;` is storing the base address of second at the end of the first.

C Program to Reverse a String using Pointer

studytonight.com/c/programs/pointer/reverse-a-string-using-pointer

Below is a program to reverse a string using pointer:

```c
#include <stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char str[100];
    char rev[100];
    char *sptr = str; // sptr stores the base address of the str
    char *rptr = rev; // rptr stores the base address of the reverse

    int i = -1;

    printf("\n\nEnter a string: ");
    scanf("%s", str);

    // storing the ending address of str in sptr
    while(*sptr)
    {
        sptr++;
        i++; // i is the index of the end location
    }

    // storing the string str in rev in reverse order
    while(i >= 0)
    {
        /*
            First decrementing then using as it stores
            the location after the end location due to above while loop
        */
        sptr--;
        *rptr = *sptr;  // storing the value in sptr in rptr
        rptr++; // pointing to next location
        i--;    // decrementing the index
    }
    /*
        String should always end with '\0' so explicitly
        putting it at the end of the string
    */
    *rptr = '\0';
    rptr = rev; // restoring the base address of the reverse string

    // storing the reverse string in the original string
    while(*rptr)
    {
        *sptr = *rptr;
        sptr++;
        rptr++;
    }

    // printing the reverse string
    printf("\n\nReverse of the string is: %s ", str);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter a string : coding
Reverse of the string is : gnidoc
Coding is Fun !
Process returned 0 (0x0) execution time : 2.125 s
Press any key to continue.

C Program to Swap Two Numbers using Pointers

studytonight.com/c/programs/pointer/swapping-two-numbers-using-pointers

Below is a program to swap two numbers using pointers.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int a, b;
    int *ptra, *ptrb;
    int temp;

    printf("Enter value for a: ");
    scanf("%d", &a);

    printf("\n\nEnter value for b: ");
    scanf("%d", &b);

    printf("\n\nThe values before swapping are: a = %d     b = %d", a, b);

    ptra = &a;    // to store the location of a
    ptrb = &b;    // to store the location of b

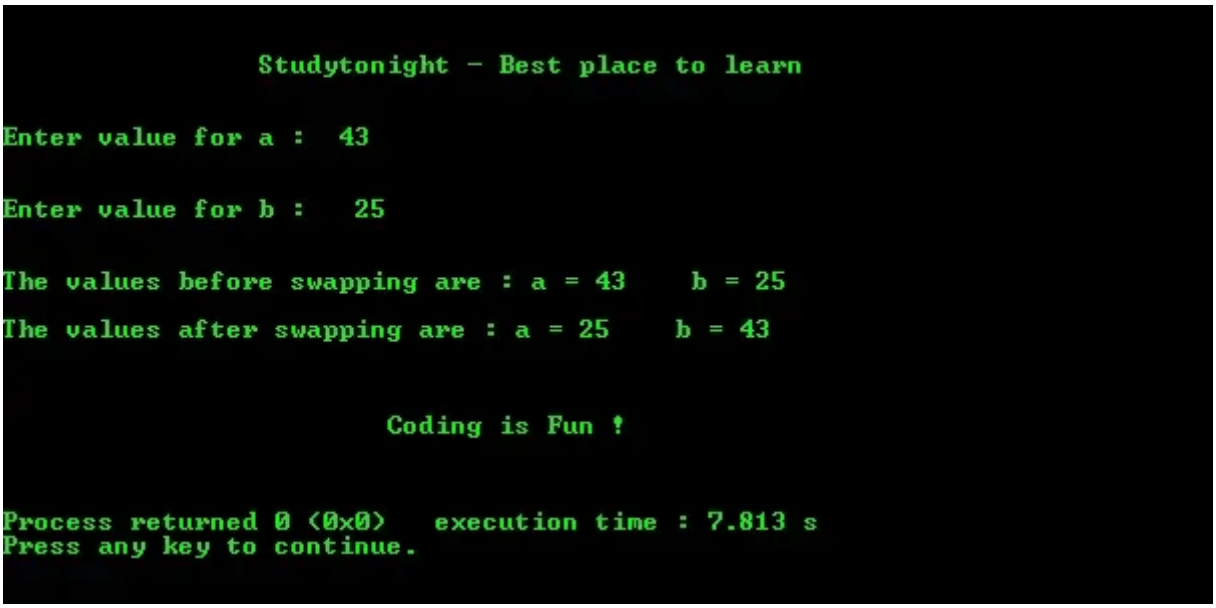
    temp = *ptra;   // temp stores the value at location ptra
    *ptra = *ptrb;  // assigning value at location  ptrb to ptra
    *ptrb = temp;   // assign value of themp to  the variable at location ptrb

    printf("\n\nThe values after swapping are: a = %d    b = %d", a, b);

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;

}
```

Output:


Studytonight - Best place to learn
Enter value for a : 43
Enter value for b : 25
The values before swapping are : a = 43 b = 25
The values after swapping are : a = 25 b = 43
Coding is Fun !
Process returned 0 (0x0) execution time : 7.813 s
Press any key to continue.

C program for Pointer to a Function

studytonight.com/c/programs/pointer/pointer-to-a-function

Below is a program on pointer to a function.

Here is the C language tutorial explaining Pointer with Functions → Pointer to a Function

```
#include<stdio.h>

int func(int a, int b)  // function definition
{
    printf("\n\n a = %d \n", a);
    printf("\n\n b = %d \n", b);
}

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    // function pointer
    int(*fptr)(int , int);

    // assign address to function pointer
    fptr = func;

    // function calling
    func(2,3);
    fptr(2,3);  // calling a function referring to pointer to a function

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
a = 2
b = 3
a = 2
b = 3
Coding is Fun !
Process returned 0 (0x0) execution time : 0.000 s
Press any key to continue.

Using Null Pointer Program

studytonight.com/c/programs/pointer/null-pointer-program

NULL is a macro in C, defined in the **<stdio.h>** header file, and it represent a null pointer constant. Conceptually, when a pointer has that Null value it is not pointing anywhere.

If you declare a pointer in C, and don't assign it a value, it will be assigned a garbage value by the C compiler, and that can lead to errors.

Void pointer is a specific pointer type. `void *` which is a pointer that points to some data location in storage, which doesn't have any specific type.

Don't confuse the `void *` pointer with a NULL pointer.

NULL pointer is a value whereas, **Void pointer** is a type.

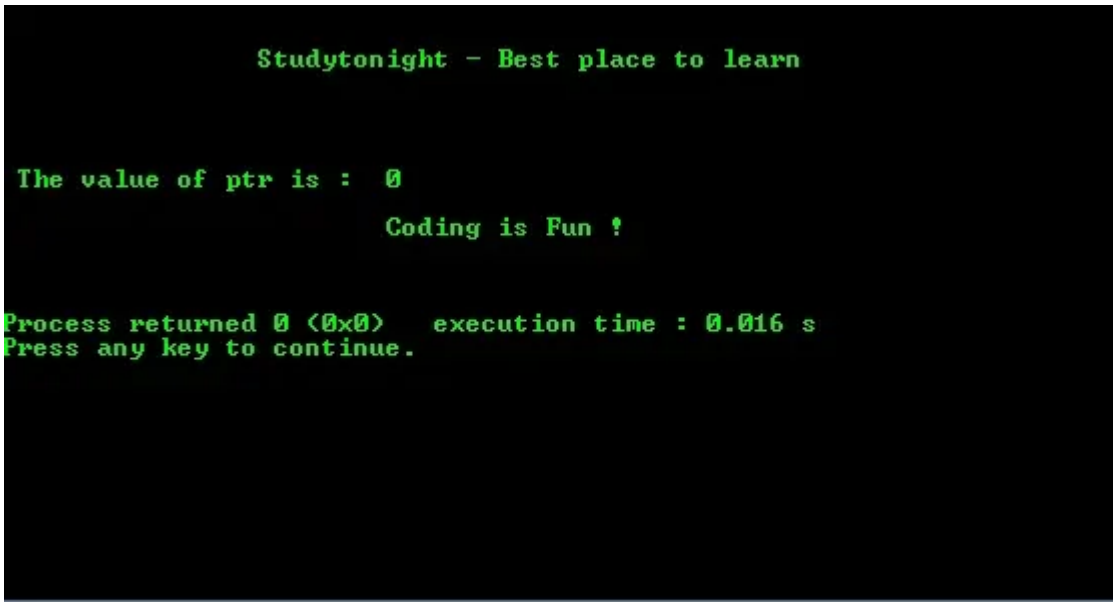
Below is a program to define a NULL pointer.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int *ptr = NULL;    // ptr is a NULL pointer

    printf("\n\n The value of ptr is: %x ", ptr);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Program Output:



Use Null Pointer to mark end of Pointer Array in C

Now let's see a program in which we will use the NULL pointer in a practical usecase.

We will create an array with string values (`char *`), and we will keep the last value of the array as NULL. We will also define a `search()` function to search for name in the array.

Inside the `search()` function, while searching for a value in the array, we will use NULL pointer to identify the end of the array.

So let's see the code,

```
#include <stdio.h>
#include <string.h>

// declaring the search function
int search(char *ptr[], char* name);

char *names[] = {
    "John",
    "Peter",
    "Thor",
    "Chris",
    "Tony",
    NULL
};

int main(void)
{
    if(search(names, "Peter") != 1) {
        printf("Peter is in the list. \n");
    }

    if(search(names, "Scarlett") == -1)  {
        printf("Scarlett not found. \n");
    }

    return 0;
}

// define the search method
int search(char *ptr[], char*name)
{
    register int i;

    for(i=0; ptr[i]; ++i)
    {
        if(!strcmp(ptr[i], name))  return i;

        return -1;  /* name not found */
    }
}
```

Peter is in the list.
Scarlett not found.

This is a simple program to give you an idea of how you can use the NULL pointer. But there is so much more that you can do. You can ask the user to input the names for the array. And then the user can also search for names. So you just have to customize the program a little to make it support user input.

C Program to check lowercase using islower() Function

studytonight.com/c/programs/ctype/islower()-function

In this program we will check whether a character is in lowercase or not. We will be using a system defined function `islower()` defined under the **ctype.h** library. This method is used for various valid password checking options where atleast one lowercase character is mandatory.

The function returns `0` for any character which is not is lowercase, and it returns a value greater than or equal to `1` if the character is lowercase.

Below is a program on `islower()` function.

```
#include<stdio.h>
#include<ctype.h>

int main()
{
    /*
        As datatypes are int, hence all the variables store
        the ASCII values of the character on the rhs
    */
    int var1 = 'D';
    int var2 = '2';
    int var3 = 'a';
    int var4 = ' ';

    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    printf("Range of ASCII values of lowercase character is 97 to 122\n\n");
    /*
        Returns true if the input parameter is an ASCII value
        within the range of a lowercase alphabet
    */
    if(islower(var1))
    {
        printf("\n var1 = |%c| is lowercase character\n", var1);
    }
    else
    {
        printf("\nvar1 =|%c| is not lowercase character\n", var1);
    }

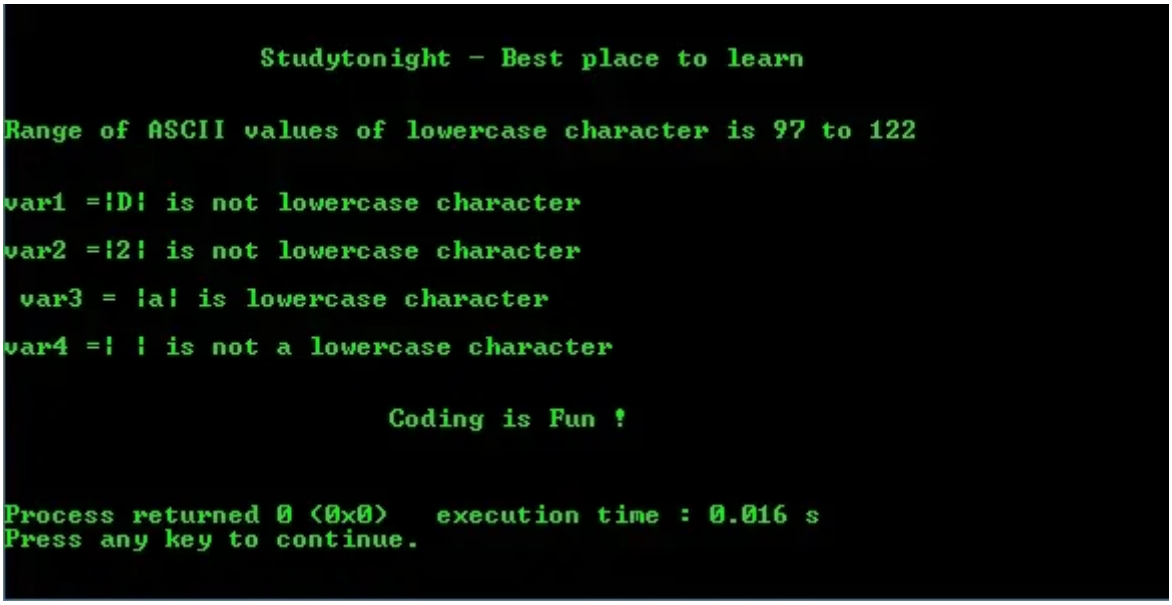
    if(islower(var2))
    {
        printf("\n var2 = |%c| is lowercase character\n", var2);
    }
    else
    {
        printf("\nvar2 =|%c| is not lowercase character\n", var2);
    }

    if(islower(var3))
    {
        printf("\n var3 = |%c| is lowercase character\n", var3);
    }
    else
    {
        printf("\nvar3 =|%c| is not lowercse character\n", var3);
    }

    if(islower(var4))
    {
        printf("\n var4 = |%c| is a lowercase character\n", var4);
    }
    else
    {
        printf("\nvar4 =|%c| is not a lowercase character\n", var4);
    }
```

```
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C Program to check uppercase using isupper() Function

studytonight.com/c/programs/ctype/isupper()-function

In this program we will check whether a character is in upper-case or not. We will be using a system defined function `isupper()` defined under the **ctype.h** library.

This can also be done using if-else blocks condition using the range of the ASCII values of the upper-case alphabets which is **65 to 90**.

This method is used for various valid password checking options where atleast one upper-case character is mandatory.

Below is a program on `isupper()` function.

```
#include<stdio.h>
#include<ctype.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    /*
        As datatypes are int, hence all the variables store
        the ascii values of the character on the rhs
    */
    int var1 = 'D';
    int var2 = 65;
    int var3 = 'a';
    int var4 = 98;

    printf("Range of ASCII values of upper-case characters is 65 to 90\n\n");
    printf("\n\nintegers and characters are interconvertible using the ASCII
values.\n\n\nThe only condition is that correct format specifier shall be used
\nas per what is desired to be printed.\n\n\nThe characters must be initialized in
between single quotes. \' \'\n\n");

    /*
        Returns true if the input parameter is an ascii value
        of a upper-case alphabet
    */
    if(isupper(var1))
    {
        printf("\n var1 = |%c| is a upper-case character\n", var1);
    }
    else
    {
        printf("\n var1 =|%c| is not a upper-case character\n", var1);
    }

    if(isupper(var2))
    {
        printf("\n var2 = |%c| is a upper-case character\n", var2);
    }
    else
    {
        printf("\n var2 =|%c| is not a upper-case character\n", var2);
    }

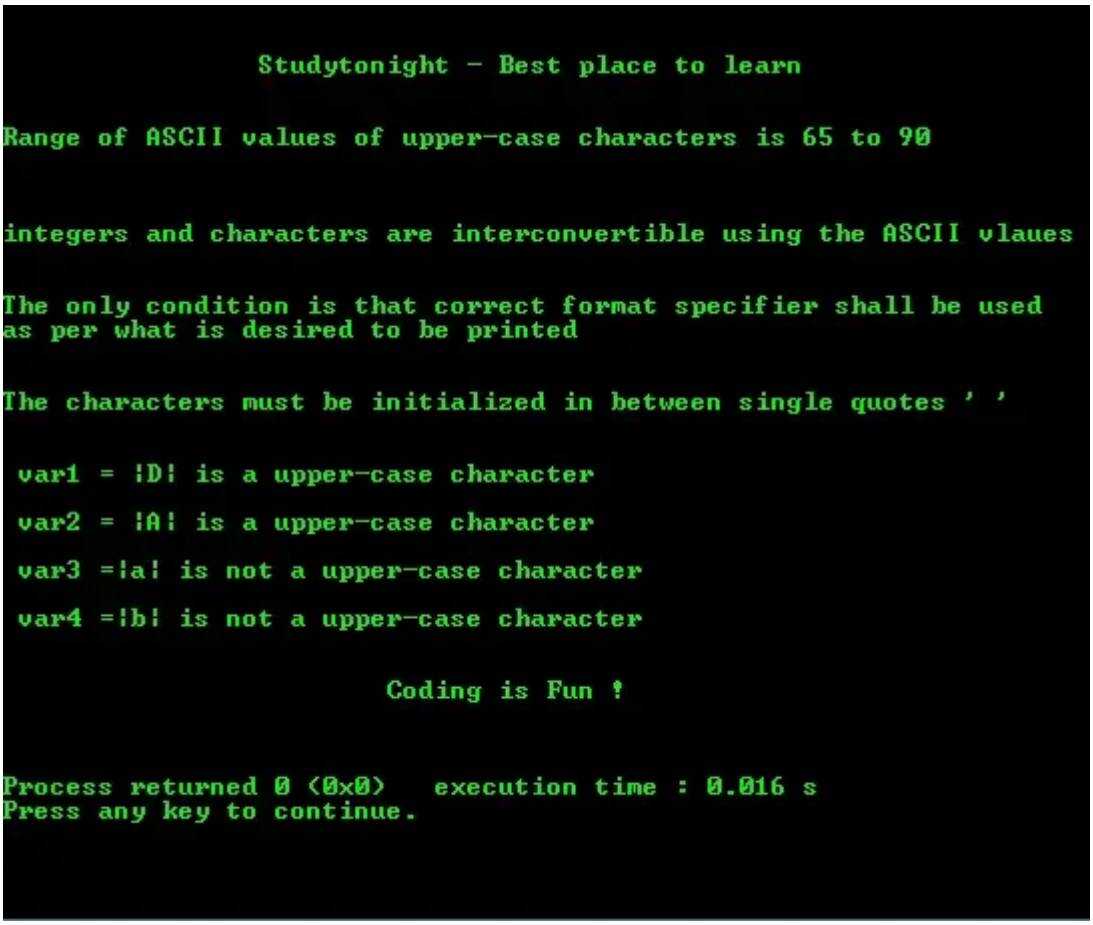
    if(isupper(var3))
    {
        printf("\n var3 = |%c| is a upper-case character\n", var3);
    }
    else
    {
        printf("\n var3 =|%c| is not a upper-case character\n", var3);
    }

    if(isupper(var4))
    {
        printf("\n var4 = |%c| is a upper-case character\n", var4);
    }
```

```
    else
    {
        printf("\n var4 =|%c| is not a upper-case character\n", var4);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C Program to covert character to lowercase using tolower() Function

studytonight.com/c/programs/ctype/tolower()-function

In this program we will convert all the characters of an input string into string of lowercase characters. We will be using a system defined function `tolower()` defined under the **ctype.h** library.

`tolower()` takes a character as an input and converts it into an equivalent lowercase character

`putchar()` is used to print a single character at a time on the console.

Below is the program on `tolower()` function, try running the program and see how the program will print all the information along with the program.

```
#include<stdio.h>
#include<ctype.h>

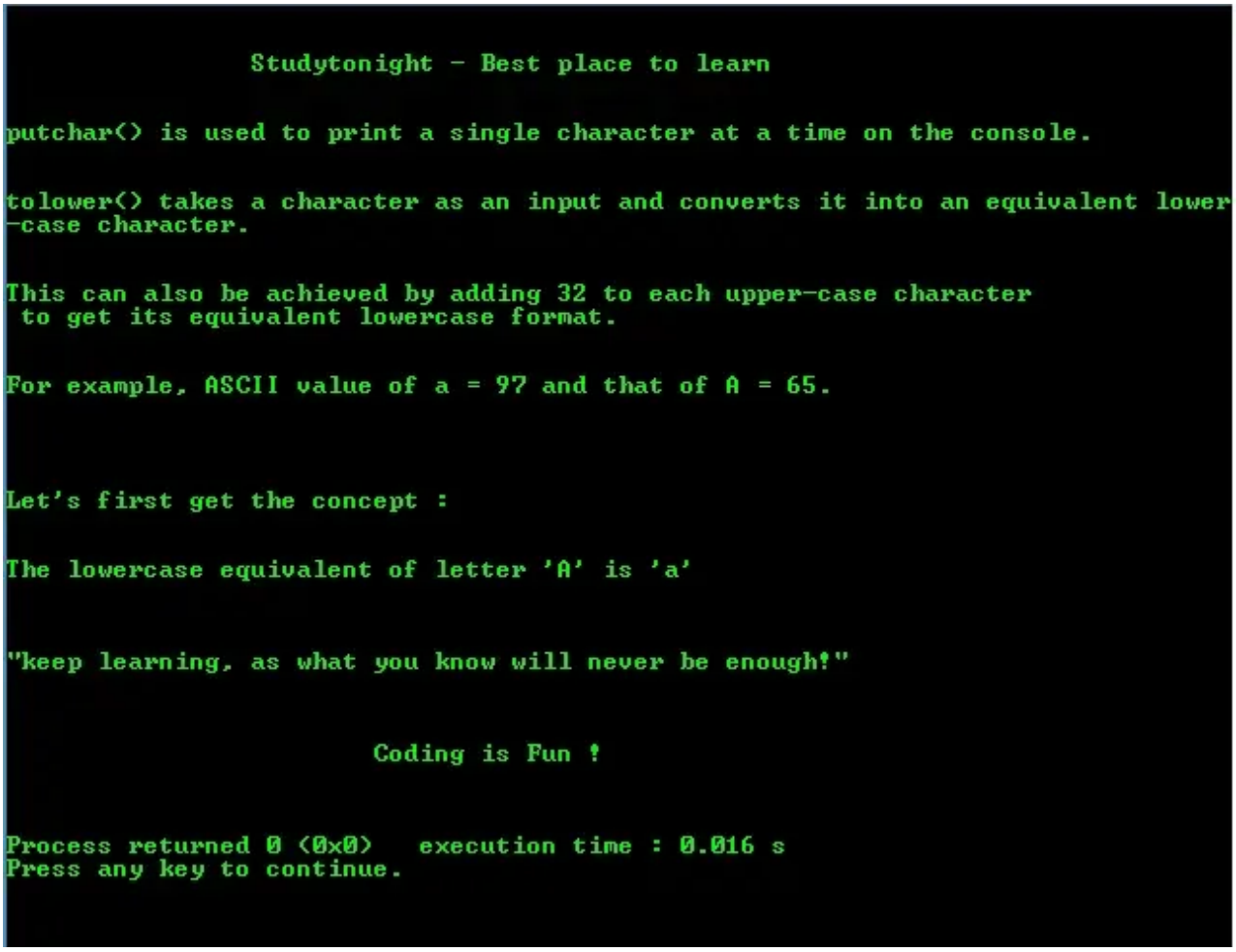
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    printf("putchar() is used to print a single character at a time on the
console.\n\n\n");
    printf("tolower() takes a character as an input and converts it into an
equivalent lower-case character.\n\n\n");
    printf("This can also be achieved by adding 32 to each uppercase character\n
to get its equivalent lowercase format in ASCII. \n\n\n");
    printf("For example, ASCII value of a = %d and that of A = %d.\n\n\n ", 'a',
'A');
    printf("\n\nLet's first get the concept:\n ");
    printf("\n\nThe lowercase equivalent of letter 'A' is '%c' ",'A'+32);

    int aj = 0;
    char mj[] = " \n\n\n\n\"KEEP LEARNING, AS WHAT YOU KNOW WILL NEVER BE
ENOUGH!\"\n\n";    // original uppercase string

    while(mj[aj])   // until string terminates
    {
        // converting one character at a time
        putchar(tolower(mj[aj]));
        aj++;
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
putchar() is used to print a single character at a time on the console.
tolower() takes a character as an input and converts it into an equivalent lower
-case character.
This can also be achieved by adding 32 to each upper-case character
to get its equivalent lowercase format.
For example, ASCII value of a = 97 and that of A = 65.
Let's first get the concept :
The lowercase equivalent of letter 'A' is 'a'
"keep learning, as what you know will never be enough!"
Coding is Fun !
Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

C Program to convert character(s) to uppercase using the toupper() Function

studytonight.com/c/programs/ctype/toupper()-function

In this program we will convert all the characters of an input string to uppercase characters. We will be using a system defined function `toupper()` defined under the **ctype.h** library.

This can also be achieved by subtracting `32` from each lowercase character to get its equivalent uppercase format.

Below is the program on `toupper()` function. Try running the program and you will see all the explanation printed on the output screen. Yay!

```
#include<stdio.h>
#include<ctype.h>

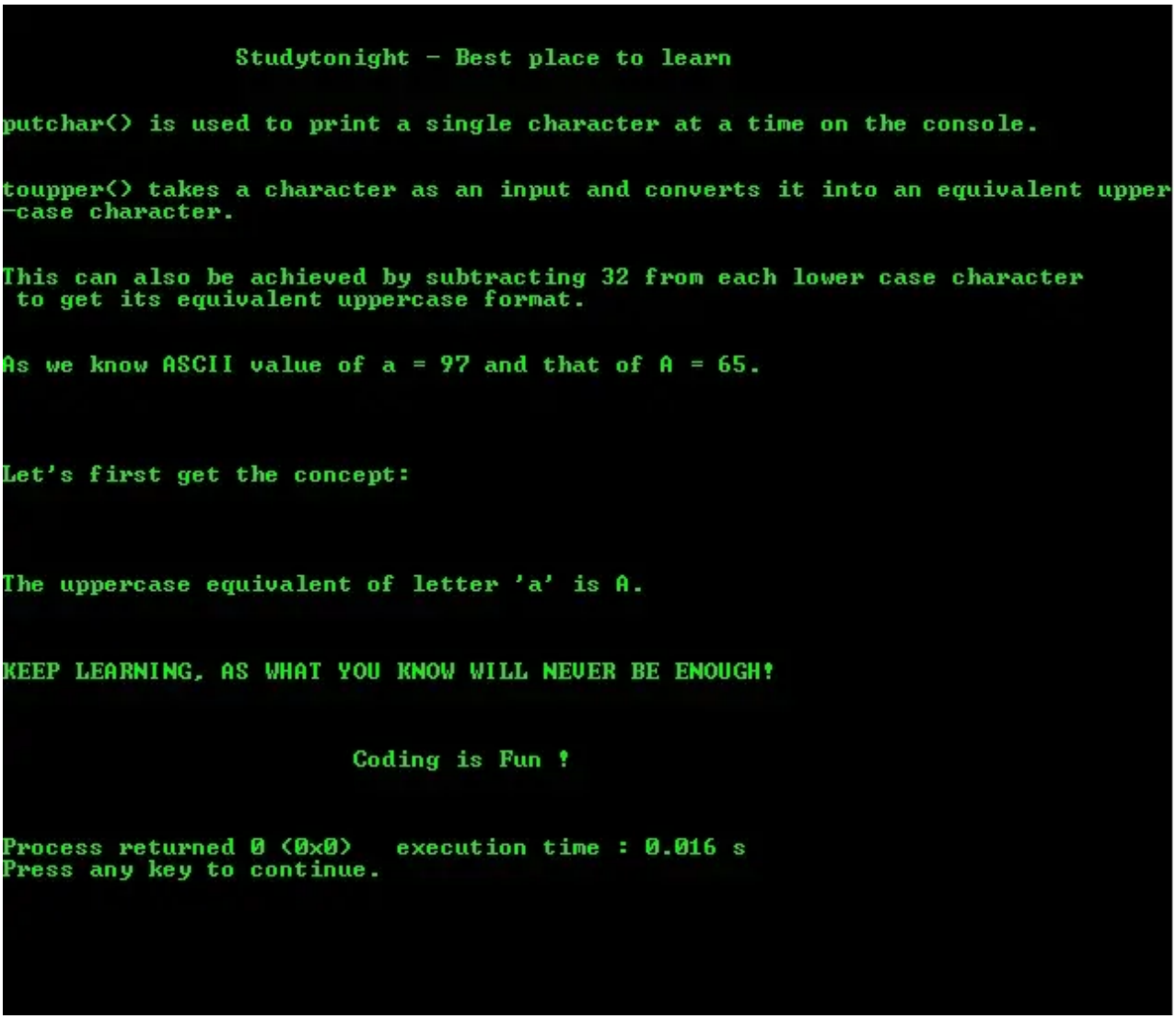
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    printf("putchar() is used to print a single character at a time on the
console.\n\n\n");
    printf("toupper() takes a character as an input and converts it into an
equivalent upper-case character.\n\n\n");
    printf("This can also be achieved by subtracting 32 from each lowercase
character\n to get its equivalent uppercase format. \n\n\n");
    printf("As we know ASCII value of a = %d and that of A = %d.\n\n\n ",'a','A');
    printf("\n\nLet's first get the concept:\n\n\n ");
    printf("\n\nThe uppercase equivalent of letter 'a' is %c.\n\n ",'a'-32);

    int aj = 0;
    char mj[] = " \n\nKeep learning, as what you know will never be enough!\n\n";

    while(mj[aj])   // until string terminates
    {
        // converting one character at a time
        putchar(toupper(mj[aj]));
        aj++;
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
putchar() is used to print a single character at a time on the console.
toupper() takes a character as an input and converts it into an equivalent upper
-case character.
This can also be achieved by subtracting 32 from each lower case character
to get its equivalent uppercase format.
As we know ASCII value of a = 97 and that of A = 65.
Let's first get the concept:
The uppercase equivalent of letter 'a' is A.
KEEP LEARNING, AS WHAT YOU KNOW WILL NEVER BE ENOUGH!
Coding is Fun !
Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

C Program to check if character is alphabet using isalpha() Function

studytonight.com/c/programs/ctype/isalpha()-function

In this program we will check if a character is an alphabet or not. We will be using a system defined function `isalpha()` defined under the **ctype.h** library.

This can also be done using if-else blocks condition using the range of the ASCII values of the alphabets. This method is used for various valid password checking options.

Below is a program on `isalpha()` function.

```
#include<stdio.h>
#include<ctype.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    /*
        As datatypes are int, hence all the variables store
        the ASCII values of the character on the rhs
    */
    int var1 = 'D';
    int var2 = '2';
    int var3 = 'a';
    int var4 = ' ';

    /*
        Returns true if the input parameter is an ASCII value
        of an alphabet character
    */
    if(isalpha(var1))
    {
        printf("\n var1 = |%c| is alphabet\n", var1);
    }
    else
    {
        printf("\n var1 =|%c| is not alphabet\n", var1);
    }

    if(isalpha(var2))
    {
        printf("\n var2 = |%c| is alphabet\n", var2);
    }
    else
    {
        printf("\n var2 =|%c| is not alphabet\n", var2);
    }

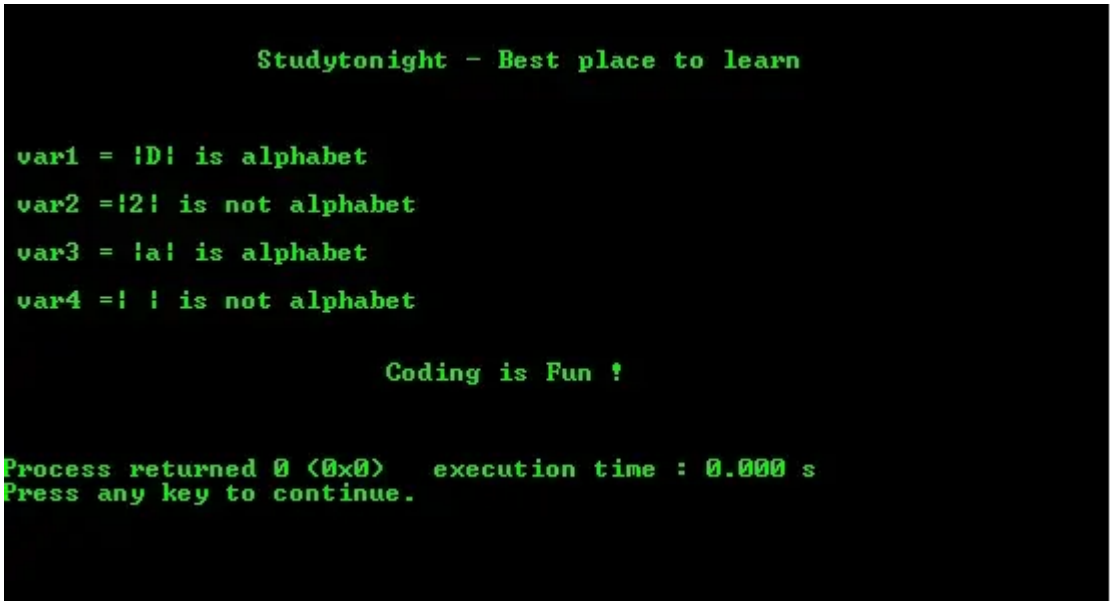
    if(isalpha(var3))
    {
        printf("\n var3 = |%c| is alphabet\n", var3);
    }
    else
    {
        printf("\n var3 =|%c| is not alphabet\n", var3);
    }

    if(isalpha(var4))
    {
        printf("\n var4 = |%c| is alphabet\n", var4);
    }
    else
    {
        printf("\n var4 =|%c| is not alphabet\n", var4);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
```

```
    return 0;
}
```

Output:



C Program to check if character string is alphanumeric using isalnum() Function

studytonight.com/c/programs/ctype/isalnum()-function

In this program we will check if character string is alphanumeric(either an alphabet or a number or both). We will be using a system defined function `isalnum()` defined under the **ctype.h** library for this. This method is used for various valid password checking options.

Below is a program on `isalnum()` function.

```
#include<stdio.h>
#include<ctype.h> // to make use of system defined function isalnum()

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    /*
        As datatypes are int, hence all the variables store the
        ASCII values of the character on the RHS
    */
    int var1 = 'd';
    int var2 = '2';
    int var3 = '%';
    int var4 = ' ';

    /*
        Returns true if the input parameter is an ASCII value
        of an alphanumberic character
    */
    if(isalnum(var1))
    {
        printf("\n var1 = |%c| is alphanumeric\n", var1);
    }
    else
    {
        printf("\nvar1 =|%c| is not alphanumeric\n", var1);
    }

    if(isalnum(var2))
    {
        printf("\n var2 = |%c| is alphanumeric\n", var2);
    }
    else
    {
        printf("\nvar2 =|%c| is not alphanumeric\n", var2);
    }

    if(isalnum(var3))
    {
        printf("\n var3 = |%c| is alphanumeric\n", var3);
    }
    else
    {
        printf("\nvar3 =|%c| is not alphanumeric\n", var3);
    }

    if(isalnum(var4))
    {
        printf("\n var4 = |%c| is alphanumeric\n", var4);
    }
    else
    {
        printf("\nvar4 =|%c| is not alphanumeric\n", var4);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
```

```
    return 0;
}
```

Output:



C Program to check for whitespace using isspace() Function

studytonight.com/c/programs/ctype/isspace()-function

In this program we will check white spaces as well as tabs. We will be using a system defined function `isspace()` defined under the **ctype.h** library.

This can be done using if-else block, condition being the ASCII value of the space character.

This method is used for various valid password checking options as well as the variable names in the C compiler as valid names shall not contain white spaces.

Below is a program on `isspace()` function.

`int` always considers the last character as valid, of a given series of characters.

Leading spaces of a character are not considered as spaces.

Trailing spaces are considered as spaces.

```
#include<stdio.h>
#include<ctype.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int var1 = '   d';
    int var2 = '2';
    int var3 = '\t';
    int var4 = 'a  ';

    /*
        Returns true if the input parameter is an ASCII value
        of a white-space character
    */
    if(isspace(var1))
    {
        printf("\n var1 = |%c| is a white-space character\n", var1);
    }
    else
    {
        printf("\n var1 =|%c| is not a white-space character\n", var1);
    }

    if(isspace(var2))
    {
        printf("\n var2 = |%c| is a white-space character\n", var2);
    }
    else
    {
        printf("\n var2 =|%c| is not a white-space character\n", var2);
    }

    if(isspace(var3))
    {
        printf("\n var3 = |%c| is a white-space character\n", var3);
    }
    else
    {
        printf("\n var3 =|%c| is not a white-space character\n", var3);
    }

    if(isspace(var4))
    {
        printf("\n var4 = |%c| is a white-space character\n", var4);
    }
    else
    {
        printf("\n var4 =|%c| is not a white-space character\n", var4);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
var1 =|d| is not a white-space character
var2 =|2| is not a white-space character
var3 = | | is a white-space character
var4 = | | is a white-space character
Coding is Fun !
Process returned 0 (0x0) execution time : -0.000 s
Press any key to continue.

C program to check for punctuation characters using the ispunct() Function

studytonight.com/c/programs/ctype/ispunct()-function

In this program we will check if a character is a punctuation character or not. We will be using a system defined function `ispunct()` defined under the **ctype.h** library.

Again, this can be done using if-else block condition using the range of ASCII values of punctuation characters. This method is used for various valid password checking options where atleast one punctuation character is mandatory.

Below is a program on `ispunct()` function.

```
#include<stdio.h>
#include<ctype.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    /*
        As datatypes are int, hence all the variables store the
        ASCII values of the character on the rhs
    */
    int var1 = 'D';
    int var2 = '2';
    int var3 = '/';
    int var4 = ',';

    /*
        Returns true if the input parameter is an ASCII value
        of a punctuation character
    */
    if(ispunct(var1))
    {
        printf("\n var1 = |%c| is a punctuation character\n", var1);
    }
    else
    {
        printf("\n var1 =|%c| is not a punctuation character\n", var1);
    }

    if(ispunct(var2))
    {
        printf("\n var2 = |%c| is a punctuation character\n", var2);
    }
    else
    {
        printf("\n var2 =|%c| is not a punctuation character\n", var2);
    }

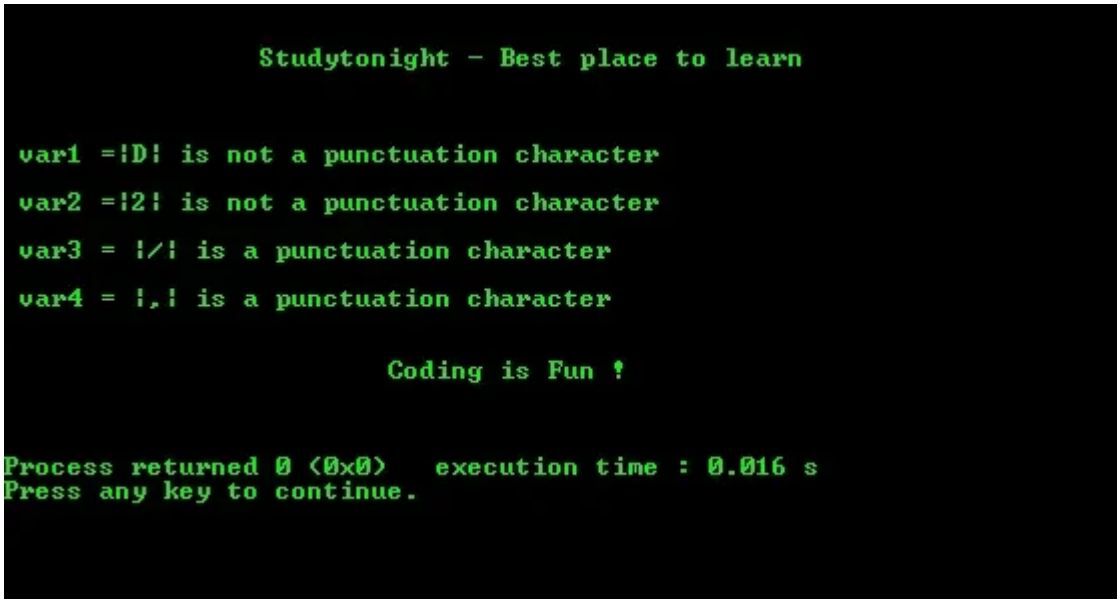
    if(ispunct(var3))
    {
        printf("\n var3 = |%c| is a punctuation character\n", var3);
    }
    else
    {
        printf("\nvar3 =|%c| is not a punctuation character\n", var3);
    }

    if(ispunct(var4))
    {
        printf("\n var4 = |%c| is a punctuation character\n", var4);
    }
    else
    {
        printf("\n var4 =|%c| is not a punctuation character\n", var4);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
```

```
    return 0;
}
```

Output:



Using isgraph() and isprint() Function in C Program

studytonight.com/c/programs/ctype/isgraph()-and-isprint()-function

In this program we will check if a particular character can be printed or not. This functionality can be achieved by making use of the `isgraph()` and `isprint()` method.

Though it is recommended to go through the basic difference among them, but they are almost alike. Both of them are defined under **ctype.h** library.

Below is a program on `isgraph()` and `isprint()` function.

```
#include<stdio.h>
#include<ctype.h> // to use isgraph() and isprint() method

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    /*
        As datatypes are int, hence all the variables store the
        ASCII values of the character on the rhs
    */
    int var1 = 'D';
    int var2 = '2';
    int var3 = ' \t';
    int var4 = 'Y';

    /*
        Returns true if the input parameter is an
        ASCII value of an alphanumberic character
    */
    if(isgraph(var1))
    {
        printf("\n var1 = |%c| can be printed\n", var1);
    }
    else
    {
        printf("\n var1 =|%c| cannot be printed\n", var1);
    }

    if(isprint(var2))
    {
        printf("\n var2 = |%c| can be printed\n", var2);
    }
    else
    {
        printf("\n var2 =|%c| can't be printed\n", var2);
    }

    if(isgraph(var3))
    {
        printf("\n var3 = |%c| can be printed\n", var3);
    }
    else
    {
        printf("\n var3 =|%c| can't be printed\n", var3);
    }

    if(isprint(var4))
    {
        printf("\n var4 = |%c| can be printed\n", var4);
    }
    else
    {
        printf("\n var4 =|%c| can't be printed\n", var4);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
```

```
    return 0;
}
```

Output:



C Program to copy string using strcpy() Function

studytonight.com/c/programs/string/strcpy()-function

In this program we will copy one string into another, using the `strcpy()` method defined under the **string.h** library.

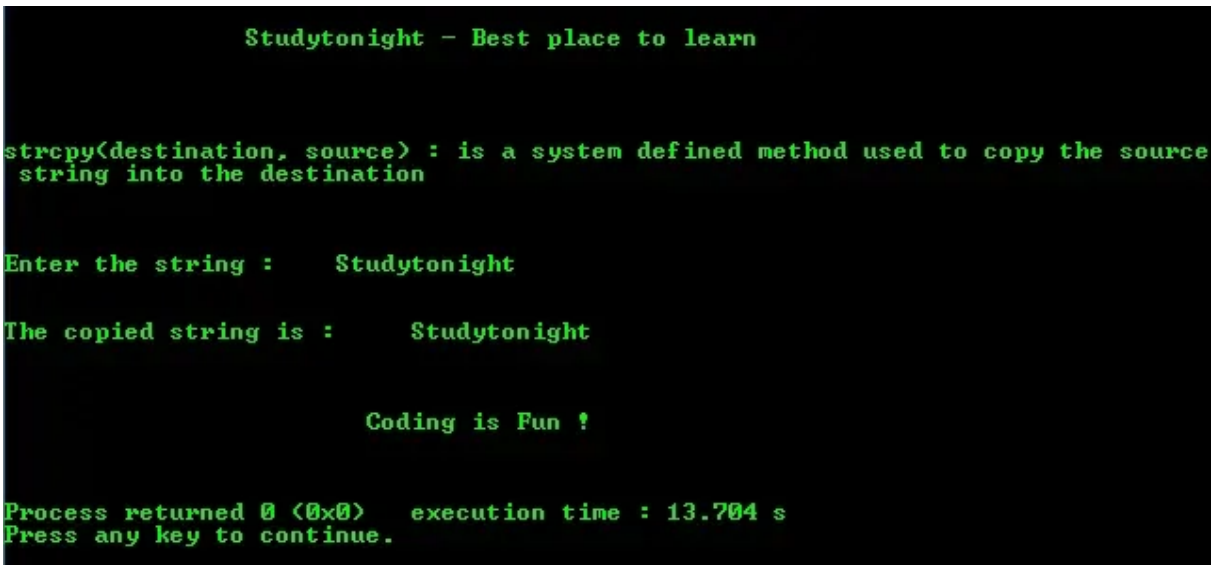
`strcpy(destination, source)` is a system defined method used to copy the source string into the destination variable.

Below is a program on `strcpy()` function.

```
#include<stdio.h>
#include<string.h>    // for using strcpy() function

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char mj[100];
    char aj[100];
    printf("\n\nstrcpy(destination, source): is a system defined method used
to copy the source string into the destination.\n\n");
    printf("\n\nEnter the string: ");
    gets(aj);
    strcpy(mj, aj);
    printf("\n\nThe copied string is: %s\n\n", mj);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C Program to copy string without `strcpy()` function

What if you do not want to use the system defined function?

While initializing a string, there is no need to put a null character at the end, as the compiler provides it automatically.

For example, `char aj[] = "aditya";` is a correct statement. However while copying one string into another, **terminating the destination string with a null character is mandatory**, to use it in future.

Below is a program for copying string without the use of `strcpy()` function.

```
#include<stdio.h>

void stringCopy(char[], char[]);    // function prototype declaration

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj[100], mj[100];
    printf("\n\nEnter 1st string: ");
    gets(aj);
    stringCopy(aj, mj); // function call
    printf("\n\nThe copied string is: %s\n\n", mj);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

void stringCopy(char a[], char b[]) // function definition
{
    int i = 0;
    while(a[i] != '\0')
    {
        b[i] = a[i];
        i++;
    }

    /*
        The string must always terminate with
        a '\0' - null character
    */
    b[i] =' \0';    // most important step
}
```

Output:


Studytonight - Best place to learn
Enter 1st string : Keep Learning!
The copied string is : Keep Learning!
Coding is Fun !
Process returned 0 (0x0) execution time : 14.782 s
Press any key to continue.

C Program to compare two strings using strcmp() Function

studytonight.com/c/programs/string/strcmp()-function

In this program we will compare strings using `strcmp()` function defined in the **string.h** library.

`strcmp(a, b)` returns **0** if both the strings `a` and `b` are exactly same else returns **-1**.

It is case sensitive so **'ABC'** and **'abc'** are considered to be different strings. It can be used with with `strrev()` function(function to reverse a string) to check whether a given string is a Pallindrome or not.

Below is a program on `strcmp()` function.

```
#include<stdio.h>
#include<string.h>    // for strcmp() function

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj1[100], aj2[100];
    int compare;

    printf("\n\nstrcmp(a,b) returns : 0 if both the strings are exactly same
and -1 otherwise.\n\n\n");
    printf("\n\nstrcmp(a,b) is Case Sensitive.\n\n\n");

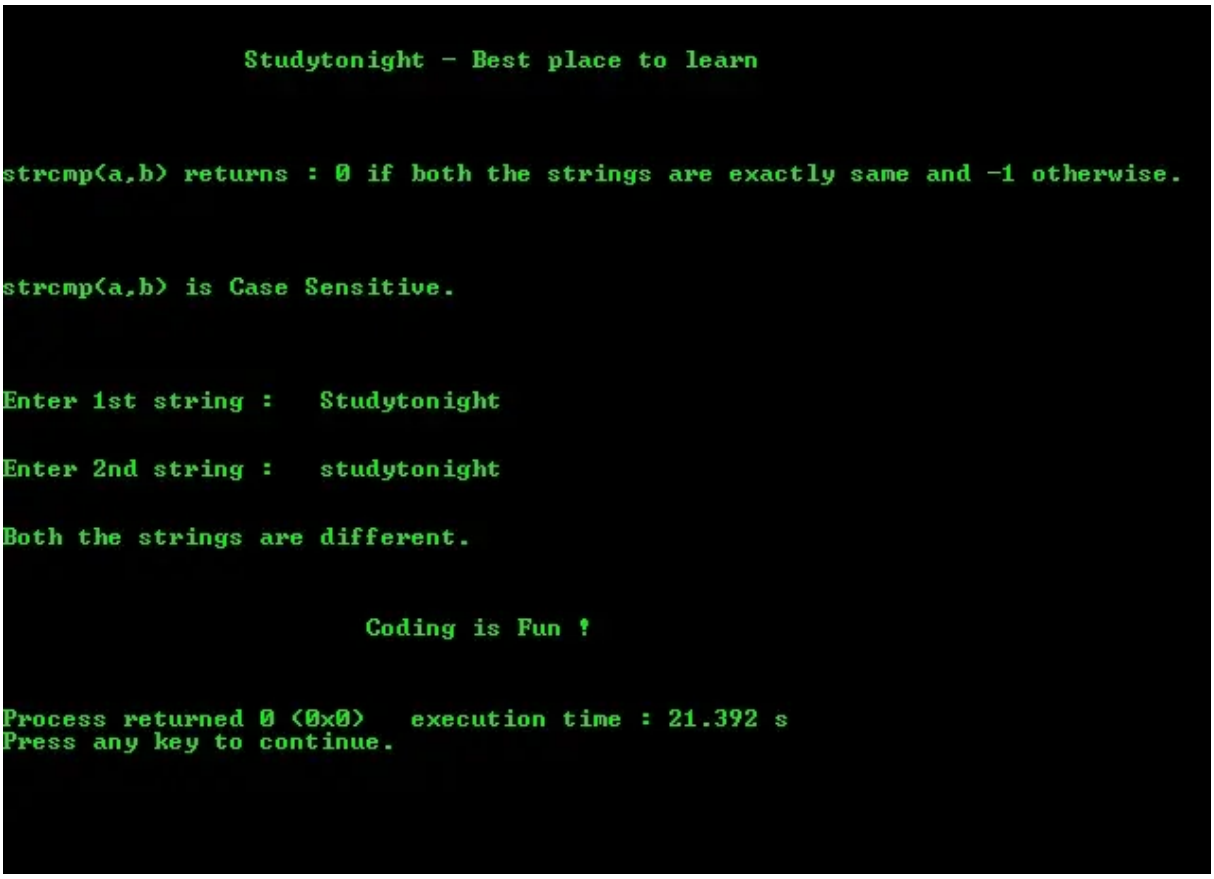
    printf("\n\nEnter 1st string: ");
    gets(aj1);

    printf("\n\nEnter 2nd string: ");
    gets(aj2);

    compare = strcmp(aj1, aj2);
    if(compare == 0)
        printf("\n\nBoth the strings are exactly same.\n\n");
    else
        printf("\n\nBoth the strings are different.\n");

    printf("\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C Program to compare strings without `strcmp()` function

Below is a program to compare strings, without the use of `strcmp()` function.

```
#include<stdio.h>

int stringCompare(char[], char[]);  // function prototype declaration

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj1[100], aj2[100];
    int compare;

    printf("\n\nEnter 1st string: ");
    scanf("%s", aj1);

    printf("\n\nEnter 2nd string: ");
    scanf("%s", aj2);

    compare = stringCompare(aj1, aj2);  // function call

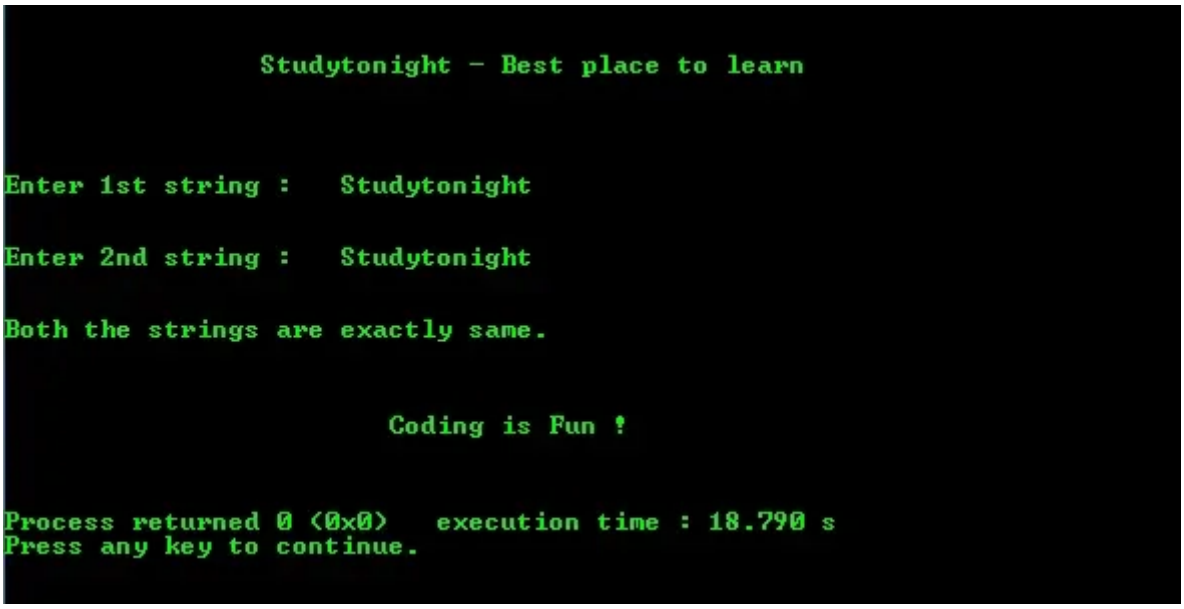
    if(compare == 1)
        printf("\n\nBoth the strings are exactly same.\n\n");
     else
        printf("\n\nBoth the strings are different.\n");

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

int stringCompare(char mj1[], char mj2[])  // function definition
{
    int i = 0, flag = 0;
    while(mj1[i] != '\0' && mj2[i] != '\0') // until atleast one string ends
    {
        /*
            Even if they differ by a single character,
            don't iterate further
        */
        if(mj1[i] != mj2[i])
        {
            flag = 1;
            break;
        }
        i++;
    }

    /*
        If all the characters are sequentially same as well as
        both strings have ended
    */
    if(flag == 0 && mj1[i] == '\0' && mj2[i] == '\0')
        return 1;
    else
        return 0;
}
```

Output:


Studytonight - Best place to learn
Enter 1st string : Studytonight
Enter 2nd string : Studytonight
Both the strings are exactly same.
Coding is Fun !
Process returned 0 (0x0) execution time : 18.790 s
Press any key to continue.

C Program to reverse a string using strrev() Function

studytonight.com/c/programs/string/strrev()-function

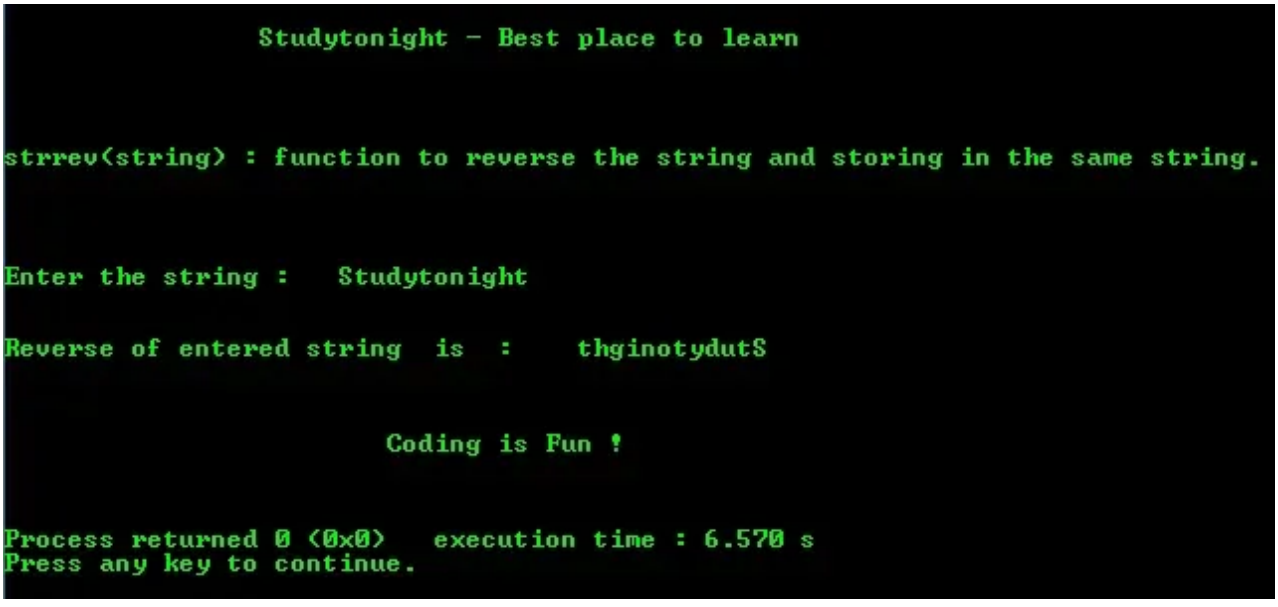
In this program we will compare strings using the `strrev()` defined within the **string.h** library. It can be used to check if the string is pallindrome or not by using this function along with the `strcmp()` function.

Below is a program on `strrev()` function.

```
#include<stdio.h>
#include<string.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj[1000];
    printf("\n\nstrrev(string): Function to reverse the string and storing in the
same string.\n\n\n");
    printf("\n\nEnter the string: ");
    gets(aj);
    /*
        system defined function to reverse the string
        and storing in the same string
    */
    strrev(aj);
    printf("\n\nReverse of entered string is: %s", aj);
    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C Program for removing Whitespaces in a String

studytonight.com/c/programs/string/removing-whitespaces-in-string

Below is a program to remove whitespaces from a given string.

```
#include<stdio.h>
#include<string.h>

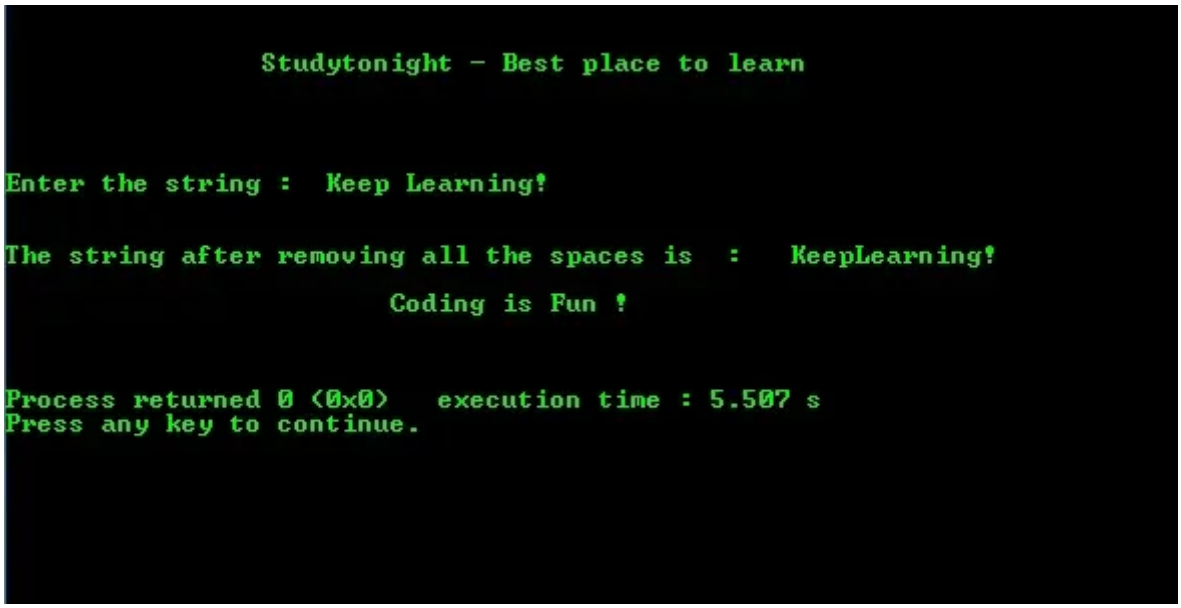
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj[1000], mj[1000];
    int i = 0, j = 0, len;

    printf("\n\nEnter the string: ");
    gets(aj);

    len = strlen(aj);   // len stores the length of the input string

    while(aj[i] != '\0')    // till string doesn't terminate
    {
        if(aj[i] != ' ')  // if the char is not a white space
        {
            /*
                incrementing index j only when
                the char is not space
            */
            mj[j++] = aj[i];
        }
        /*
            i is the index of the actual string and
            is incremented irrespective of the spaces
        */
        i++;
    }
    mj[j] = '\0';
    printf("\n\nThe string after removing all the spaces is: %s", mj);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the string : Keep Learning!
The string after removing all the spaces is : KeepLearning!
Coding is Fun !
Process returned 0 (0x0) execution time : 5.507 s
Press any key to continue.

gets() and strlen() Function Examples in C Language

studytonight.com/c/programs/string/gets()-and-strlen()-function

Let's start by learning about `gets()` and `strlen()` functions in C. These functions are used for managing strings in C language.

What is gets() function in C?

The syntax of `gets()` is as follows:

```
char * gets(char * str);
```

It reads characters from the *standard input* (**stdin**) and stores them as a C string into `str` until a newline character or the *end-of-file* is reached. The newline character, if found, is not copied into `str`.

A terminating **null** character is automatically appended after the characters which are copied to `str`.

It has `str` as a parameter which is pointer to a block of memory (array of char) where the string read is copied as a C string.

On success, the function returns `str`.

If the *end-of-file* is encountered while attempting to read a character, the *eof indicator* is set (feof). If this happens before any characters could be read, the pointer returned is a null pointer (and the contents of *str* remain unchanged). In any case of error, a null pointer in returned.

What is strlen() function in C?

The syntax of `strlen()` is as follows:

```
size_t strlen(const char * str);
```

It returns the length of the C string `str`. When it encounters a null-character, it terminates the string.

It take a C string as a parameter.

1. C Program to find Length of a string using `strlen()` function

Let's see the steps followed,

- Declare a char array according to length of the string you want to input.

- Read input in the string using `gets()`.
- Calculate length of the string using `strlen()` and copy the value returned by it in a variable named length.
- Print length of the string.

```
#include<stdio.h>
#include<string.h>

int main()
{
    printf("Studytonight - Best place to learn\n");
    char a[100];
    int length;
    printf("Enter a string you wish to calculate the length of : ");
    /*
        to take a single string (including spaces)
        as input at a time
    */
    gets(a);
    /*
        Return the length of the string
        or the number of characters in the string
    */
    length = strlen(a);
    printf("The length of the input string is: %d", length);
    printf("\nCoding is Fun !");
    return 0;
}
```

Studytonight - Best place to learn
Enter a string you wish to calculate the length of : studytonight
The length of the input string is: 12
Coding is Fun !

Note: While compiling the above code, we get a warning that `gets()` is dangerous and should not be used. The use of `gets()` is depreciated. It is so because the function doesn't know how big the buffer is, so it continues reading until it finds a newline or encounters EOF, and may **overflow the bounds of the buffer** it was given. In order to use it safely, we have to know exactly how many characters we will be reading, so that we can make your buffer large enough. We will only know that if we know exactly what data we will be reading.

The first internet worm used `gets()` to propagate from system to system.

Some important points about `gets()` and `scanf()` methods are:

- Both `scanf()` and `gets()` are used to take input from user.
- `scanf()` skips the input string after the first space is entered.

- `gets()` is used to input a complete sentence with any desired number of spaces until the **Enter** key is pressed.

---

2. C Program to find Length of a string without `strlen()` function

Before moving on to the code, remember that a string is terminated by a **null** character. So, the last character in every string is **'\0'**.

Let's see the steps followed,

- Declare a `char` array according to length of the string you want to input.
- Read input in the string using `gets()`.
- To calculate length of the string, we will iterate over the string till we reach the null character. We will use a for loop to accomplish this.
- Print length of the string.

Below is a program to find the length of a string without the use of `strlen()` function.

```
#include<stdio.h>
int main()
{
    printf("Studytonight - Best place to learn\n");
    char a[1000];
    int i = 0;
    printf("Enter the string: ");
    gets(a);
    // \0 marks the end of the string
    for(i = 0; a[i] != '\0' ;i++);   // updation is done after checking
condition
    printf("The length of the input string is : %d", i);
    printf("\nCoding is Fun !");
    return 0;
}
```

Studytonight - Best place to learn
Enter the string: abcdefghi
The length of the input string is : 9
Coding is Fun !

Explanation:

You must be wondering why we have not added any statement to our `for` loop. It's because we are traversing our string until we encounter `\0` and are incrementing `i` every time, hence we do not have to do anything else inside the loop, as the value

of `i` will be equal to the length of the string by the time we encounter the `\0` character.

Difference between strlen() and sizeof() function

studytonight.com/c/programs/string/difference-between-strlen()-and-sizeof()-function

Below is a program on `strlen()` and `sizeof()` function.

`strlen()` is a system defined function that does not consider the null character `\0`.

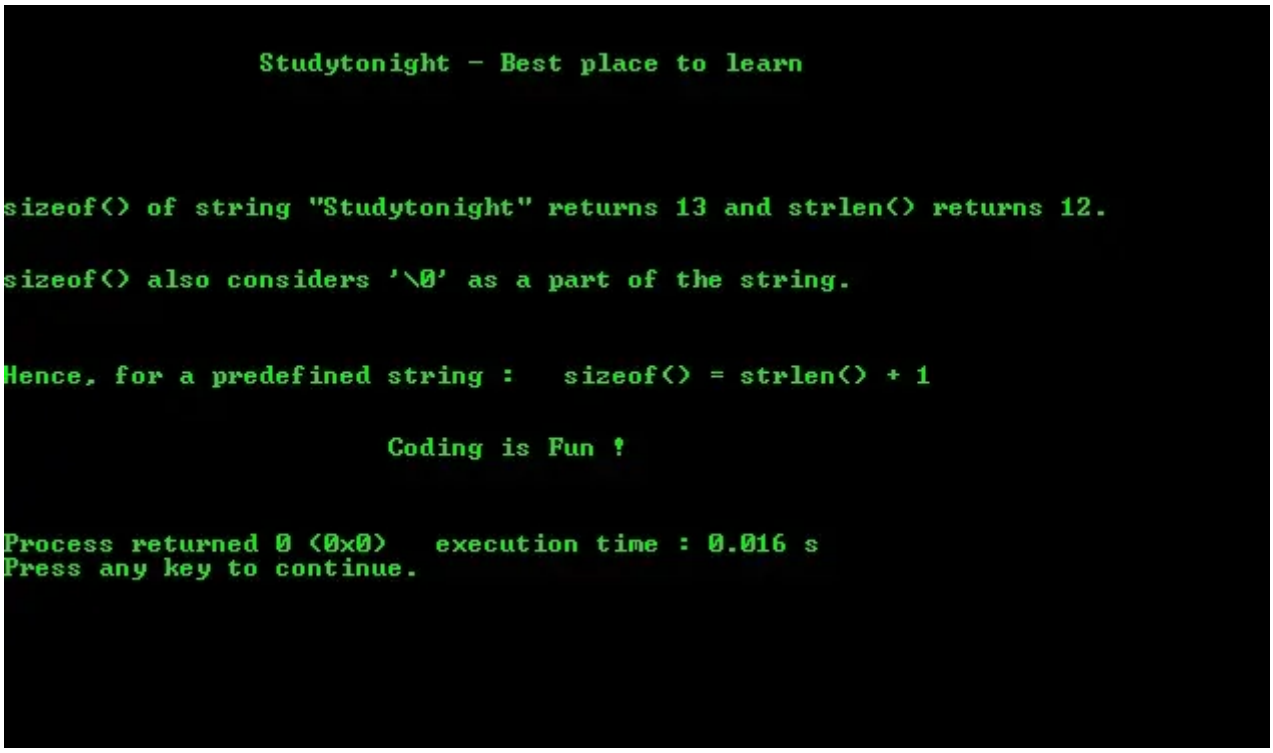
```
#include<stdio.h>
#include<conio.h>
#include<string.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj[] = "Studytonight";
    int length1, length2;

    length1 = sizeof(aj);
    length2 = strlen(aj);

    // need to put in backslash infront to print the quote symbol
    printf("\n\n\nsizeof() of string \"Studytonight\" returns %d and strlen()
returns %d.", length1, length2);
    printf("\n\n\nsizeof() also considers \'\\0\' as a part of the string.\n");
    printf("\n\n\nHence, for a predefined string: sizeof() = strlen() + 1 \n");
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
sizeof() of string "Studytonight" returns 13 and strlen() returns 12.
sizeof() also considers '\0' as a part of the string.
Hence, for a predefined string : sizeof() = strlen() + 1
Coding is Fun !
Process returned 0 (0x0) execution time : 0.016 s
Press any key to continue.

C Program to calculate the Frequency of characters in a string

studytonight.com/c/programs/string/frequency-of-characters-in-string

Below is a program to find the frequency of characters in a given string.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj[1000];
    int c = 0;
    int count[26] = {0};    // initializing all the elements of array to 0

    printf("\n\nEnter the string :   ");
    gets(aj);

    while(aj[c] != '\0')
    {
        if(aj[c] >= 'a' && aj[c] <= 'z')
        {
            /*
                Incrementing the count of the letter in the array,
                converting its index within the range  0 to 25
            */
            count[aj[c] - 'a']++;
        }
        c++;
    }

    for(c = 0; c < 26; c++)
    {
        /*
             If the character appears in the input string
             atleast once
        */
        if(count[c] != 0)
        {
            printf("\n\t\t%c occurs %d times in the string, \n",c+'a' ,count[c]);
        }
    }

    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:

```
                Studytonight - Best place to learn


Enter the string :   Studytonight is the best place to learn
                a occurs 2 times in the string,
                b occurs 1 times in the string,
                c occurs 1 times in the string,
                d occurs 1 times in the string,
                e occurs 4 times in the string,
                g occurs 1 times in the string,
                h occurs 2 times in the string,
                i occurs 2 times in the string,
                l occurs 2 times in the string,
                n occurs 2 times in the string,
                o occurs 2 times in the string,
                p occurs 1 times in the string,
                r occurs 1 times in the string,
                s occurs 2 times in the string,
                t occurs 6 times in the string,
                u occurs 1 times in the string,
                y occurs 1 times in the string,


                        Coding is Fun !


Process returned 0 (0x0)   execution time : 12.553 s
Press any key to continue.
```

C Program to Count number of vowels in a String

studytonight.com/c/programs/string/count-number-of-vowels

Below is a program to count number of vowels in a given string.

```
#include<stdio.h>

// Global method declaration
int count_vowels(char []);
int check_vowel(char);

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    char aj[100];
    int mj;

    printf("\n\nEnter a string :   ");
    gets(aj);   // inputting a string from user

    mj = count_vowels(aj);  // function call

    printf("\n\nNumber of vowels in string \'%s\' is: %d", aj, mj);
    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

/*
    Collecting the input string in char array
    adi[], all at a time
*/
int count_vowels(char adi[])
{
    int count = 0, c= 0, flag = 0;
    char d;
    do
    {
        /*
            Storing each character of string in
            char 'd' one at a time
        */
        d = adi[c];
        /*
            Calling function to check if char is vowel or not
        */
        flag = check_vowel(d);

        if(flag ==1)    // if the char is vowel
        {
            count++;    // increment the count of the vowel
        }
        c++;    // updating the string character to be checked
    }while(d != '\0');  // terminating condition till it reaches the end of the
string

    return count;
}

int check_vowel(char ajj)
{
    if(ajj >= 'A' && ajj <= 'Z')  // if the char is uppercase
```

```
        ajj = ajj+'a' - 'A';    // converting to lowercase using the ASCII values

    // if the char is any of the vowel
    if(ajj == 'a' || ajj == 'e' || ajj == 'i' || ajj == 'o' || ajj == 'u')
        return 1;   // making flag=1

    return 0;   // if not a vowel, flag remains 0
}
```

Output:



C Program for Adding Two Numbers Using Recursion

studytonight.com/c/programs/recursion/adding-two-numbers-using-recursion

Below is a program on adding two numbers using recursion.

```
#include<stdio.h>

int y;

/*
    Function to add two numbers and
    return the result
*/
int add(int m, int n)
{
    if(n == 0)
        return m;

    /*
        Recursion: adding 1, n times and
        then at the end adding m to it
    */
    y = add(m, n-1) + 1;
    return y;   // return the result
}

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int a, b, r;
    printf("Enter the two numbers:\n");
    scanf("%d%d", &a, &b);
    r = add(a, b);     // function call
    printf("\n\nSum of two numbers is: %d\n\n", r);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the two numbers
5
6
Sum of two nos is 11
Coding is Fun !
Process returned 0 (0x0) execution time : 6.235 s
Press any key to continue.

Program to find Factorial of a Number using Recursion

studytonight.com/c/programs/recursion/factorial-using-recursion

Below is a program for finding factorial of a given number using recursion.

```
#include<stdio.h>
// declaring the function
int fact(int);

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int num, f;
    printf("\n\nEnter a number: ");
    scanf("%d", &num);
    f= fact(num);
    printf("\n\nFactorial of  %d is  %d\n\n", num, f);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

int fact(int aj)
{
    if(aj==1 || aj==0)
        return 1;
    else
        return (aj*fact(aj-1));
}
```

Output:



Program to print Fibonacci Series using Recursion

studytonight.com/c/programs/recursion/fibonacci-series-using-recursion

A Fibonacci series is defined as a series in which each number is the sum of the previous two numbers with 1, 1 being the first two elements of the series.

`static` keyword is used to initialize the variables only once.

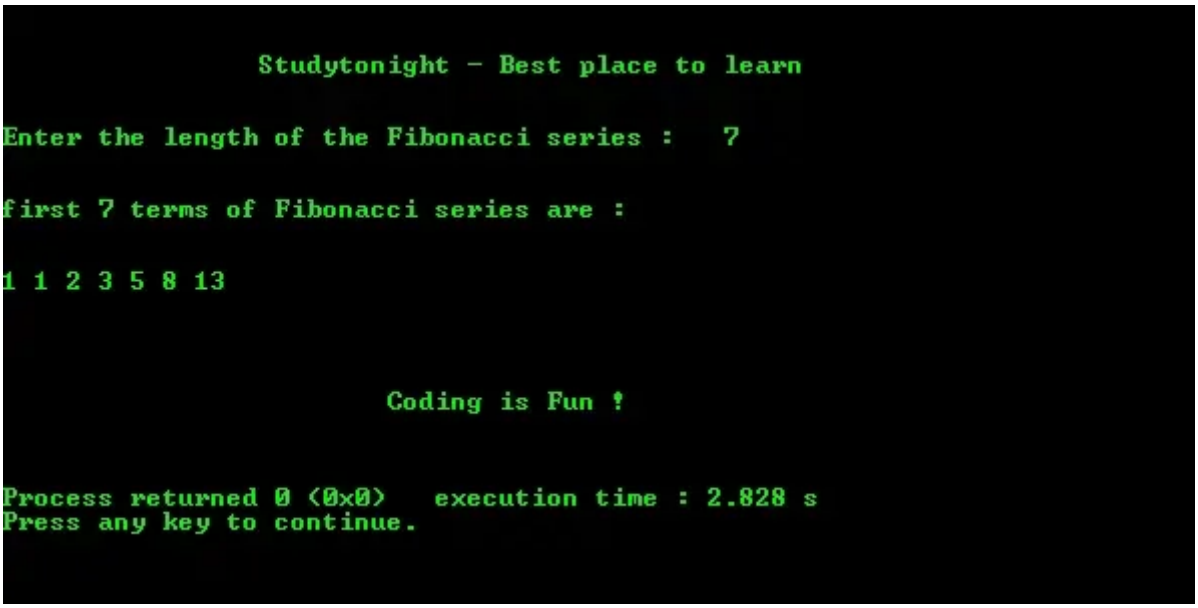
Below is a program to print the fibonacci series using recursion.

```
#include<stdio.h>
// declaring the function
void printFibo(int );

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int k, n;
    long int i = 0, j = 1;
    printf("Enter the length of the Fibonacci series: ");
    scanf("%d", &n);
    printf("\n\nfirst %d terms of Fibonacci series are:\n\n\n",n);
    printf("%d ", 1);
    printFibo(n);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

void printFibo(int aj)
{
    static long int first = 0, second = 1, sum;
    if(aj > 1)
    {
        sum = first + second;
        first = second;
        second = sum;
        printf("%ld ", sum);
        printFibo(aj-1);    // recursive call
    }
    else
    {
        // after the elements, for line break
        printf("\n\n\n");
    }
}
```

Output:


Studytonight - Best place to learn
Enter the length of the Fibonacci series : 7
first 7 terms of Fibonacci series are :
1 1 2 3 5 8 13
Coding is Fun !
Process returned 0 (0x0) execution time : 2.828 s
Press any key to continue.

Program to find Sum of First N Numbers

studytonight.com/c/programs/recursion/sum-of-first-n-numbers

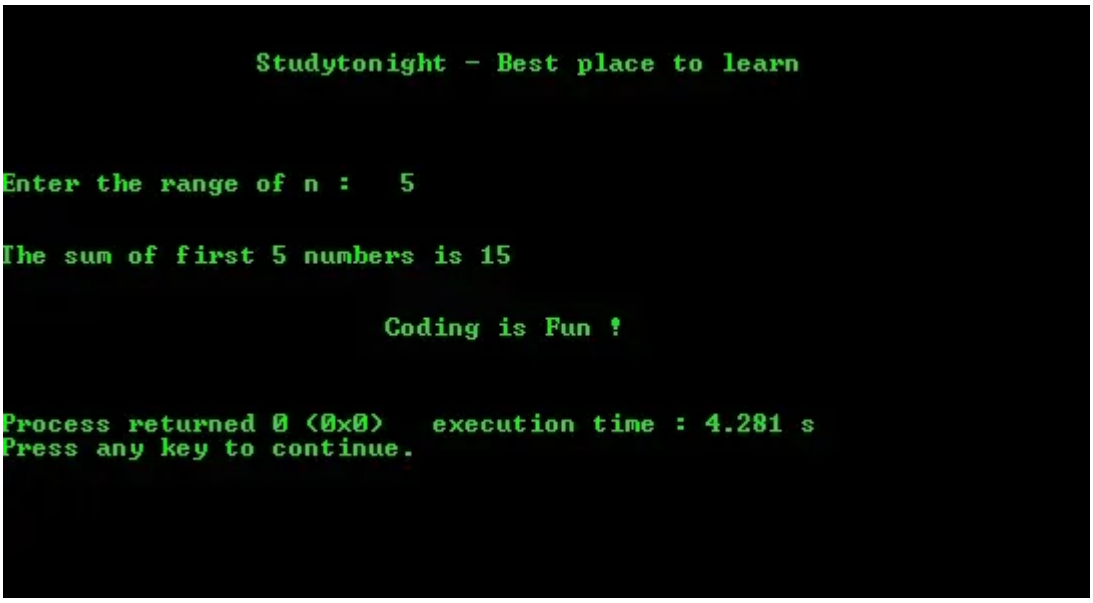
Below is a program to find sum of the first `n` numbers using recursion, where the value of `n` is provided by the user.

```
#include<stdio.h>
// declaring the recursive function
int getSum(int);

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n, sum;
    printf("\n\nEnter the range of n: ");
    scanf("%d", &n);
    sum = getSum(n);
    printf("\n\nThe sum of first %d numbers is %d\n", n, sum);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

// function definition
int getSum(int aj)
{
    /*
        static variables hold their values
        till the end of the program
    */
    static int sum = 0;
    if(aj > 0)
    {
        sum = sum + aj;
        getSum(aj-1);
    }
    return sum ;
}
```

Output:


Studytonight - Best place to learn
Enter the range of n : 5
The sum of first 5 numbers is 15
Coding is Fun !
Process returned 0 (0x0) execution time : 4.281 s
Press any key to continue.

Program to find Sum of digits of a Number using Recursion

studytonight.com/c/programs/recursion/sum-of-digits-using-recursion

Below is a program to find sum of digits of a given number using recursion.

```
#include<stdio.h>
#include<conio.h>

//declaring the recursive function
int sumOfDigit(int num);

void main()
{
    int num, sum;
    clrscr();
    printf("Enter a number:\t");
    scanf("%d", &num);
    sum = sumOfDigit(num);
    printf("The sum of digits of %d is: %d", num, sum);
    getch();
}

int sumOfDigit(int num)
{
    int s, a;
    s = s + (num%10);
    a = num/10;
    if(a > 0)
    {
        sumOfDigit(a);
    }
    return s;
}
```

Enter a number: 108 The sum of digits of 108 is: 9

Program to find Palindrome using Recursion

studytonight.com/c/programs/recursion/palindrome-using-recursion

A Palindrome is a sequence that if reversed looks identical to the original sequence Eg : abba, level, 999 etc.

Below is a simple C program to find whether the user input number is a palindrome or not using recursion:

```
#include<stdio.h>

// declaring the recursive function
int isPal(int );

/*
    global declaration to use the same value
    in both the functions
*/
int n;

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int palindrome;
    printf("\n\nEnter a number to check for Palindrome: ");
    scanf("%d", &n);
    palindrome = isPal(n);
    if(palindrome == 1)
        printf("\n\n\n%d is palindrome\n\n", n);
    else
        printf("\n\n\n%d is not palindrome\n\n", n);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

int isPal(int aj)
{
    static int sum = 0;
    if(aj != 0)
    {
        sum = sum *10 + aj%10;
        isPal(aj/10);   // recursive call same as while(n!=0) using loop
    }
    else if(sum == n)
        return 1;
    else
        return 0;
}
```

Output:


Studytonight - Best place to learn
Enter a number to check if Pallandrome : 34543
34543 is pallandrome
Coding is Fun !
Process returned 0 (0x0) execution time : 4.906 s
Press any key to continue.

C Program to calculate a Number raised to the Power of N using Recursion

studytonight.com/c/programs/recursion/power-of-n-using-recursion

Below is a program to calculate the result of a given number, raised to the power of `n` using recursion.

```
#include<stdio.h>

// function prototype declaration
int power(int n1, int n2);

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int base, exp;
    printf("Enter base number: ");
    scanf("%d", &base);
    printf("\n\nEnter Power factor: ");
    scanf("%d", &exp);
    printf("\n\n\n\t\t\t%d^%d = %d", base, exp, power(base, exp));
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

int power(int b, int e)
{
    if(e == 0)
        return 1;

    return (b*power(b, e-1));
}
```

Output:


Studytonight - Best place to learn
Enter base number : 5
Enter power number : 4
5^4 = 625
Coding is Fun !
Process returned 0 (0x0) execution time : 1.859 s
Press any key to continue.

C Program to find the largest Element in an Array using Recursion

studytonight.com/c/programs/recursion/largest-array-element-using-recursion

Below is a program to find the largest array element in a given array using recursion.

`#define` is used to initialize a value which is more like a constant.

```
#include<stdio.h>

#define MAX 100

int getMaxElement(int []);  // takes array of int as parameter
int size;

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int arr[MAX], max, i;
    printf("\n\nEnter the size of the array: ");
    scanf("%d", &size);
    printf("\n\nEnter %d elements\n\n", size);

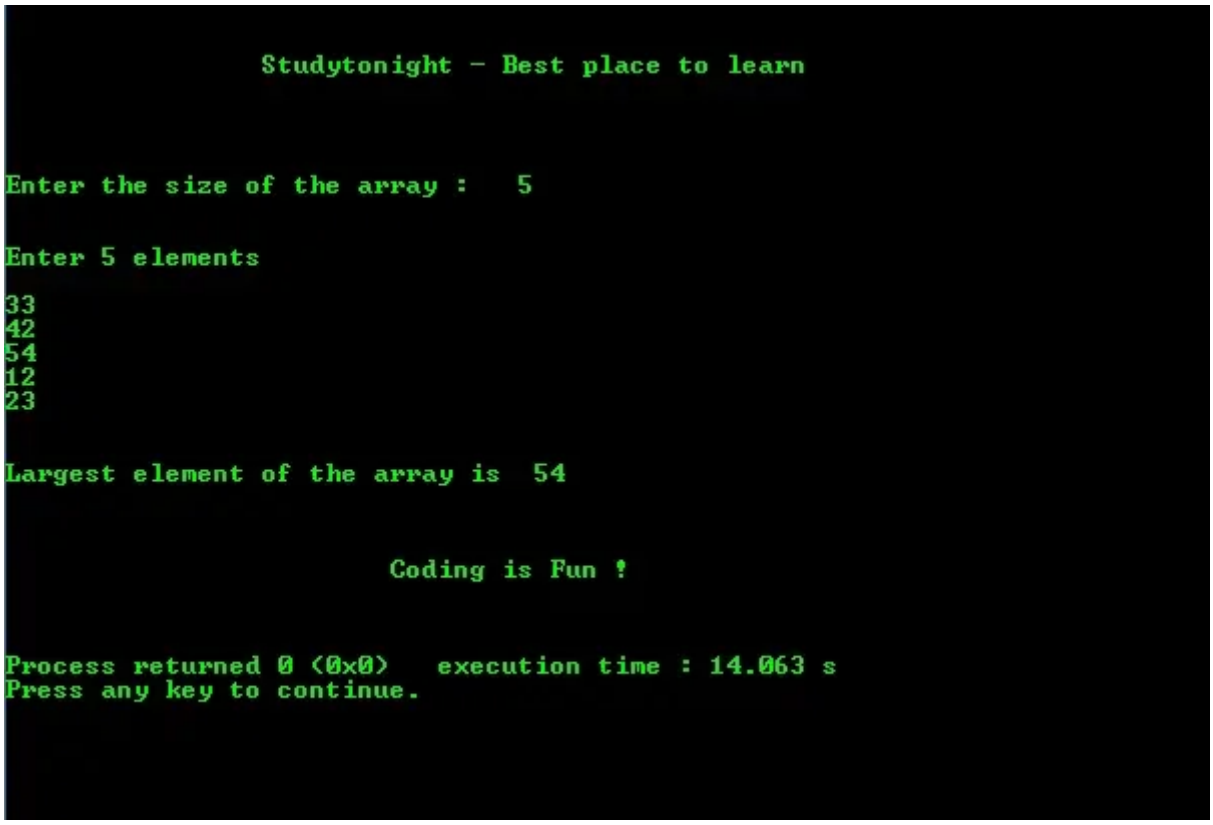
    for(i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    max = getMaxElement(arr);   // passing the complete array as parameter
    printf("\n\nLargest element of the array is %d\n\n", max);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

int getMaxElement(int a[])
{
    static int i = 0, max =- 9999;  // static int max=a[0] is invalid
    if(i < size)   // till the last element
    {
        if(max < a[i])
        max = a[i];

        i++;    // to check the next element in the next iteration
        getMaxElement(a);   // recursive call
    }
    return max;
}
```

Output:


Studytonight - Best place to learn
Enter the size of the array : 5
Enter 5 elements
33
42
54
12
23
Largest element of the array is 54
Coding is Fun !
Process returned 0 (0x0) execution time : 14.063 s
Press any key to continue.

C Program to find whether a Number is Prime Or Composite using Recursion

studytonight.com/c/programs/recursion/prime-or-composite-using-recursion

- **Prime Number:** A number that is only divisible by 1 and itself.
- **Composite Number:** A number that is not a prime number.

> ***Note:*** *1 is neither prime nor composite.*

Below is a program to find whether the user input number is a prime number or a composite number using recursion.

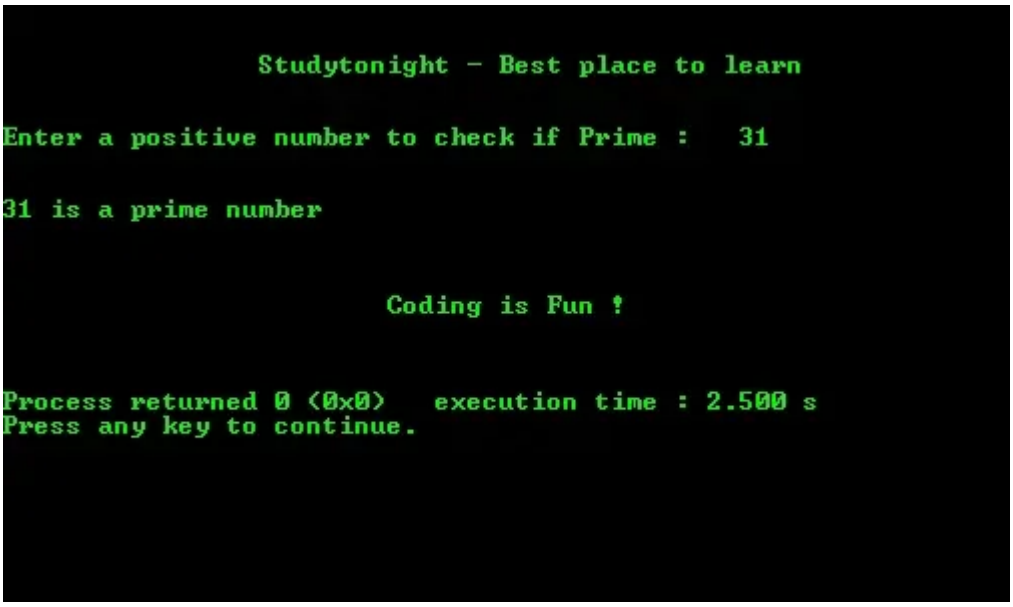
```
#include<stdio.h>

// declaring the recursive function
int isPrime(int, int);

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int num, prime;
    printf("Enter a positive number to check if Prime: ");
    scanf("%d", &num);
    prime = isPrime(num, num/2);
    if(prime == 1)
    {
        printf("\n\n%d is a prime number\n\n", num);
    }
    else
    {
        printf("\n\n%d is a Composite number\n\n", num);
    }
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

// function definition
int isPrime(int n, int i)
{
    if(i == 1)
        return 1;   // return statement terminates the recursive funtion
    else
    {
        if(n%i == 0)
            return 0;
        else
            isPrime(n, i-1);    // recursive call not using return statement
    }
}
```

Output:


Studytonight - Best place to learn
Enter a positive number to check if Prime : 31
31 is a prime number
Coding is Fun !
Process returned 0 (0x0) execution time : 2.500 s
Press any key to continue.

C Program to find LCM of two Numbers using Recursion

studytonight.com/c/programs/recursion/lcm-of-two-numbers-using-recursion

LCM: Least Common Multiple of two numbers is the number that is a common multiple of the both the numbers.

Below is a program to find LCM of two numbers using recursion.

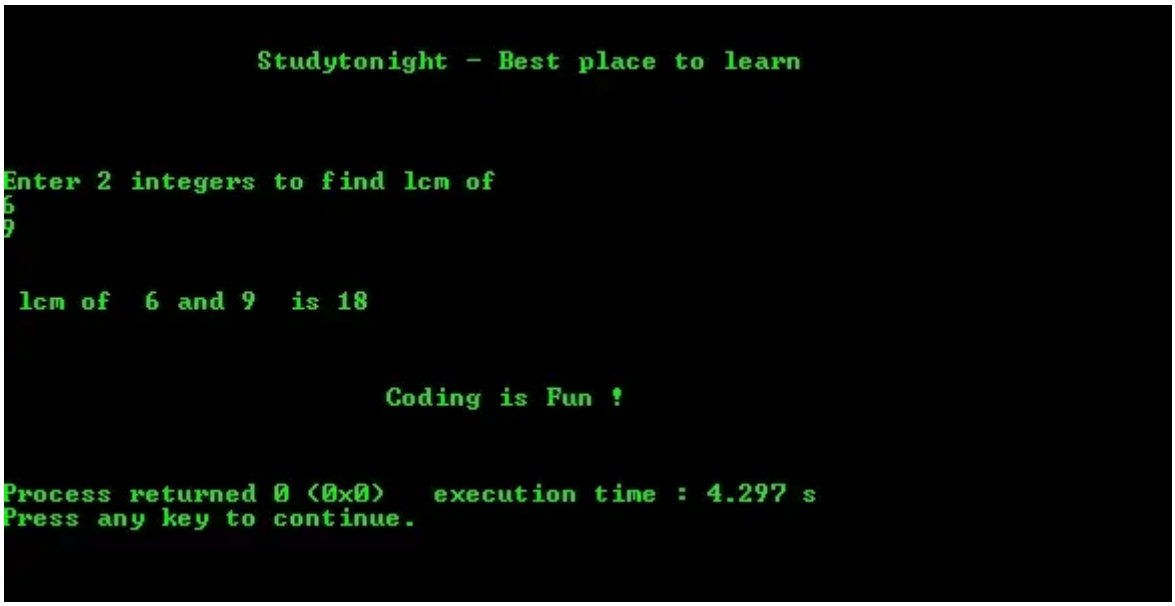
```
#include<stdio.h>

int find_lcm(int, int);   // function prototype declaration

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int a, b, lcm;
    printf("\n\nEnter 2 integers to find LCM of:\n");
    scanf("%d%d", &a, &b);
    lcm = find_lcm(a,b);    // function call
    printf("\n\n LCM of %d and %d is: %d\n\n", a, b, lcm);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

int find_lcm(int a, int b)  // function definition
{
    /*
        static variable is initialized only once
        for each function call
    */
    static int temp = 1;
    if(temp%a == 0 && temp%b == 0)
    {
        return temp;
    }
    else
    {
        temp++;
        find_lcm(a,b);
        return temp;
    }
}
```

Output:


Studytonight - Best place to learn
Enter 2 integers to find lcm of
6
9
lcm of 6 and 9 is 18
Coding is Fun !
Process returned 0 (0x0) execution time : 4.297 s
Press any key to continue.

C Program to find GCD of two Numbers using Recursion

studytonight.com/c/programs/recursion/gcd-of-two-numbers-using-recursion

Greatest Common Divisor(GCD) of two numbers is a number that divides both of them.

Below is a program to the GCD of the two user input numbers using recursion.

```
#include<stdio.h>

// declaring the recursive function
int find_gcd(int , int );

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int a, b, gcd;
    printf("\n\nEnter two numbers to find GCD of \n");
    scanf("%d%d", &a, &b);
    gcd = find_gcd(a, b);
    printf("\n\nGCD of %d and %d is: %d\n\n", a, b, gcd);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

// defining the function
int find_gcd(int x, int y)
{
    if(x > y)
        find_gcd(x-y, y);

    else if(y > x)
        find_gcd(x, y-x);
    else
    return x;
}
```

Output:


Studytonight - Best place to learn
Enter two numbers to find GCD of
14
21
GCD of 14 and 21 is 7
Coding is Fun !
Process returned 0 (0x0) execution time : 4.404 s
Press any key to continue.

Program to Reverse a String Using Recursion

studytonight.com/c/programs/recursion/reverse-a-string-using-recursion

Below is a program to reverse a user input string using recursion in C language.

```
#include<stdio.h>
#include<conio.h>

// declaring recursive function
char* reverse(char* str);

void main()
{
    int i, j, k;
    char str[100];
    char *rev;
    printf("Enter the string:\t");
    scanf("%s", str);
    printf("The original string is: %s\n", str);
    rev = reverse(str);
    printf("The reversed string is: %s\n", rev);
    getch();
}

// defining the function
char* reverse(char *str)
{
    static int i = 0;
    static char rev[100];
    if(*str)
    {
        reverse(str+1);
        rev[i++] = *str;
    }
    return rev;
}
```

Enter the string: studytonight The original string is: studytonight The reverse string is: thginotyduts

C Program to Print names of all Files present in a Directory

studytonight.com/c/programs/files-and-streams/program-to-list-files-in-directory

dirent.h header file contains variables and functions related to directory streams.

Below is a program to print the names of all the files in a directory.

```
#include<stdio.h>
#include<dirent.h>

int main(void)
{
    DIR *d;
    struct dirent *dir;
    d = opendir(".");
    if (d)
    {
        while ((dir = readdir(d)) != NULL)
        {
            printf("%s\n", dir->d_name);
        }
        closedir(d);
    }
    return(0);
}
```

File1.txt File2.txt File3.txt File4.txt File5.txt File6.txt File7.txt

We can also take the directory name as input from the user, and can also create a simple C program to search for a particular file in a directory.

C Program to find the Size of any File

studytonight.com/c/programs/files-and-streams/program-to-find-size-of-file

We will be using `fseek()` and `ftell()` functions to find the size of the file. There are others ways to find the file size as well, like looping on the whole content of file and finding out the size, but the File Handling functions makes it a lot easier.

Below is a program to find size of file.

Here is the C language tutorial explaining File Handling in C → File Handling in C

```
#include<stdio.h>
#include<conio.h>

void main()
{
    FILE *fp;
    char ch;
    int size = 0;

    fp = fopen("MyFile.txt", "r");
    if (fp == NULL)
    {
        printf("\nFile unable to open...");
    }
    else
    {
        printf("\nFile opened...");
    }
    fseek(fp, 0, 2);    /* File pointer at the end of file */
    size = ftell(fp);   /* Take a position of file pointer in size variable */
    printf("The size of given file is: %d\n", size);
    fclose(fp);
}
```

C Program to create a File & write Data in it

studytonight.com/c/programs/files-and-streams/program-to-write-in-file

Below is a program to create a new file and then storing information in it.

```
#include<stdio.h>
#include<conio.h>

void main()
{
    FILE *fptr;
    char name[20];
    int age;
    float salary;

    /* open for writing */
    fptr = fopen("emp.txt", "w");

    if (fptr == NULL)
    {
        printf("File does not exist.\n");
        return;
    }
    printf("Enter the name:\n");
    scanf("%s", name);
    fprintf(fptr, "Name  = %s\n", name);

    printf("Enter the age:\n");
    scanf("%d", &age);
    fprintf(fptr, "Age  = %d\n", age);

    printf("Enter the salary:\n");
    scanf("%f", &salary);
    fprintf(fptr, "Salary  = %.2f\n", salary);

    fclose(fptr);
}
```

You can add any information in the file, like we have added Name, Age and Salary for some employees, you can change the program as per your requirements.

You can even initialise a `for` loop, to add details of multiple employees to the file. All you have to do is, ask user for number of employees for which data has to be stored, run the `for` loop that many times and keep on adding the data to the file.

C Program to reverse the content of a File

studytonight.com/c/programs/files-and-streams/program-to-reverse-content-of-file

Below is a program to reverse the contents of a file.

Here is the C language tutorial explaining File Handling in C → File Handling in C

```
#include<stdio.h>
#include<errno.h>

/*
    to count the total number of characters
    inside the source file
*/
long count_characters(FILE *);

void main()
{
    int i;
    long cnt;
    char ch, ch1;
    FILE *fp1, *fp2;

    if (fp1 = fopen("File_1.txt", "r"))
    {
        printf("The FILE has been opened...\n");
        fp2 = fopen("File_2.txt", "w");
        cnt = count_characters(fp1);

        /*
            Make the pointer fp1 to point at the
            last character of the file
        */
        fseek(fp1, -1L, 2);
        printf("Number of characters to be copied %d\n", ftell(fp1));

        while (cnt)
        {
            ch = fgetc(fp1);
            fputc(ch, fp2);
            fseek(fp1, -2L, 1); // shifts the pointer to the previous character
            cnt--;
        }
        printf("\n**File copied successfully in reverse order**\n");
    }
    else
    {
        perror("Error occured\n");
    }
    fclose(fp1);
    fclose(fp2);
}

/*
    Count the total number of characters in the file
    that *f points to
*/
long count_characters(FILE *f)
{
    fseek(f, -1L, 2);
    /*
        returns the position of the
        last element of the file
```

```
    */
    long last_pos = ftell(f);
    last_pos++;
    return last_pos;
}
```

C Program to copy content of one File into another File

studytonight.com/c/programs/files-and-streams/program-copy-file-to-another-file

We already know how to open a file, read contents of a file and write into a file. So in this program, we will read from one file and simultaneously write into the other file, till we reach end of first file.

```
#include<stdio.h>
#include<stdio.h>

void main()
{
    /*
        File_1.txt is the file with content and,
        File_2.txt is the file in which content of File_1
        will be copied.
    */
    FILE *fp1, *fp2;
    char ch;
    int pos;

    if ((fp1 = fopen("File_1.txt", "r")) == NULL)
    {
        printf("\nFile cannot be opened.");
        return;
    }
    else
    {
        printf("\nFile opened for copy...\n ");
    }
    fp2 = fopen("File_2.txt", "w");
    fseek(fp1, 0L, SEEK_END);   // File pointer at end of file
    pos = ftell(fp1);
    fseek(fp1, 0L, SEEK_SET);   // File pointer set at start
    while (pos--)
    {
        ch = fgetc(fp1);    // Copying file character by character
        fputc(ch, fp2);
    }
    fcloseall();
}
```

Identify a Leap Year Program in C

studytonight.com/c/programs/important-concepts/identify-a-leap-year

Before we write the program to find a leap year in C language, let's understand how we can in general find which year is a leap year and then we will write a program for it.

What is a Leap Year?

For 1 rotation, Earth takes 24 hours. It takes approximately 365.25 days (365 days and 6 hours) for the Earth to complete one revolution around the sun. That is what we call a year. But since we only count 365 days in a year, we make up for it by adding those 6 hours for 4 years. We end up adding a day to a year every four years. This year is called a **Leap year**. In a leap year, February has 29 days instead of 28, so, in total, **it has 366 days**.

2017, 2018, 2019 are not leap years but 2020 is.

A year which is evenly divisible by 4 is said to be leap year. But this caused some errors because 1700, 1800, 1900, etc. are evenly divisible by 4 but not leap years.

To correct this mistake, in addition to being divisible by 4, if an year is divisible by 100, then it should also be divisible by 400, only then it will be a leap year.

Algorithm to find Leap Year

To check even divisibility of the number, we will use **modulo operator**(`%`). It results in **zero** if the number is evenly divisible by its quotient.

The algorithm for checking if the given year is a leap year or not is as follows:

1. Take the user input.
2. If the year is evenly divisible by 4, it maybe a leap year. Otherwise, it is not a leap year.
3. If the year is divisible by 4, check if the year is evenly divisible by 100 too. If it is divisible by 4 and not by 100, then it is a leap year, otherwise, if it's divisble by 100 also, then we need to do one more check.
4. If the year is divisble by both 4 and 100, we check if the year is evenly divisible by 400. If yes, then it is a leap year. Otherwise, it is not a leap year.

Program to identify whether the Input Year is a Leap Year or not

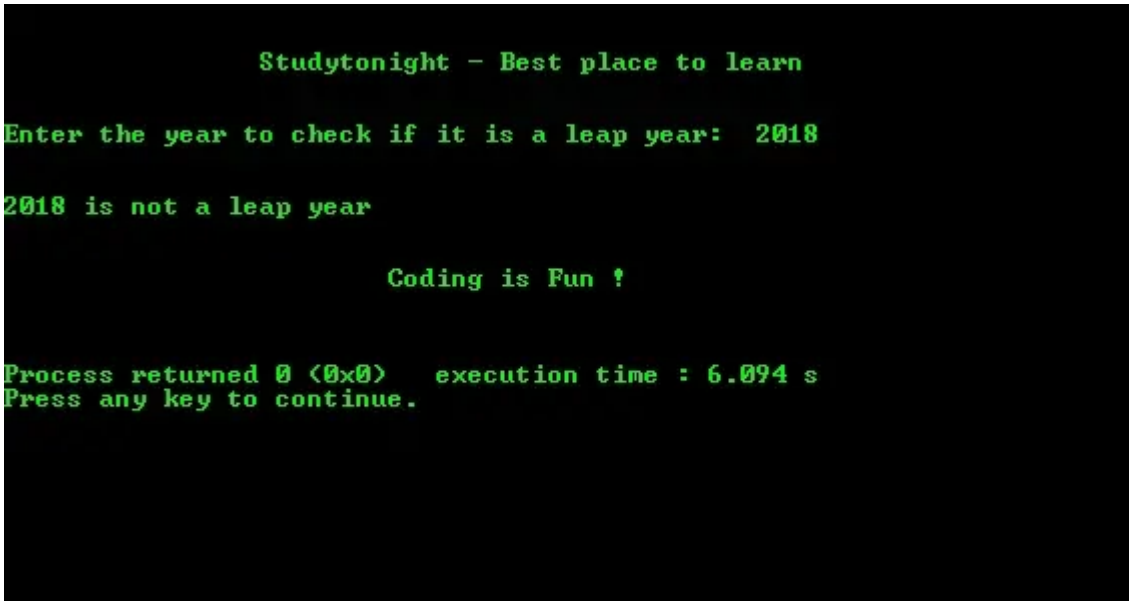
Now let's see the program to identify whether the input year is a leap year or not in C language.

```
#include<stdio.h>
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int year;
    printf("Enter the year to check if it is a leap year: ");
    scanf("%d", &year);

    if(year % 4 == 0){
        if(year % 100 == 0){
            if(year % 400 == 0)
                printf("\n\n%d is a leap year\n", year);
            else
                printf("\n\n%d is not a leap year\n", year);
        }
        else
            printf("\n\n%d is a leap year\n", year);
    }
    else
        printf("\n\n%d is not a leap year\n", year);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Program Output:



Conclusion

In this tutorial, we learned what a leap year is and how do we identify a Leap Year program in C. To learn more such interesting C Programs, follow our complete C tutorial.

C Program to find the Largest of Three Numbers

studytonight.com/c/programs/important-concepts/largest-of-three-numbers

Below is a program to find the largest amongst the three user input numbers.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    float a, b, c;

    printf("Enter 3 numbers:\n\n");
    scanf("%f%f%f", &a, &b, &c);

    if(a >= b && a >= c)
    {
        /*
            %.3f prints the floating number
            upto 3 decimal places
        */
        printf("\n\nLargest number = %.3f ", a);
    }
    else if(b >= a && b >= c)
    {
        printf("\n\nLargest number is = %.3f", b);
    }
    else
    {
        printf("\n\nLargest number is = %.3f", c);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter 3 numbers:
3.4
4.56
5.34
Largest number is = 5.340
Coding is Fun !
Process returned 0 (0x0)   execution time : 22.282 s
Press any key to continue.

C program to find the second Largest number among Three user input Numbers

studytonight.com/c/programs/important-concepts/second-largest-among-three-numbers

`.2lf` restricts the number till 2 decimal places

Below is a program to find the second largest number out of the three user input numbers using nested if-else loops:

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    double a, b, c;

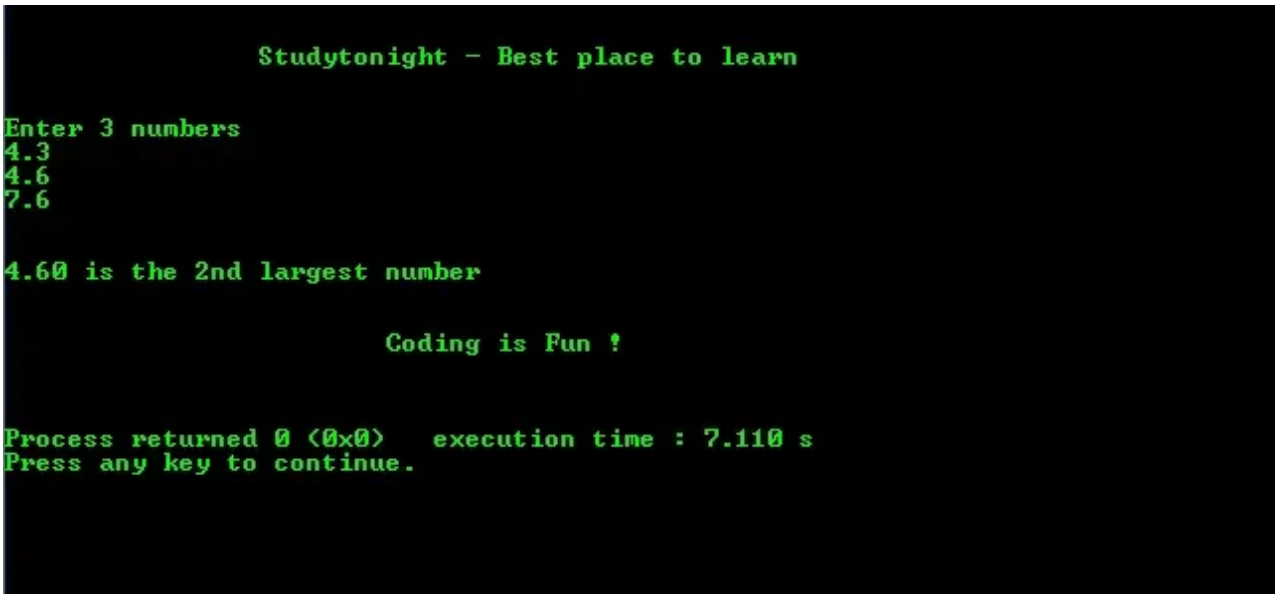
    printf("Enter 3 numbers:\n");
    scanf("%lf%lf%lf", &a, &b, &c);  //lf is a format specifier to take double as
input

    // a is the largest
    if(a >= b && a >= c)
    {
        if(b >= c)
        {
            /*
                .2lf restricts the number till
                2 decimal places
            */
            printf("\n\n%.2lf is the 2nd largest number\n", b);
        }
        else
        {
            printf("\n\n%.2lf is the 2nd largest number\n", c);
        }
    }
    else if(b >= a && b >= c)
    {
        if(a >= c)
        {
            printf("\n\n%.2lf is the 2nd largest number\n",a);
        }
        else
        {
            printf("\n\n%.2lf is the 2nd largest number\n",c);
        }
    }

    // c is the largest number of the three
    else if(a >= b)
    {
        printf("\n\n%.2lf is the 2nd largest number\n", a);
    else
    {
        printf("\n\n%.2lf is the 2nd largest number\n", b);
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter 3 numbers
4.3
4.6
7.6
4.60 is the 2nd largest number
Coding is Fun !
Process returned 0 (0x0) execution time : 7.110 s
Press any key to continue.

C Program to Add two user input numbers using Pointers

studytonight.com/c/programs/important-concepts/adding-two-numbers-using-pointers

Few important points to remember:

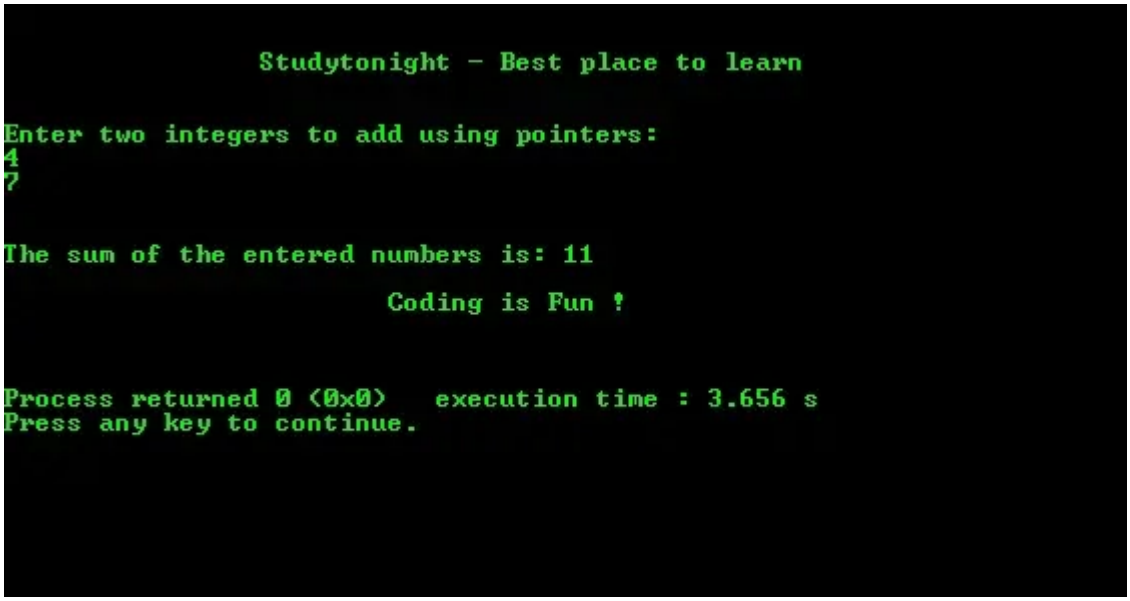
- `*` operator returns **'value at'** on the given address location.
- `&` operator returns **'address of'** a given value.

Below is a program adding two numbers using pointers.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int first, second, *p , *q, sum;
    printf("Enter two integers to add using pointers:\n");
    scanf("%d%d", &first, &second);
    p = &first;
    q = &second;
    sum = *p + *q;
    printf("\n\nThe sum of the entered numbers is: %d", sum);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C program to find Sum of First and Last Digits of a Number

studytonight.com/c/programs/important-concepts/sum-of-first-and-last-digit

Note: `%` or the **mod** operator is used to find the remainder of division operation.

Below is a program to find sum of first and last digits of the user input number.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int input, firstNumber, lastNumber;

    printf("Enter a number: ");
    scanf("%d", &input);

    /*
        gives remainder when the input value is divided by 10
        i.e. the digit at units place
    */
    lastNumber = input%10;

    firstNumber = input;

    while(firstNumber >= 10)
    {
        firstNumber /= 10;  // same as firstNumber = firstNumber/10
    }

    printf("\n\n\nAddition of first and last number is: %d + %d = %d\n",
firstNumber, lastNumber, firstNumber+lastNumber);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter a number : 5467
Addition of first and last number is : 5 + 7 = 12
Coding is Fun !
Process returned 0 (0x0) execution time : 3.313 s
Press any key to continue.

C Program to find the Area and Circumference of a Circle

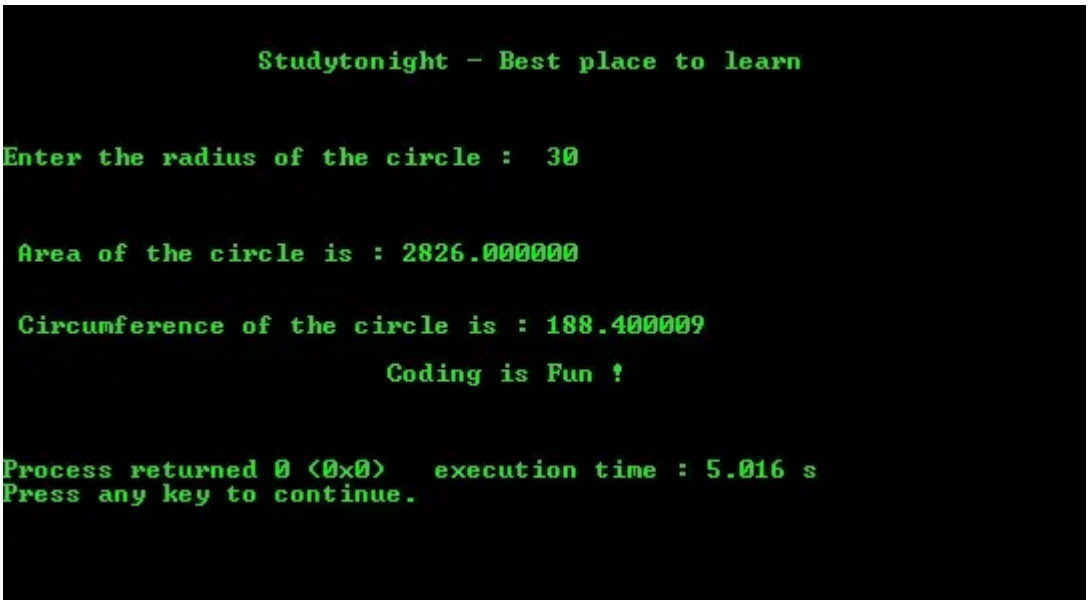
studytonight.com/c/programs/important-concepts/area-and-circumference-of-a-circle

Below is a program to find the area and circumference of a circle.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int rad;
    float PI = 3.14, area, ci;
    printf("\nEnter the radius of the circle: ");
    scanf("%d", &rad);
    area = PI*rad*rad;
    printf("\n\n\n Area of the circle is: %f ", area);
    ci = 2*PI*rad;
    printf("\n\n\n Circumference of the circle is: %f", ci);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C Program to find the Area of Triangle using Base and Height

studytonight.com/c/programs/important-concepts/area-of-triangle

Below is a program to find the area of triangle using base and height.

```
#include<stdio.h>

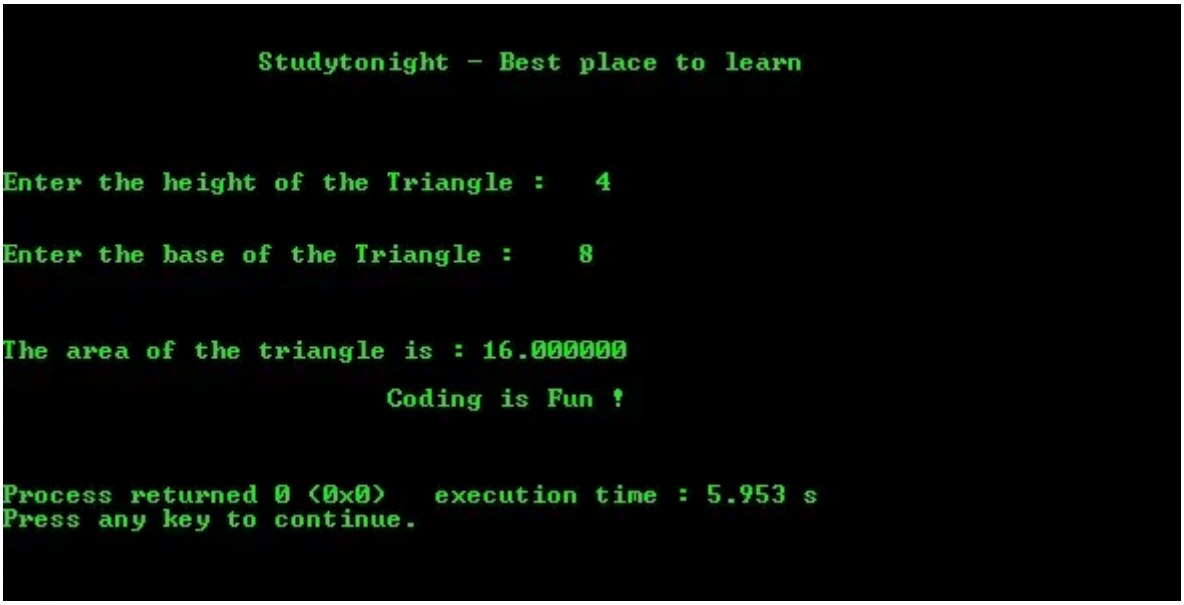
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int h, b;
    float area;
    printf("\n\nEnter the height of the Triangle: ");
    scanf("%d", &h);
    printf("\n\nEnter the base of the Triangle: ");
    scanf("%d", &b);

    /*
        Formula for the area of the triangle = (height x base)/2

        Also, typecasting denominator from int to float
        to get the output in float
    */
    area = (h*b)/(float)2;
    printf("\n\n\nThe area of the triangle is: %f", area);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to find the Area of Triangle using Heron's Formula

Below is a program to find the area of triangle using heron's formula.

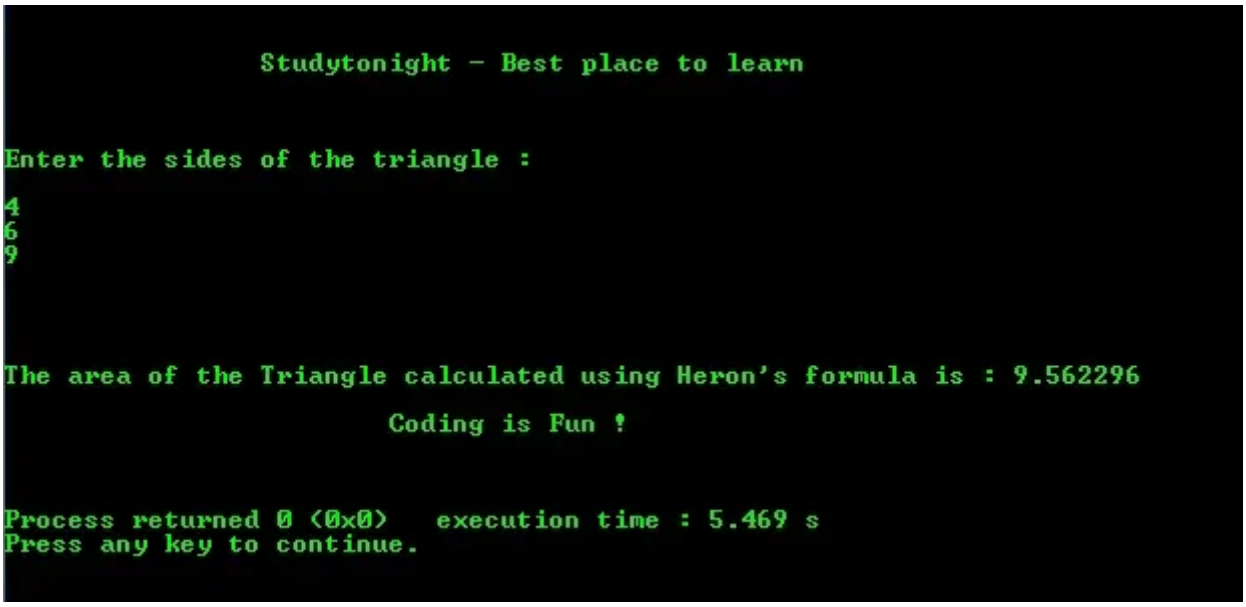
```
#include<stdio.h>
#include<math.h>  // to use sqrt() function

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    double a, b, c, area, s;
    printf("\nEnter the sides of the triangle:\n\n");
    scanf("%lf%lf%lf", &a, &b, &c);   // lf is format specifier for double input

    s = (a+b+c)/2;
    /*
        sqrt is a predefined system function that
        returns the square root of the input value
    */
    area = sqrt(s*(s-a)*(s-b)*(s-c));
    printf("\n\n\n\nThe area of the Triangle calculated using Heron's formula is:
%lf", area);

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C program to show basic Arithmetic Operations and role of Typecasting

studytonight.com/c/programs/important-concepts/basic-arithmetic-operations

Here we have 2 simple programs to showcase various basic arithmetic operations and how typecasting is handled by C language with and without explicitly using Typecasting in our program.

Arithmetic Operations without Typecasting

Below is a program to perform basic arithmetic operations without typecasting.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int a, b, add, subtract, multiply;
    float divide;

    printf("Enter two integers: \n");
    scanf("%d%d", &a, &b);

    add = a+b;
    subtract = a-b;
    multiply = a*b;
    divide = a/b;

    printf("\nAddition of the numbers = %d\n", add);
    printf("Subtraction of 2nd number from 1st = %d\n", subtract);
    printf("Multiplication of the numbers = %d\n", multiply);
    printf("Dividing 1st number from 2nd = %f\n", divide);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Arithmetic Operations with Typecasting

C Language do handle typecasting implicitly, still user can handle it in their programs too.

Below is a simple program to perform basic arithmetic operations with typecasting.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

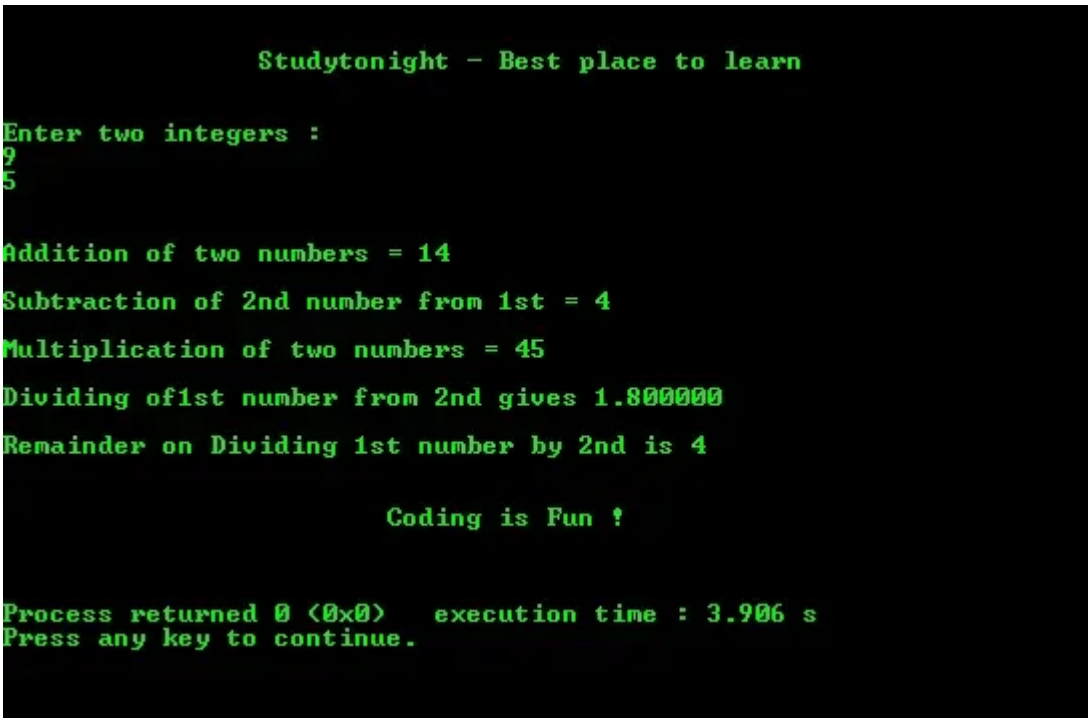
    int a, b, add, subtract, multiply, remainder;
    float divide;

    printf("Enter two integers: \n");
    scanf("%d%d", &a, &b);

    add = a+b;
    subtract = a-b;
    multiply = a*b;
    divide = a/(float)b;
    remainder = a%b;

    printf("\n\nAddition of the numbers = %d\n", add);
    printf("\nSubtraction of 2nd number from 1st = %d\n", subtract);
    printf("\nMultiplication of the numbers = %d\n", multiply);
    printf("\nDividing 1st number from 2nd = %f\n", divide);
    printf("\nRemainder on Dividing 1st number by 2nd is %d\n", remainder);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter two integers :
9
5
Addition of two numbers = 14
Subtraction of 2nd number from 1st = 4
Multiplication of two numbers = 45
Dividing of1st number from 2nd gives 1.800000
Remainder on Dividing 1st number by 2nd is 4
Coding is Fun !
Process returned 0 (0x0) execution time : 3.906 s
Press any key to continue.

C Programs to showcase the conversion between various Number Systems

studytonight.com/c/programs/important-concepts/conversion-between-number-system

Here we have multiple programs to showcase conversion between various Number systems like Binary to Decimal, Octal to Decimal, Decimal to Binary and we will even use **recursion** to help you understand how recursion can be used in such programs. So let's start.

Program to convert Binary to Decimal Equivalent

Below is a program to convert binary number to its decimal equivalent.

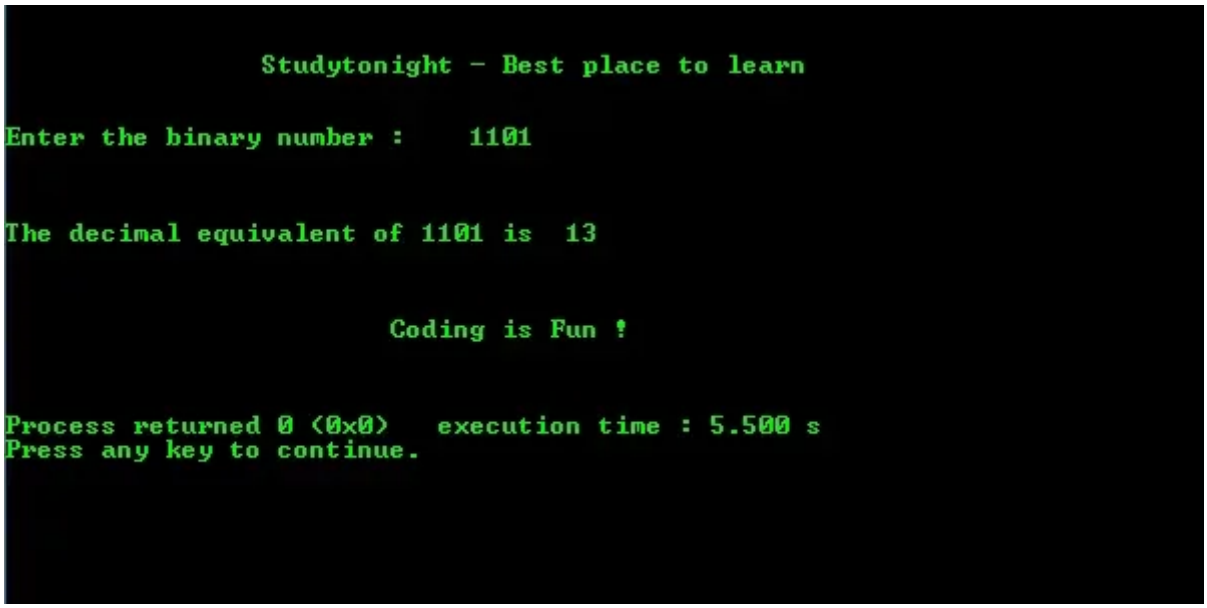
```
#include<stdio.h>
#include<math.h>

// Function prototype declaration
int binary_decimal(int n);

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n;
    char c;
    printf("Enter the binary number: ");
    scanf("%d", &n);
    printf("\n\n\nThe decimal equivalent of %d is  %d\n\n", n,
binary_decimal(n)); // function calling
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

// Definition of the function to convert binary to decimal.
int binary_decimal(int n)
{
    int decimal = 0, i = 0, rem;
    while(n != 0)
    {
        rem = n%10;   // gives the digit at the units place
        n = n/10; // gives the number excluding its units digit
        /*
            pow is a system defined function that takes
            two integers as input parameters
        */
        decimal += rem*pow(2, i++);
    }
    /*
        return the decimal equivalent of the input
        binary number to the function call
    */
    return decimal;
}
```

Output:



Program to convert Octal to Decimal Equivalent

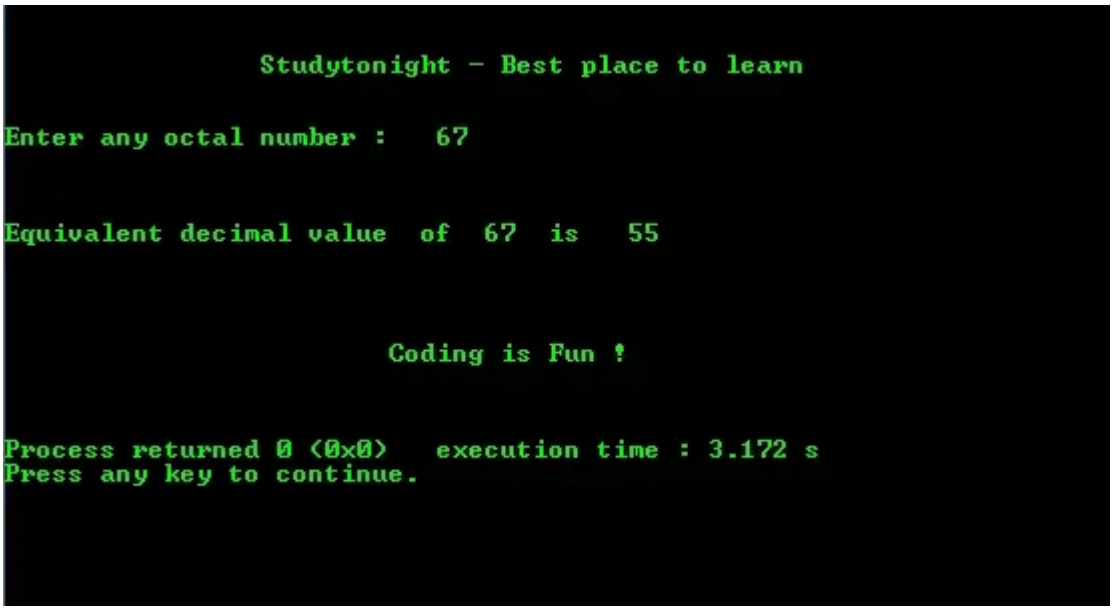
Below is a program to convert octal number to its decimal equivalent.

`%ld` is the format specifier to input a long value.

```
#include<stdio.h>
#include<stdio.h>
#include<math.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    long int octal, val, decimal = 0;
    int i = 0;
    printf("Enter any octal number: ");
    scanf("%ld", &val);
    octal = val;
    while(octal != 0)
    {
        /*
            i++ is post increment, where value is
            first assigned and then incremented
        */
      decimal += (octal % 10)*pow(8, i++);
      octal/=10;    // same as octal=octal/10
    }
    printf("\n\n\nEquivalent decimal value of %ld is %ld\n\n\n", val,
decimal);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to convert Decimal to Binary Equivalent

Below is a program to convert decimal number to its binary equivalent without recursion.

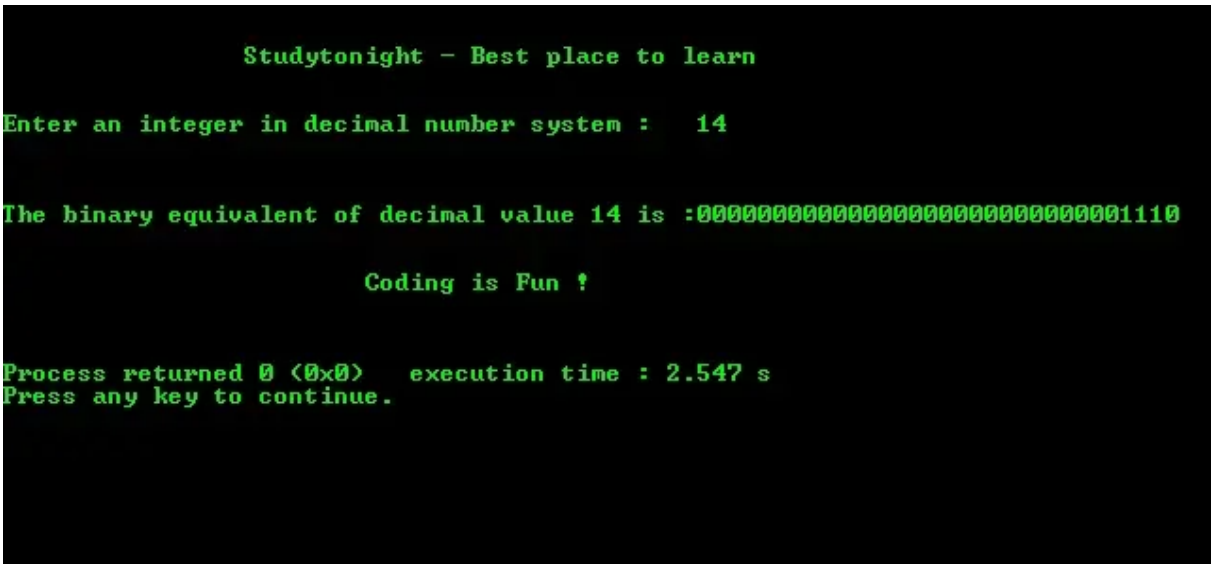
```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n,c,k;
    printf("Enter an integer in decimal number system: ");
    scanf("%d", &n);

    // In 31 bit format
    printf("\n\n\nThe binary equivalent of decimal value %d is:", n);

    for(c = 31; c >= 0; c--)
    {
        k = n>>c;
        /*
            num&1 = returns true if the last digit of num is 1
            else false
        */
        if(k&1)
            printf("1");
        else
            printf("0");
    }
    printf("\n");
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



Program to convert Decimal to Binary Equivalent using Recursion

Below is a program to convert decimal number to its binary equivalent with recursion:

```
#include<stdio.h>

//Function prototype declarations
void decimal_binary(int );
void F(int );
void reverse(int );

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int n;
    printf("\n\nEnter an integer in decimal number system: ");
    scanf("%d", &n);

    //In 31 bit format
    printf("\n\nThe binary equivalent of decimal value %d using decimal_binary
method is: ", n);

    decimal_binary(n);  // function call

    printf("\n\nThe binary equivalent of decimal value %d using F() method is:
", n);
    F(n);   // function call
    printf("\n\nThe Reverse of the binary representation of value %d is: ",
n);
    reverse(n); // function call
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

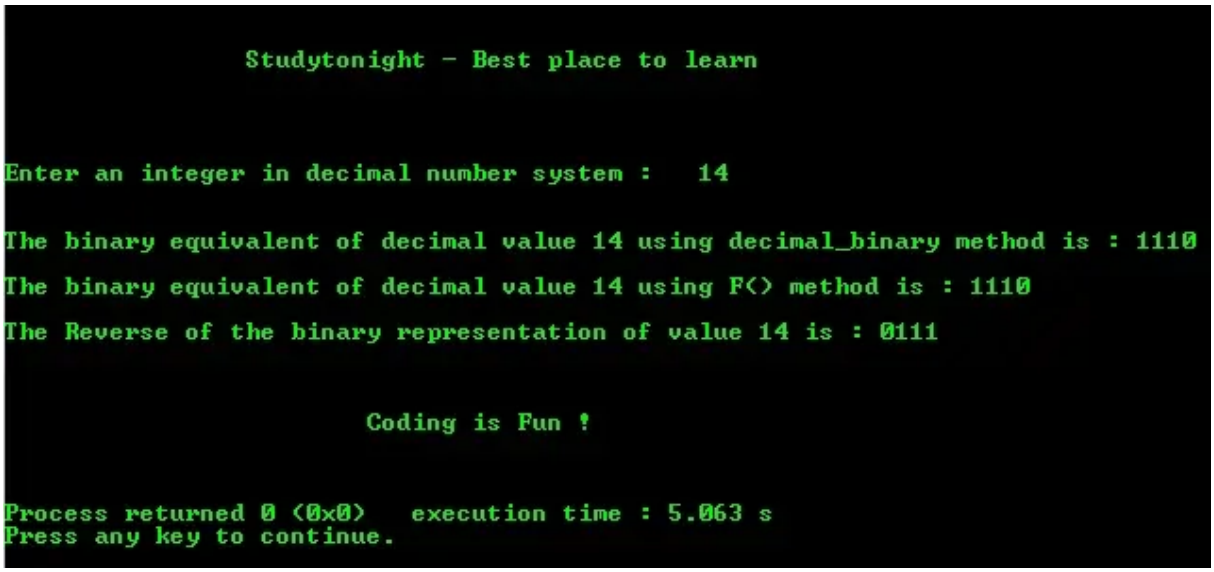
// function definition
void decimal_binary(int i)
{
    if(i <= 1)
    printf("%d", i);   // to print in up to down format
    else
    {
        decimal_binary(i/2);
        printf("%d", i%2);
    }
}

void F(int j)
{
    if(j/2)
    {
        F(j/2);
    }
    printf("%d", j%2);
}

void reverse(int k)
{
    if(k <= 1)
        printf("%d", k);
    else
```

```
    {
        printf("%d", k%2);
        F(k/2);
    }
    printf("\n\n");
}
```

Output:



C Program to convert Temparature in Celsius to Fahrenheit

studytonight.com/c/programs/important-concepts/celsius-to-fahrenheit

Below is a program for temperature conversion from Celsius to Fahrenheit. All we have to do is use the simple formula in our program, which is, if a temparature value is in Celsius, multiply it with `1.8` or `9/5` and add `32` to the result, this will give the equivalent Fahrenhiet value.

```
#include<stdio.h>

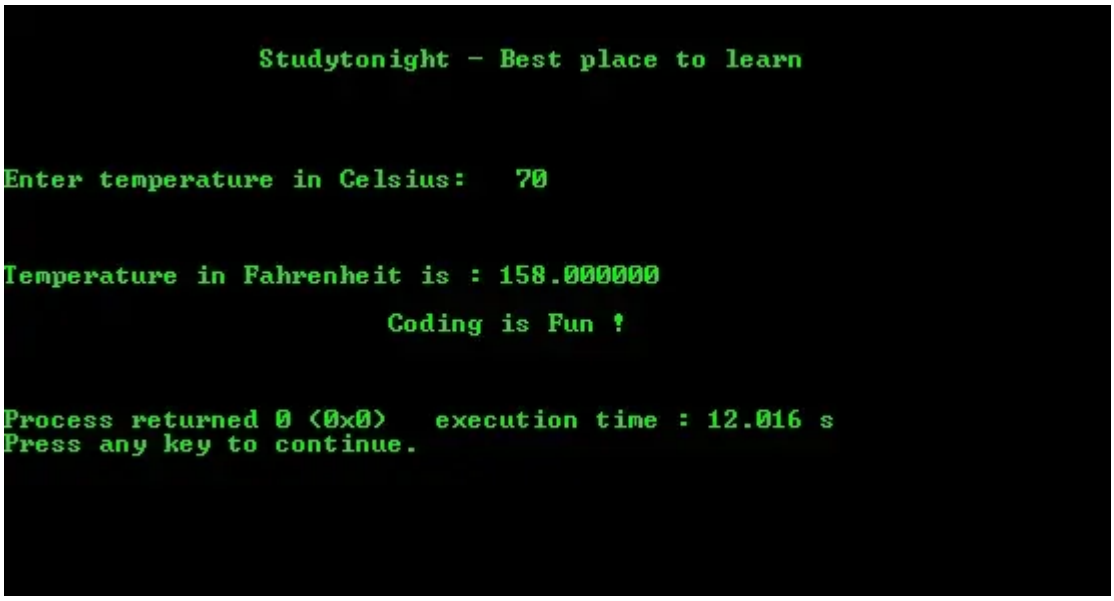
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    float celsius, fahrenheit;
    printf("\n\nEnter temperature in Celsius: ");
    scanf("%f", &celsius);

    fahrenheit = (1.8*celsius) + 32;

    printf("\n\n\nTemperature in Fahrenheit is: %f ", fahrenheit);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



To convert a temparature in Fahrenheit to Celsius, the formula will be, to subtract `32` from the value and then multiply it with `0.5556` or `5/9`

So all you have to do is replace the conversion formula in the avove program. Go ahead and try it out yourself.

C Program to calculate Simple Interest

studytonight.com/c/programs/important-concepts/simple-interest-program

Below is a program to calculate the Simple Interest for a given prinicipal amount, rate of interest and time duration.

`%7.3f` means the float value will be represented in 7 digits including 3 digits after decimal places.

```
#include<stdio.h>

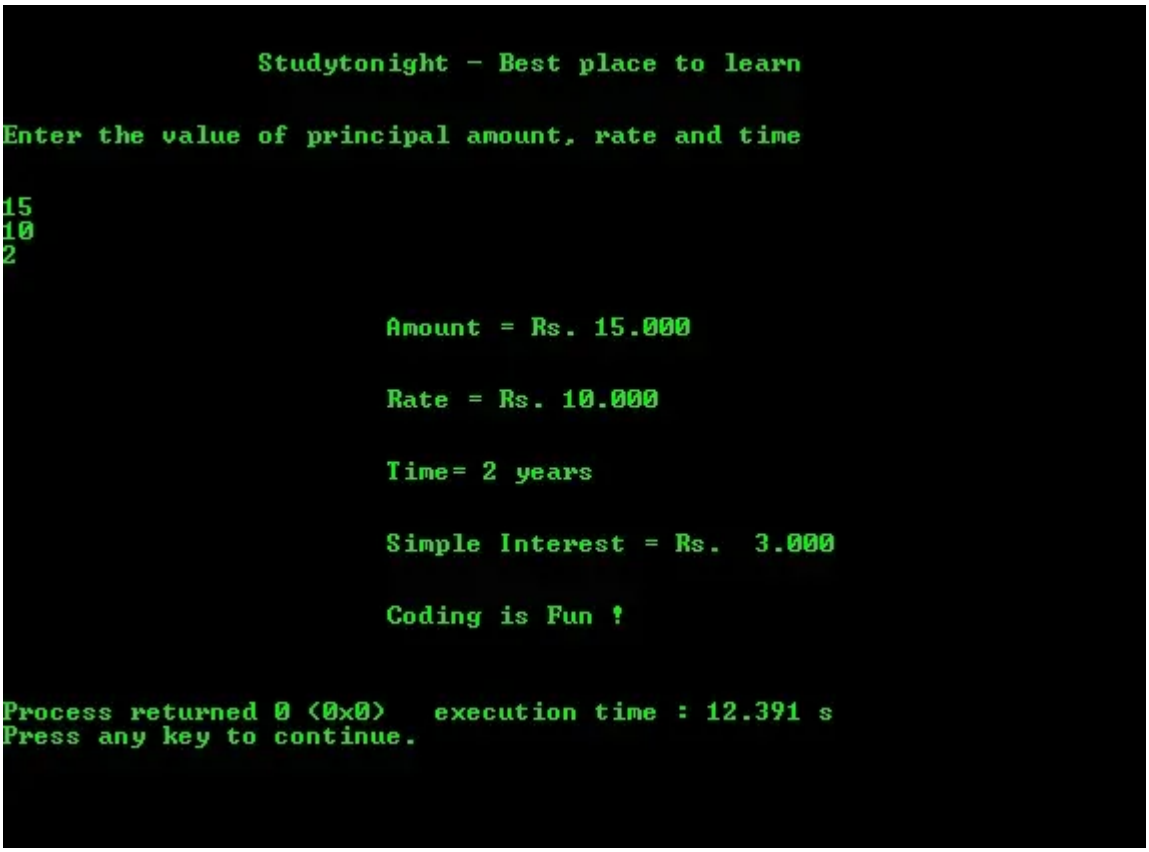
void main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    float principal_amt, rate, simple_interest;
    int time;
    printf("Enter the value of principal amount, rate and time\n\n\n");
    scanf("%f%f%d", &principal_amt, &rate, &time);

    // considering rate is in percentage
    simple_interest = (principal_amt*rate*time)/100.0;

    // usually used to align text in form of columns in table
    printf("\n\n\t\t\tAmount = Rs.%7.3f\n ", principal_amt);

    printf("\n\n\t\t\tRate = Rs.%7.3f\n ", rate);
    printf("\n\n\t\t\tTime= %d years \n", time);
    printf("\n\n\t\t\tSimple Interest = Rs.%7.3f\n ", simple_interest);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter the value of principal amount, rate and time
15
10
2
Amount = Rs. 15.000
Rate = Rs. 10.000
Time= 2 years
Simple Interest = Rs. 3.000
Coding is Fun !
Process returned 0 (0x0) execution time : 12.391 s
Press any key to continue.

C Program to find GCD of N Numbers

studytonight.com/c/programs/important-concepts/greatest-common-divisor-program

Below is a program to find GCD of `N` user input numbers.

```
 #include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int x, y =- 1;
    printf("Enter numbers. To exit enter 0\n");
    while(1)    // infinite loop to take input
    {
        scanf("%d", &x);
        if(x < 1)
            break;
        else if(y ==- 1)    // only 1 number entered, its GCD is itself
            y = x;
        else if(x < y)
            y = gcd(x, y);
        else
            y = gcd(y, x);
    }
    printf("\n\n\nGCD of all the entered number is: %d", y);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

// GCD of 2 numbers is calculated at a time
int gcd(int a, int b)
{
    int i;
    /*
        a is the smallest of the two numbers
        of which GCD is to be calculated
    */
    for(i = a; i >= 1; i--)
    {
        // Greatest number that divides both the numbers
        if(a%i == 0 && b%i == 0)
            break;  // exits the loop
    }
    return i;
}
```

Output:



C Program to find GCD of Two Numbers Using Functions

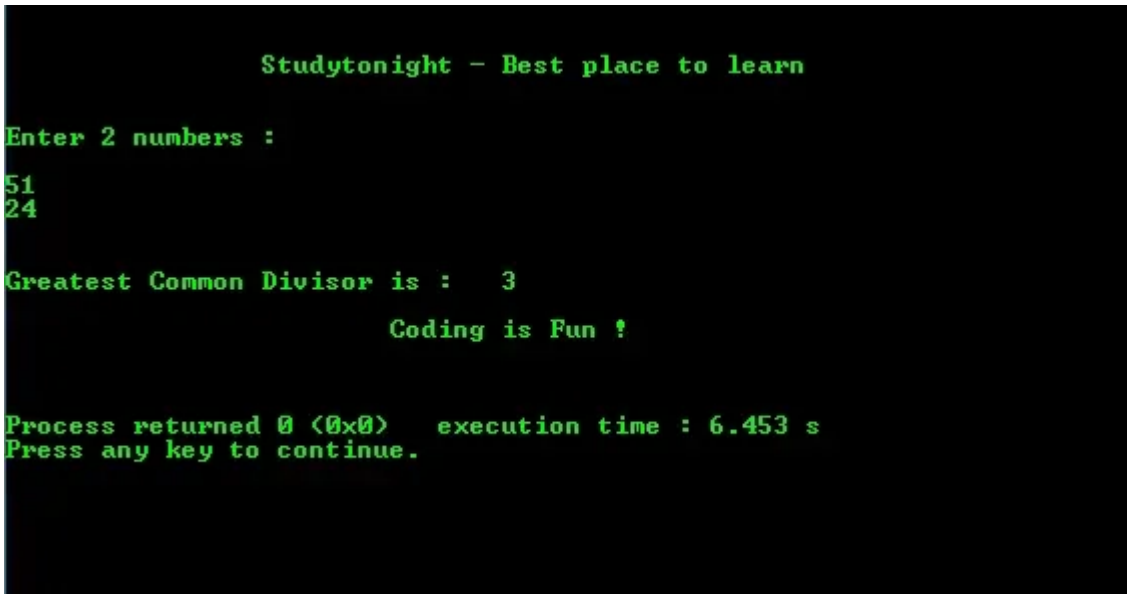
Below is a program to find GCD of two numbers using a function.

```
 #include<stdio.h>

int gcd(int c, int d)   // function definition
{
    if(d == 0)
        return c;
    else
        return gcd(d, c%d);
}

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int a, b;
    printf("Enter 2 numbers: \n\n");
    scanf("%d%d", &a, &b);
    printf("\n\nGreatest Common Divisor is: %d", gcd(a, b)); // function calling
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter 2 numbers :
51
24
Greatest Common Divisor is : 3
Coding is Fun !
Process returned 0 (0x0) execution time : 6.453 s
Press any key to continue.

C Program to Find Roots of Quadratic Equation

studytonight.com/c/programs/important-concepts/finding-roots-of-quadratic-equation

Below is a program to find roots of a quadratic equation.

`%.nf` is used to limit the value of a decimal value to `n` decimal places.

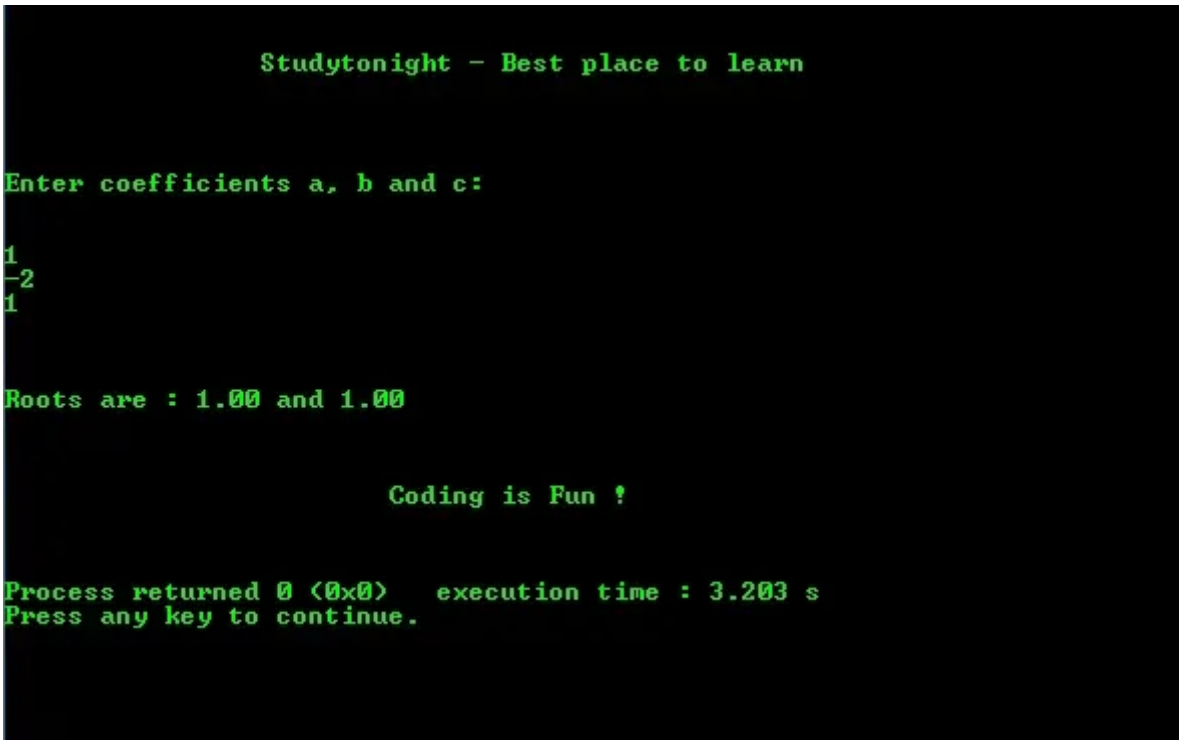
```
#include<stdio.h>
#include<math.h>  // This is needed to use sqrt() function

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    float a, b, c, determinant, r1, r2, real, imag;
    printf("\n\nEnter coefficients a, b and c: \n\n\n");
    scanf("%f%f%f", &a, &b, &c);

    /*
        mathematical formula to know the
        nature of the roots
    */
    determinant == b*b - 4*a*c;

    if(determinant > 0)    // both roots are real
    {
        r1 = (-b + sqrt(determinant))/2*a;  // Brackets are important
        r2 = (-b - sqrt(determinant))/2*a;
        printf("\n\n\nRoots are: %.2f and %.2f ", r1, r2);
    }
    else if(determinant == 0)   // both roots are real and equal
    {
        r1 = r2 = -b/(2*a); // brackets are important
        printf("\n\n\nRoots are: %.2f and %.2f ", r1, r2);
    }
    /*
        Determinant < 0 - both roots are imaginary of the
        form real + i*imaginary
    */
    else
    {
        real = -b/(2*a);
        imag = sqrt(-determinant)/(2*a);
        printf("\n\n\nRoots are %.2f + i%.2f and %.2f - i%.2f ", real, imag, real,
imag);
    }
    printf("\n\n\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Enter coefficients a, b and c:
1
-2
1
Roots are : 1.00 and 1.00
Coding is Fun !
Process returned 0 (0x0)   execution time : 3.203 s
Press any key to continue.

C program to Check for a Perfect Square

studytonight.com/c/programs/important-concepts/checking-for-perfect-square

Below is a program to check whether the user input number is a perfect square or not.

```
#include<stdio.h>
int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    // variable declaration
    int i, number;

    // take user input
    printf("Enter a number: ");
    scanf("%d", &number);

    // loop to check number is perfect square or not
    for(i = 0; i <= number; i++)
    {
        if(number == i*i)
        {
            printf("\n\n\n\t\t\t%d is a perfect square\n\n\n", number);

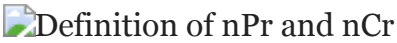
            printf("\n\n\t\t\tCoding is Fun !\n\n\n");
            return 0;   // same as using break in this case to end the program
        }
    }
    printf("\n\n\n\t\t\t%d is not a perfect square\n", number);
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:



C Program to Calculate Permutation (nPr) and Combination (nCr)

studytonight.com/c/programs/important-concepts/calculate-nPr-and-nCr



Below is a program to calculate `nPr` and `nCr`.

Global declaration scope is within the complete program.

Local declarations scope is limited to the function only.

```c
#include<stdio.h>

// function prototype declarations
long factorial(int);
long find_npr(int, int);
long find_ncr(int, int);

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    int n, r;
    long npr, ncr;

    printf("Enter the value of n and r respectively: \n\n");
    scanf("%d%d", &n, &r);

    // function calls
    npr = find_npr(n, r);
    ncr = find_ncr(n, r);

    printf("\n\n\n\t\t%dC%d = %ld\n", n, r, ncr);
    printf("\n\n\t\t%dP%d = %ld\n", n, r, npr);

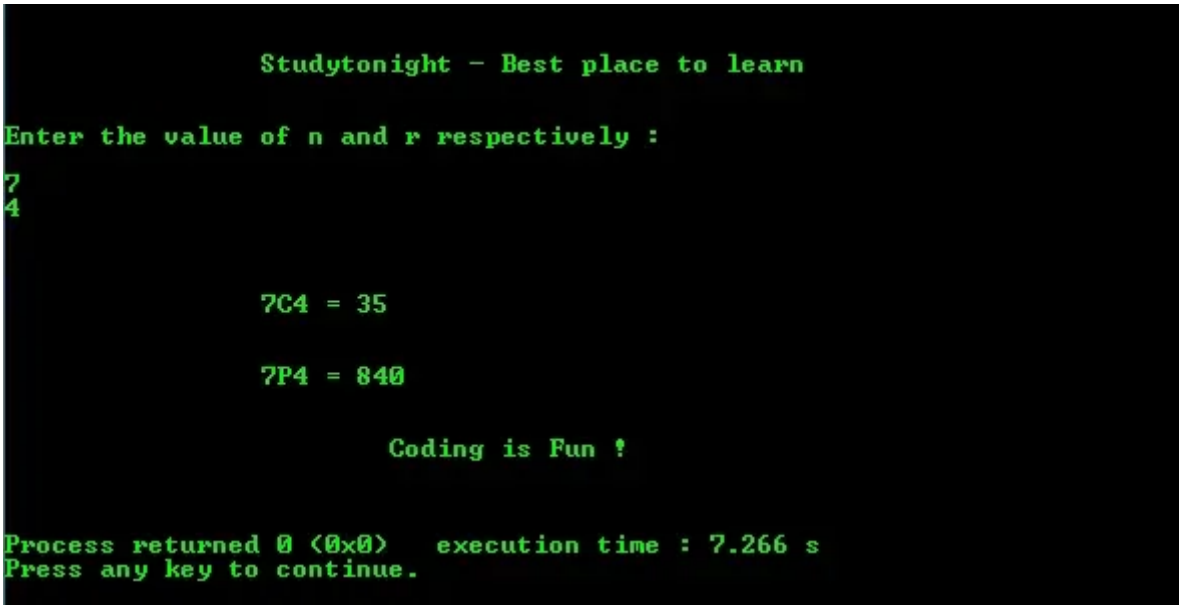
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}

/*
    function definition for nCr
*/
long find_ncr(int a, int b)
{
    return (factorial(a)/(factorial(b)*factorial(a-b)));
}

/*
    function definition for nPr
*/
long find_npr(int a, int b)
{
    return (factorial(a)/factorial(a-b));
}

/*
    recursive function definition for finding
    factorial of a number
*/
long factorial(int c)
{
    if(c == 1 || c == 0)
        return 1;
    else
        return c*factorial(c-1);
}
```

Output:


Studytonight - Best place to learn
Enter the value of n and r respectively :
7
4
7C4 = 35
7P4 = 840
Coding is Fun !
Process returned 0 (0x0) execution time : 7.266 s
Press any key to continue.

C Program to shutdown Windows/Linux Shutdown Machine

studytonight.com/c/programs/misc/windows-shutdown-program

This program turns off i.e shutdown your computer system. System function of `stdlib.h` is used to run an executable file shutdown.exe which is present in `C:\WINDOWS\system32` folder in Windows 7 and XP.

Below is a program to shutdown Windows 7.

```
#include<stdio.h>
#include<stdlib.h>    // to use system() method

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char ch;

    printf("Do you want to shutdown your pc now (y/n)?");
    scanf("%c", &ch);

    if(ch == 'y'|| ch == 'Y')
    {   /*
            /s is used to order the compiler
            to shutdown the PC
        */
        system("C:\\WINDOWS\\System32\\shutdown /s");
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

You can use various options while executing **shutdown.exe**, for example you can use `/t` option to specify number of seconds after which the shutdown occurs.

- **Syntax:** `"shutdown /s /t x";` here x is the number of seconds after which shutdown will occur.
- **Example:** By default shutdown occurs after 30 seconds. To shutdown immediately you can write `"shutdown /s /t 0"`

If you wish to **restart** your computer then you can use `"shutdown /r"`.

Program to Shutdown Windows XP Machine

Below is a program to shutdown Windows XP.

```
#include<stdio.h>
#include<stdlib.h>    // to use system() function

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char ch;

    printf("Do you want to shutdown the PC- (y/n) ?\n");
    scanf("%c", &ch);

    if(ch == 'y' || ch == 'Y')
    {
        system("C:\\WINDOWS\\System32\\shutdown -s");
    }

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

- To shutdown immediately use `"C:\\WINDOWS\\System32\\shutdown -s -t 0"`. To restart use `"-r"` instead of `"-s"`.
- For better understanding go through the program for shutting down Windows 7, in which there is a detailed explanation of using **t** and `r` instead of `s`.

> ***Note:*** *A **'-'** performs the same function in Windows XP as that performed by '/' incase of Windows 7.*

Program to Shutdown Linux OS

Below is a program to shutdown Linux operating system.

```
#include<stdio.h>
#include<stdlib.h>    // to use system() function

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    char ch;

    printf("Do you want to shutdown your pc now(y/n)?");
    scanf("%c", &ch);

    if(ch == 'y' || ch == 'Y')
    system("shutdown -P now");

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

- You need to be logged in as user for above program to execute otherwise you will get the message shutdown: **"Need to be root"**.
- `'-P'` option specifies you want to **power off** your machine.
- You can specify minutes as: `shutdown -P "number of minutes"`
- For more help or options type at terminal: `man shutdown`

C Program without a main() function

studytonight.com/c/programs/misc/program-without-main-function

The `main()` function is the starting point from where the program execution starts as per programmer's perspective. It usually has `int` or `void` return type.

It is followed by opening and closing curly braces which contain the program body. We can also pass parameters to it according to our requirement.

Can I execute a Program without `main()` method?

Well, Yes, you can.

The program execution starts from the `main()` function as per the programmer's perspective but it is not entirely true. Before the `main()` function is executed, several other functions are called which prepare the environment variables for the program's execution, setup arguments, etc.

One such calls is made to the function `_start()`. The `_start()` function prepares the input arguments for another function `_libc_start_main()` which then calls the `main()` function. So, if we **override** the `_start()` function, we can have any custom function from which our program will start execution. It does not have to be named `main()`.

To do so, we will use **-nostartfiles** while compiling our program.

Below is a program without `main()`. Save it in a file, say **program.c** file.

```
#include<stdio.h>
#include<stdlib.h>
int fun() // our custom main function
{
    printf("Welcome to studytonight!\n");
    return 0;
}
void _start()
{
    int x = fun(); //calling custom main function
    exit(x);
}
```

Compile it using this command,

```
gcc -nostartfiles -o program program.c
```

And then run it like,

```
./program
```

Welcome to studytonight!

Hiding the main() function in C Program

Below is a program without the `main()` function.

In the below program, `main()` function is there, but hidden using the preprocessors.

As you can see in the second line, `#define decode()` function is used, which holds a character combination of `m,a,i,n` and is followed by `##m##a##i##n`.

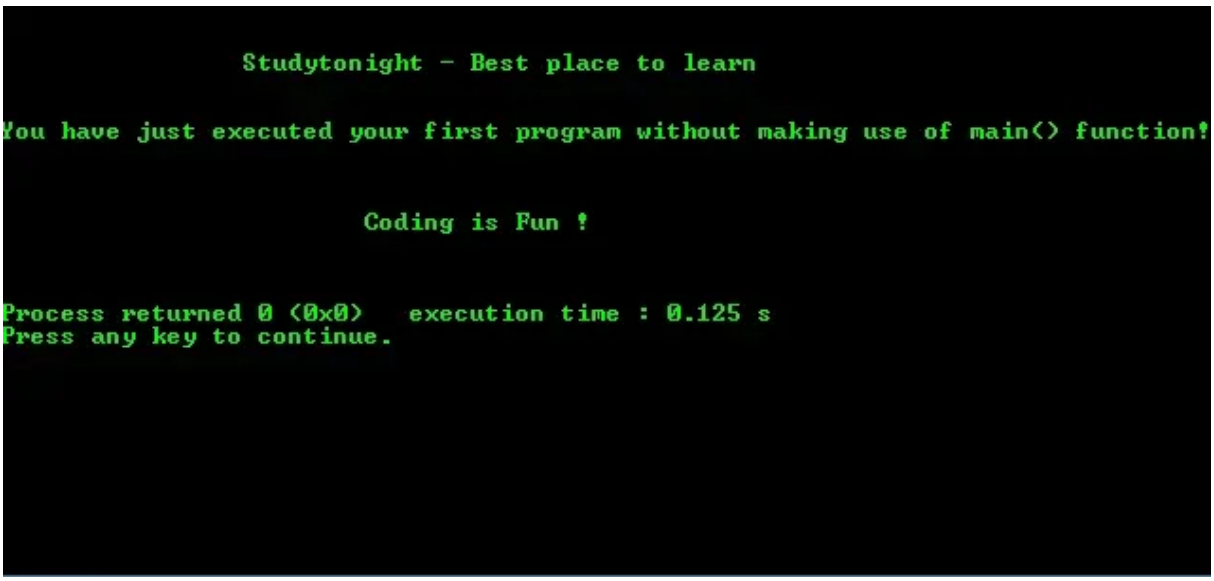
Here `##` operator is used to merge the characters in the order mentioned using `##`, which is `main`

In the 3rd line `#define go decode(m,a,i,n)` as we have specified the characters in same order, the `decode` function will assign value `main` for `go`.

```
#include<stdio.h>
//Need to include the following statements in same manner
#define decode(m,a,i,n) m##a##i##n
#define go decode(m,a,i,n)

int go()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    printf("You have just executed your first program without making use of
main() function!\n");
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Program Output:



We can use different words and combination here, like

```
#define decode(s,t,u,m,p,e,d) m##s##u##t
#define go decode(a,n,i,m,a,t,e)
```

Here as per the first line, 4th, 1st, 3rd and 2nd chcracters have to be formed into a word. When the same combination is taken out from the word **animate** it makes **main**.

Using macro to define main

There is one more way to do this. Yes, using macro. Well in this technique, we will just create a facade, or in simpler words, we can say an illusion, where we are actually using main, but in the program, we use a different name for it.

```
#include<stdio.h>
#define go main
int go(void)
{
    printf("Welcome to Studytonight");
    return 0;
}
```

This is is the simplest technique, where all we have done is provided our `main()` function with a different name, which is set the `main` before the program is executed.

Using Token-Pasting operator

This is another way, again to just create an illusion. If someone else is reading this program, they will confused.

```
#include<stdio.h>
#define go m##a##i##n
int go(void)
{
    printf("Welcome to Studytonight");
    return 0;
}
```

Welcome to studytonight

C Program to create a Menu Driven software using Switch Case

studytonight.com/c/programs/misc/menu-driven-program

Below is a menu driven program using switch case.

`unsigned` is doubled the size of signed as it only considers positive values.

`%lu` is the format specifier for unsigned integer.

A number divisible by an integer other than 1 and itself is called a **composite number**.

```
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");
    int choice, num, i;
    unsigned long int fact;

    while(1)
    {
        printf("1. Factorial \n");
        printf("2. Prime\n");
        printf("3. Odd\\Even\n");
        printf("4. Exit\n\n\n");
        printf("Enter your choice :  ");
        scanf("%d",&choice);

        switch(choice)
        {
            case 1:
                printf("Enter number:\n");
                scanf("%d", &num);
                fact = 1;
                for(i = 1; i <= num; i++)
                {
                    fact = fact*i;
                }
                printf("\n\nFactorial value of %d is = %lu\n\n\n\n",num,fact);
                break;

            case 2:
                printf("Enter number:\n");
                scanf("%d", &num);
                if(num == 1)
                printf("\n1 is neither prime nor composite\n\n");
                for(i = 2; i < num; i++)
                {
                    if(num%i == 0)
                    {
                        printf("\n%d is not a prime number\n\n", num);
                        break;
                    }

                }
                /*
                    Not divisible by any number other
                    than 1 and itself
                */
                if(i == num)
                {
                    printf("\n\n%d is a Prime number\n\n", num);
                    break;
                }

            case 3:
                printf("Enter number:\n");
```

```
                scanf("%d", &num);

                if(num%2 == 0) // 0 is considered to be an even number
                    printf("\n\n%d is an Even number\n\n",num);
                else
                    printf("\n\n%d is an Odd number\n\n",num);
                break;

            case 4:
                printf("\n\n\t\t\tCoding is Fun !\n\n\n");
                exit(0);    // terminates the complete program execution
        }
    }
    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
1. Factorial
2. Prime
3. Odd\Even
4. Exit
Enter your choice : 1
Enter number : 5
Factorial value of 5 is = 120
1. Factorial
2. Prime
3. Odd\Even
4. Exit
Enter your choice : 2
Enter number:
1
1 is neither prime nor composite
1. Factorial
2. Prime
3. Odd\Even
4. Exit
Enter your choice : 3
Enter number:
7
7 is an Odd number
1. Factorial
2. Prime
3. Odd\Even
4. Exit
Enter your choice : 4
Coding is Fun !
Process returned 0 (0x0) execution time : 17.563 s

C program to Change the Text Background Color

studytonight.com/c/programs/misc/changing-text-background-color

Few important points regarding this program are:

- `SetConsoleTextAttribute` : Sets the attributes of characters written to the console screen buffer by the **WriteFile** or **WriteConsole** function, or echoed by the **ReadFile** or **ReadConsole** function.
- This function affects text written after the function call.
- **Syntax:**

```
BOOL WINAPI SetConsoleTextAttribute(_In_ HANDLE hConsoleOutput , _In_ WORD
wAttributes);
```

> *For more Details, you may visit the Official Documentation here : https://docs.microsoft.com/en-us/windows/console/setconsoletextattribute*

Below is the program for changing text background color.

```
#include<windows.h>
#include<stdio.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    //BACKGROUND_RED| BACKGROUND_GREEN| BACKGROUND_BLUE| BACKGROUND_INTENSITY

SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),BACKGROUND_BLUE|BACKGROUND_

    printf("\n\nStudytonight just showed you how to put colors to your code!!");

    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),BACKGROUND_GREEN);
    printf("\n\nIsn't this Awesome?");


SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),BACKGROUND_INTENSITY|BACKGR

    printf("\n\nYou just did something that only 1 out of 10 coders are familiar
of :)\n");


SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),BACKGROUND_GREEN|BACKGROUND

    printf("\n\nYou are doing great!!");


SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),BACKGROUND_BLUE|BACKGROUND_

    printf("\n\nThe best is yet to come!");


SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE),BACKGROUND_GREEN|BACKGROUND

    printf("\n\nWhat are you waiting for?? Just play with it!!");

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:


Studytonight - Best place to learn
Studytonight just showed you how to put colors to your code!!
Isn't this Awesome?
You just did something that only 1 out of 10 coders are familiar of :)
You are doing great!!
The best is yet to come!
What are you waiting for?? Just play with it!!
Coding is Fun !
Process returned 0 (0x0) execution time : 0.219 s
Press any key to continue.

C Program to Display the current Date and Time

studytonight.com/c/programs/misc/display-current-date-and-time

The C library function `char *ctime(const time_t *timer)` returns a string representing the localtime based on the argument **timer**. The returned string has the following format: **Www Mmm dd hh:mm:ss yyyy**. Here **Www** is the weekday, **Mmm** the month in letters, **dd** the day of the month, **hh:mm:ss** the time and **yyyy** the year.

Below is the program to display current date and time.

`#include<time.h>` is used for `time` and `ctime` function and `time_t` datatype.

```
#include<stdio.h>
#include<time.h>

int main()
{
    printf("\n\n\t\tStudytonight - Best place to learn\n\n\n");

    time_t t;   // not a primitive datatype
    time(&t);

    printf("\nThis program has been writeen at (date and time): %s", ctime(&t));

    printf("\n\n\t\t\tCoding is Fun !\n\n\n");
    return 0;
}
```

Output:

